

Universidad de Sevilla

Máster en lógica, computación e inteligencia artificial

Sistemas conversacionales basados en repositorios

Nerea Almenta Camacho

Tutorizado por:
José Francisco Quesada Moreno

4 julio 2025

Índice general

1. Introducción **7**

2. Contexto **9**

2.1. Sistemas conversacionales 9

2.1.1. Lenguaje natural 10

2.1.2. Tipos de interacciones 11

Diálogos iniciados por el usuario 11

2.1.3. Diálogos dirigidos por el sistema. 13

Diálogos pro-activos 13

Diálogos instruccionales 13

Diálogos slot-filling 13

2.1.4. Diálogos multi-turno de dominio abierto 15

2.2. Tipos de sistemas de diálogo 16

2.2.1. Basado en reglas 16

2.2.2. Basado en datos estadísticos. 18

Corpus-based o example-based. 18

Derivado por interacción. 19

2.2.3. Sistemas end-to-end de diálogo neuronales 21

Seq2Seq 21

DNNs 22

Word embedding 23

Redes codificador-decodificador 24

2.3. Pre-trained Language Models (PLM) 25

2.4. Large Language Models (LLM) 26

2.5. Problemas de los LLM 27

2.6. RAG 28

2.6.1. Naive RAG 28

Indexación 28

Retrieval 31

Generation 31

2.6.2. Advanced RAG 31

Proceso pre-entrenado 31

Proceso Post-Retrieval 31

2.6.3. Modular RAG 32

2.6.4. Componentes de un sistema RAG 32

Retrieve 32

Augmentation 32

Generate 33

2.6.5. Aspectos dentro de cada componente . . . 33
Retrieval . . . 33
Augmentation . . . 35
Generation . . . 35
2.7. ¿Qué aporta el RAG a los LLM? . . . 36
2.8. ¿Qué problemas tiene el RAG? . . . 37
2.8.1. Privacidad . . . 37
2.8.2. Otros puntos de fallo . . . 38
2.9. Evaluación del RAG . . . 40
2.9.1. Recuperación . . . 40
2.9.2. Generación . . . 41
2.9.3. El sistema completo . . . 41
2.9.4. LLM as a Judge . . . 43
Modo de uso . . . 43
2.9.5. Limitaciones de los LLM as a judge . . . 44
2.9.6. Estrategias para su mejora . . . 45
Diseño del prompt . . . 45
2.9.7. Evaluación de los LLM as a judge . . . 46

3. Descripción del problema 47
3.1. Problemáticas . . . 49
3.1.1. Tipos de datos . . . 49
Eficiencia . . . 49
Tipo . . . 50
3.1.2. Información privada . . . 50
3.1.3. Vectorización . . . 50
Tokenización . . . 51
Chunks . . . 51
Embedding . . . 51
3.1.4. Recuperación . . . 51
Similitud . . . 52
Top K . . . 52
3.1.5. Generación . . . 53
3.1.6. Otros aspectos generales . . . 53
Extranjeros . . . 54
Fórmulas de cortesía . . . 55
Preguntas fuera de dominio . . . 55
Otros elementos . . . 56
3.1.7. Evaluación . . . 56
Información . . . 56
Módulos . . . 57

4. Solución del problema 59
4.1. Elementos técnicos . . . 59
4.1.1. Modelo embedding . . . 60
4.1.2. Modelo generativo . . . 61
4.2. Implementación . . . 62
4.2.1. Agente Decisor . . . 66
4.2.2. Traducción . . . 67
4.2.3. Prevención . . . 70

4.2.4. Fórmula cortesía . . . 71
4.2.5. Re-explicación . . . 71
4.2.6. Respuesta Genérica . . . 71
4.2.7. RAG . . . 72
Embedding . . . 72
Resumen . . . 72
Retrieval . . . 73
Re-ranker . . . 73
Clarificador . . . 74
Generation . . . 74
4.3. Módulos no usados . . . 75
4.3.1. Expansión de la pregunta . . . 75
4.4. Evaluación . . . 75
4.4.1. Traducción . . . 76
4.4.2. Fórmula cortesía . . . 78
4.4.3. Re-explicación . . . 80
4.4.4. Respuesta Genérica . . . 82
4.4.5. RAG . . . 84
4.4.6. Latencia . . . 86
4.4.7. Entrada en módulos . . . 87
4.5. Ejemplos de uso . . . 89
4.6. Flujo del ejemplo . . . 91

5. Conclusión **93**

Capítulo 1

Introducción

La falta de información siempre ha supuesto un gran problema para el ser humano. Así, en la era de la información, nos encontramos con que estamos más desinformados que nunca. Para tratar de mitigar este problema en algunos ámbitos de nuestra vida, hemos desarrollado a lo largo de este trabajo fin de máster un sistema conversacional basado en repositorios. Este, se ha centrado en responder preguntas planteadas sobre doctorados en la Universidad de Sevilla, llevando a cabo esta interacción con el usuario mediante un sistema modularizado. Este sistema incluye módulos de traducción, que permiten la interacción no solo con usuarios hispanohablantes, sino también con aquellos que hablasen otros idiomas. Asimismo, acepta también interacciones en forma de fórmulas de cortesía, como saludos o agradecimientos, permitiendo así una mejora de la experiencia del usuario. Además de estos y otros muchos módulos complementarios para tareas menores, incorporamos un módulo de RAG el cuál se encarga de responder las preguntas especificas sobre el ámbito de los doctorados.

El buen funcionamiento de todos estos módulos en conjunto, ha sido gracias a la implementación de un agente decisor el cual maneja el funcionamiento del sistema al completo, permitiendo así que este trabajase en sinergia. Cada una de las partes que lo forman ha sido implementada basándose en el modelo llama3.2:3b para tareas generativas y el modelo mxbai-embed-large para la generación de la representación vectorial de los documentos y las preguntas. Además, para esta búsqueda vectorial, nuestro estudio se ha basado en la similitud coseno.

Una vez desarrollada la implementación de cada uno de los módulos, se realizó una evaluación exhaustiva de cada uno de ellos, mediante un doble análisis. El primero de ellos, se basó en el uso de un módulo para automátizar la evaluación, mientras que el segundo, se llevó a cabo mediante una evaluación humana cuya finalidad era tanto determinar la eficacia del sistema en general como de comprobar la fiabilidad de las evaluaciones realizadas por el modelo de forma automática.

Dentro de este estudio, además del sistema también se midieron otros elementos como la latencia de las respuestas o la capacidad del agente decisor de llamar al módulo correcto. Estos elementos suelen implicar un buen funcionamiento del sistema y en general son cruciales de cara a una mejora de la

experiencia del usuario.

Para la parte automatizada se utilizó el mismo modelo que para los módulos(llama3.2:3b), al que mediante prompt engineering se le describió cómo tenía que realizar la tarea, y qué tenía que devolver. En nuestro caso, siempre le pedíamos una respuesta en formato json para facilitar el estudio posterior humano.

Capítulo 2

Contexto

A lo largo de este capítulo se introducirán una serie de conceptos que nos irán guiando hasta llegar al problema que nos concierne. Asimismo, introduciremos todas aquellas definiciones que resulten relevantes para permitir al lector una mejor comprensión del abordaje realizado al problema así como de la solución propuesta para el mismo.

2.1. Sistemas conversacionales

Los sistemas conversacionales son la base de este trabajo, sin embargo, es posible que algunos lectores nunca hayan escuchado hablar de ellos, o al menos no por ese nombre. Así, es bastante probable que cualquier lector haya escuchado hablar sobre los asistentes personales digitales, los asistentes personales virtuales, los asistentes conversacionales, los chatbots, las interfaces de usuario conversacional u otro de los 161 sinónimos que existen para denominar a los sistemas conversacionales.(Chatbots.org).

Sin embargo, a pesar de ser todos sinónimos, han ido obteniendo nombres muy dispares debido a las múltiples funcionalidades para las que han sido concebidos y a la multiplicidad de formas en las que han aparecido.

Definición 1.1 Un sistema conversacional (McTear, 2020), es un programa de ordenador que soporta conversaciones habladas, escritas o multimodales (incluye lenguaje verbal y no verbal)(Katashi Nagao, 1994) en las que interaccionan máquina y humano. Así, podemos distinguir dos tipos principales de diálogos:

- Task-oriented. Este tipo de diálogo se caracteriza porque los humanos y la máquina se involucran en una conversación para realizar alguna tarea. Un ejemplo de ello sería la reserva de una habitación en un hotel, para lo cual, la máquina va realizando una serie de preguntas al usuario como: ¿cama doble o simple?, ¿número de huéspedes?, etc. Generando una sucesión de diversos turnos de diálogo hasta que ambos consiguen llegar a la realización de la tarea, por ejemplo, la reserva de la habitación.

- Non-task-oriented. En este tipo, los humanos y la máquina interactúan en una conversación con temática general tal y como podría ser una conversación con un vecino.

Como podemos apreciar, la principal razón de ser de estos sistemas se basada en la interacción con un ser humano mediante una conversación. Sin embargo, cabe preguntarse el por qué surgieron estos sistemas para poder comprender mejor su evolución. Así, estos sistemas surgieron dentro de la propia industria, con el objetivo de reducir la barrera de entrada del usuario, permitiendo que éste abriese un chat y escribiese en lenguaje natural qué necesitaba, en vez de tener que familiarizarse con una página web o estar navegando durante horas para resolver cualquier duda. Vemos por tanto, como en sus inicios mejoraba la experiencia del usuario ya que le facilitaba las cosas.

Sin embargo, con el paso del tiempo, empezaron a destacar algunas limitaciones como la incapacidad de las máquinas de parecer humanas, generando un trato más distante con el usuario. Para evitar este trato distante, se estudió qué era lo que haría que una máquina pareciese humana. Este estudio llevó a lo que se consideró un elemento principal del ser humano, el lenguaje. Así, la clave estaba en el modo en que la máquina se comunicaba y realizaba las tareas, ya que esto podía permitir generar la sensación de naturalidad que se estaba buscando.

Esta solución que aparentemente puede resultar intuitiva y casi trivial, lleva consigo la necesidad de definir el lenguaje. Pero no solo eso, también requiere determinar por qué el lenguaje humano es para nosotros natural, mientras qué otros tipos de lenguajes los podemos considerar o percibir como artificiales.

2.1.1. Lenguaje natural

Para poder conseguir que una máquina sea capaz de replicar nuestro lenguaje, aquel que consideramos natural, primero tenemos que ser capaces de determinar qué consideramos como lenguaje, para posteriormente indicar qué hay de natural en él.

Definición 1.2 El lenguaje puede definirse (Vásquez et al.), (de Saussure, 2011) como un sistema estructurado de signos, que sirve para la representación y comunicación de información, donde los elementos se combinan siguiendo reglas específicas para crear y transmitir significado. Esto genera un lenguaje, sin embargo, aún desconocemos que elemento hace que un lenguaje pueda ser considerado o no como natural.

Definición 1.3 El lenguaje natural (Vásquez et al.) es un sistema aprendido de signos que ha evolucionado a lo largo del tiempo, para permitirnos comunicar un número ilimitado de significados sobre cualquier tema. Este se puede ver según (Akmajian et al., 2001) como un sistema adquirido culturalmente, capaz de expresar cualquier concepto, sin limitaciones temáticas, mediante la combinación creativa de elementos, en un contexto de intercambio comunicativo. Un

ejemplo de esto, son idiomas como el español o el inglés.

Viendo por tanto con esta definición como un lenguaje natural no es más que aquel lenguaje que ha evolucionado para permitir la comunicación entre seres humanos, podemos observar como contrariamente a ellos, existen los lenguajes formales (ej: Python, C). Por tanto, podemos decir que el lenguaje natural se caracteriza por:

- Estar compuesto de frases de posible longitud infinita, creadas a partir de un conjunto finito de elementos (el alfabeto), respetando una serie de reglas de formación (sintáctica) y de sentido (semántica).
- Tener un gramática, G, que se enriquece progresivamente.
- Gran capacidad expresiva gracias a los componentes semánticos.

Tenemos ya pues el primer punto para obtener un sistema que sea amigable para el ser humano, el lenguaje. Sin embargo, no es lo único a tener en cuenta, ya que nuestro objetivo último es que el sistema haga algo. Esto genera la necesidad de diferenciar los distintos tipos de interacciones que se pueden dar en una conversación.

2.1.2. Tipos de interacciones

Este es un elemento fundamental a la hora de diseñar un sistema conversacional, ya que nos permite incluir las funcionalidades necesarias para que el sistema realice las acciones que se esperan de él. Se pueden distinguir tres tipos fundamentales de interacciones, en los que los sistemas actuales pueden incurrir:

Diálogos iniciados por el usuario

Este tipo de diálogos suelen ser interacciones breves, consistentes en dos turnos de intercambio, en el que el usuario realiza una pregunta o indica algún tipo de acción o comando y el sistema le responde. Esta respuesta puede darse al mismo tiempo que el sistema realiza la tarea planteada. Un ejemplo de esto sería el asistente Siri. Un ejemplo de este tipo de interacción entre usuario y sistema podría tener la siguiente forma:

- User: Llama a Mama
- Siri: Hola, llamando a Mama
- (Se comienza a escuchar el tono de llamada y se puede observar como se esta llamando a Mama)

Vemos como en esta breve interacción, el usuario inicia al sistema con un requerimiento, en este caso una tarea a realizar (llamar por teléfono a un contacto de la agenda) y el sistema le responde y realiza dicha acción.

Para que este tipo de interacción se lleve a cabo de forma satisfactoria son necesarios los siguientes elementos:

- El input del usuario tiene que estar en una forma que el sistema pueda interpretar. Por ejemplo, si el sistema solo ha sido entrenado en texto, y el usuario final se comunica mediante voz, como en el caso anteriormente descrito, será necesario que haya un módulo de transcripción de audio a texto para permitir una correcta interpretación, ya que en caso contrario el sistema no sería capaz de comprender las peticiones del usuario.
- El usuario tiene que incluir toda la información en un único mensaje.
- El usuario necesita mandar el mensaje sin ningún tipo de interferencia. Así, si por ejemplo durante la petición anterior el usuario pasa cerca de una zona de obras o se le corta momentáneamente la cobertura, esta interacción no se podrá dar de la mejor forma, ya que el sistema no tendrá la capacidad de atender a los requerimientos del usuario.
- El sistema tiene que ser capaz de responder o de resolver la pregunta o tarea que le requiere el usuario. Este podría ser el caso de que a un sistema conversacional en el que se encargan pizzas. Si le pedimos que nos diga el tiempo, como el sistema no tiene ninguna forma de obtener dicha información, no será capaz de responder de forma adecuada al requerimiento del usuario.

Para los casos en los que las condiciones anteriores no se cumplan, el sistema suele tener respuestas predeterminadas como: "No he entendido su pregunta" o "No tengo la capacidad para consultar el tiempo". Sin embargo, estas suelen ir en detrimento de la experiencia del usuario, por lo que se han creado técnicas para permitir que el usuario realice una consulta o comando posterior:

- Slot replacement. En este caso, la petición es la misma que la original pero con un valor distinto del elemento objetivo. Para clarificar esto, veamos un ejemplo:
  - User: ¿Qué tiempo hace en Finlandia hoy?
  - System: Hoy en Finlandia hace una temperatura de 6 grados de máxima y 2 de mínima
  - User: ¿Y en Sevilla?
  - System: En Sevilla hoy hace una temperatura de 20 grados de máxima y 14 de mínima.

  En este caso, se puede observar como el objetivo de la conversación era el mismo, saber la temperatura que hacía, lo único que fue cambiando es el elemento sobre el que queríamos saber la temperatura (el slot).
- Resolución por anáfora. En este caso, la pregunta o cuestión del usuario se realiza usando una anáfora (allí, aquello ...) a un elemento de una petición anterior. Por ejemplo:
  - User: ¿Qué tiempo hace hoy en Sevilla?
  - System: En Sevilla hoy hace una temperatura de 20 grados de máxima y 14 de mínima.
  - User: ¿Cuánta población vive allí?

- System: En sevilla viven alrededor de 800.000 habitantes.

Este último, aunque parecido al primero, requiere realizar una inferencia sobre el contexto previo para encontrar la entidad a la que se hace referencia. Este problema se complica especialmente en aquellos casos en los que tenemos largas conversaciones con varios turnos.

Hecho este repaso de este primer tipo de interacción de diálogo, podemos observar como esta es preferible para conversaciones cortas en las que el usuario requiera información o una tarea en una única petición. Así, podemos considerar este como el punto más básico, en el que se espera que el sistema únicamente reaccione a las peticiones que el usuario hace. Esto nos genera la necesidad de obtener sistemas en los que no solo el usuario hable para poder llevar a cabo otro tipo de tarea. Para resolver esta necesidad, continuemos con el siguiente tipo de interacción de diálogo.

2.1.3. Diálogos dirigidos por el sistema.

En este tipo, el sistema es el que controla el diálogo. Así, este control lo puede ejercer de diversas formas:

Diálogos pro-activos

Estos sistemas de diálogo son la extensión natural a las notificaciones del móvil, sin embargo, en vez de mandar un notificación, lo que hace es iniciar una interacción con el usuario asegurándose que a este le ha llegado el recordatorio al hacer que este tenga que involucrarse de forma activa. Aunque el usuario tiene que participar en la conversación, esta participación suele reducirse a un simple sí o no. Un ejemplo de ello sería un recordatorio de cita médica en el que se le diga al paciente algo como: Mañana tienes cita con el internista, ¿vale?.

Diálogos instruccionales

Este tipo de diálogos suelen ser iniciados por el usuario en busca de instrucciones. Por ejemplo, en un sistema de GPS, el usuario podría preguntar sobre las indicaciones para llegar a un lugar determinado. Tras esta primera petición, el sistema tomaría el control realizando una descripción del paso a paso que debe llevar a cabo el usuario para llegar desde el punto de partida a su destino. Aquí, el usuario puede pedir la siguiente instrucción o pedir que se le repita alguna de las instrucciones dadas por el sistema. Otro ejemplo de este tipo podría ser un sistema que indicará las instrucciones de una receta de cocina.

Diálogos slot-filling

Estos diálogos son iniciados por un usuario requiriendo un servicio. Por ejemplo, la gestión de una reserva en un hotel o la petición de una cita médica. En este caso, el sistema toma el control realizando una serie de preguntas al usuario para determinar sus requerimientos y así poder completar la tarea. Por ejemplo, en el caso de la cita médica, podría preguntar por cuál es el médico de cabecera del usuario, a continuación preguntar por la fecha para la que quiere la cita y

la hora, etc.

Este tipo de diálogos se denominan slot-filling debido a que el sistema busca ir rellenando una serie de casillas (slots) a medida que realiza preguntas al usuario para conseguir completar una tarea determinada (ej: reservar en un hotel). Estos slots vienen definidos por las intenciones del usuario, las cuales llevan a unos actos de diálogo que posteriormente determinarán el flujo del diálogo (Louvan and Magnini, 2020). Así, si el usuario busca reservar un vuelo durante su conversación, tendrá que facilitar datos como el aeropuerto de salida, de destino, fechas, etc.

Téngase en cuenta que estas intenciones (Louvan and Magnini, 2020) corresponden con necesidades especificas del usuario (ej:bloquear una tarjeta, coger una cita médica, etc) o con necesidades generales (ej:preguntar por clarificaciones, agradecimientos, etc), por lo que tienen que ser tomadas en cuenta, ya que describen el objetivo que tiene el usuario al entrar en el chat. Esto se debe a que para el usuario no sería lo mismo hacer una transferencia bancaria, que saber cuál es la cantidad mínima de dinero para realizar una transferencia bancaria. De lo anterior se deduce como es necesario diferenciar entre una intención y una entidad, siendo una intención, lo que el usuario quiere conseguir con el diálogo (ejemplo reservar un hotel) y una entidad, los parámetros necesarios para cumplir con una intención(ej: número de personas, tipo de cama, distancia al centro, etc)

Además, los actos de diálogo (Jurafsky et al., 1997) representan el significado de una intervención dentro de un diálogo. Básicamente, es lo que alguien quiere que hagas cuando te dice algo y no es lo que dice en sí. Así, por ejemplo, si un profesor te dice: "cierra la puerta, por favor". El acto de diálogo consiste en la acción de levantarse y cerrar la puerta. Otro ejemplo de un acto de diálogo podría ser cuando al inicio de una conversación uno de los dos interlocutores dice "Hola". El acto de diálogo es saludar, es decir, es la acción en sí que se está realizando o se está queriendo que se realice. Es obvio notar por tanto, como estos actos de diálogo tienen una intención intrínseca.

Por último, es necesario tener en cuenta, tal y como se define en (Pieraccini and Huerta, 2008) y (Oliveira, 2024), que un flujo de diálogo, no es más que la definición del transcurso que toma la conversación, viniendo definido por el conjunto de entidades y por la propia evolución de la conversación, la cual va llevando de un sitio a otro de forma natural. Un ejemplo de esto sería un flujo conversacional en el que un usuario preguntase por una cita médica, el sistema le respondiese pidiéndole el médico de cabecera, la hora de la consulta, etc. Aquí podemos observar como cada flujo conversacional tiene un intención primaria (en este caso era coger una cita médica), que se irá desarrollando mediante actos de diálogo.

Esto se podría preestablecer como todos los posibles estados en los que un diálogo puede encontrarse, así como todas las posibles transiciones entre estados. Además, para representar estos flujos conversacionales habitualmente se usan gráficos de diálogo que representan las posibles acciones que el usuario puede realizar desde un punto inicial. Esto nos permite vislumbrar los posibles puntos

a los que llegaría la conversación en función de las decisiones que fuese tomando el usuario. Por tanto, el flujo de diálogo especifica el flujo de una conversación, desde un punto inicial hasta un punto final, incluidas las ramificaciones en las que se puede ir en distintas direcciones.

Vemos como este sistema se fundamenta en el rellenado de una serie de huecos para completar una tarea. Esto se realiza mediante una serie de actos de diálogo en el que el sistema le requiere dichos valores al usuario. Así, se puede observar como este último tipo de sistemas es bastante parecido a rellenar un formulario en la web, salvo que en vez de tener que rellenarlo a mano, este se va rellenando a medida que avanzamos en la conversación. Esto además, permite al usuario mayor flexibilidad al poder incluir varios valores del "formulario" dentro de un mismo mensaje o incluso que el usuario no incluya ninguno de los valores. Esto genera a su vez que el sistema solo tenga que pedir los slots que faltan para poder realizar la tarea sin tener así que ir requiriendo uno por uno todos los elementos del "formulario".

2.1.4. Diálogos multi-turno de dominio abierto

Este tipo de diálogos se desarrollan al igual que se desarrollaría una conversación entre dos personas. Es decir, cualquiera de los dos puede tomar la iniciativa de la conversación. Esto genera que el diálogo se pueda alargar durante varios turnos tratando temas de diversa índole. Un ejemplo de ello sería:

- System: Hola, ¿qué tal estás hoy?
- User: Muy bien, hoy hace sol. ¿Y tú?
- System: Yo también me encuentro fantástico. ¿Te gusta que haga sol?
- ...

Este tipo de sistemas permite, a diferencia de aquellos en los que el usuario iniciaba la conversación, que esta pueda indicar que ha comprendido o recibido el mensaje del sistema, y que el sistema entienda que esto ha sucedido, pudiendo cerrar de forma correcta el intercambio de información. Además, este tipo de diálogos permiten que ambos interlocutores utilicen expansiones en la conversación, es decir, pidan más detalles o que se repita o refraseen algún elemento concreto. Esto permite que el cliente puede usar varias o ninguna expansión de la conversación para llegar al objetivo final. Asimismo, debido a las características intrínsicas de las conversaciones, este tipo de sistemas tienen que ser capaces de lidiar con anáforas teniendo por tanto la necesidad del uso del contexto para poder responder a las preguntas que las contengan.

Vemos por tanto, como cada uno de estos tipos de interacciones requieren una serie de características del sistema. Así, los que responden a requerimientos del usuario, necesitan tener la capacidad de contestar a preguntas o realizar acciones, mientras que los que son pro-activos requieren únicamente tener la capacidad de iniciar una conversación y de recoger una respuesta simple del usuario. Por lo tanto, siempre que se diseñe un sistema deberá que tenerse en cuenta el uso para el que se quiere destinar, para poder determinar la forma

más correcta posible de su arquitectura.

Visto ya los tipos de interacción, podemos pasar ahora a uno de los puntos principales del trabajo, las formas existentes para abordar el diseño y el desarrollo del sistema de conversacional.

2.2. Tipos de sistemas de diálogo

En esta sección introduciremos los tres métodos que se usan para la creación de los sistemas de diálogo, indicando las problemáticas que presentan cada uno de ellos y cuales resuelven.

2.2.1. Basado en reglas

En estos sistemas, tanto el flujo conversacional como otros aspectos de la interfaz se hacen a mano basándose en la guías de buenas prácticas existentes. Teniendo por tanto, que son sistemas que operan siguiendo un conjunto predefinido de instrucciones o reglas para gestionar la interacción con el usuario. Estas reglas determinan cómo el sistema responde ante determinadas preguntas, guiando el flujo de la conversación mediante patrones, condiciones y acciones específicas. El uso se guías de buenas prácticas para la creación de estos sistemas nos permite describir el flujo conversacional habitual que suelen seguir:

- Introducción por parte del usuario de un acto de diálogo (pregunta, respuesta a un prompt del sistema, etc).
- Identificación de este prompt mediante el uso de una componente de lenguaje natural. Aquí, el componente de procesamiento de lenguaje natural recibe un string (el texto) y su función es determinar su significado. Esto puede realizarse de varias formas distintas:
  - Usando un análisis semántico basado en la sintaxis. En este caso, el texto primero se analiza sintácticamente para determinar como las palabras y frases se agrupan en un componente. Posteriormente, se obtiene el significado derivado del significado semántico de cada uno de los componentes.
  - La otra alternativa consiste en obtener el significado directamente de la expresión introducida por el usuario. Esto se realiza utilizando una gramática semántica que se basa en la creación de reglas semánticas tal y como se haría con las sintácticas, pero basando la clasificación en términos de significado o función, en vez de en términos de categorías sintácticas.
- Incorporación de este prompt al contexto del sistema de diálogo para apoyar el proceso del manejo del diálogo. Este contexto de diálogo debe contener:
  - Historial de diálogo: Consiste en tener guardado de qué entidades o proposiciones se ha hablado durante el diálogo.

  - Grabación de la tarea: Indica la información que ya se ha obtenido y la información que falta para poder realizar una tarea determinada. En muchas ocasiones, esto recibe el nombre de agenda, ya que marca los pasos a seguir por el sistema.
  - Modelo de dominio: Contiene información específica sobre el dominio sobre el que se está tratando, por ejemplo: reserva de un hotel.
  - Modelos de competencia conversacional: Incluyen conocimientos genéricos sobre cómo actuar ante diferentes actos de diálogo. Por ejemplo, que ante una pregunta sepa cuál es la manera correcta de proceder para facilitar o suministrar la información al usuario.
  - Modelo de preferencia del usuario: Esto contiene información estable del usuario que puede ser relevante de cara al diálogo (ej: edad, género, gustos, etc) y información que cambia a lo largo de la conversación (ej: creencias, intenciones, etc).

- Elección de la siguiente acción del sistema en base al contexto del sistema de diálogo.
  Este tipo de sistemas se basan en decisiones predefinidas que están determinadas por factores como el nivel de confianza con el input del usuario, el cual tiene en cuenta las instrucciones previas. Por ejemplo, si este nivel de confianza está por encima de una cierta cantidad, el sistema puede suponer que ha entendido correctamente el input del usuario y continuar con la siguiente acción. Sin embargo, si está por debajo, el sistema podría pedir clarificaciones ya que no tiene seguro haber comprendido de forma correcta al usuario.

- Generación de la respuesta en lenguaje natural del mensaje correspondiente al resultado de la acción ejecutada por el sistema. Un ejemplo de ellos sería decir "hola" o "¿qué tal estas?".

  Los sistemas basados en reglas tienen una componente por cada uno de los elementos descritos anteriormente, lo cual genera la necesidad de crear y optimizar cada una de ellas, todo esto sin contar que la comunicación pudiese darse por otros canales como el hablado, en el que el sistema tendría primero que transcribir lo que se le ha comunicado oralmente a texto, teniendo así que añadir dos componentes más, traducción voz a texto y traducción texto a voz.

  Podemos ver como también en este caso, al realizar la generación en lenguaje natural se devuelve un acto de diálogo. Este representa una función del mensaje o prompt del usuario, que puede contener preguntas, amenazas, promesas o comandos. Así, el usuario puede estar buscando con su instrucción dos tipos de actos de diálogo distintos:

  - Actos de futuro. Algunos ejemplos de este tipo serían actos como declaraciones, acciones-directivas o solicitudes de información que esperan una respuesta.
  - Actos retrospectivos: por ejemplo, actos como acordar, aceptar y reconocer que responden a un actos de futuro.

Puede observarse como un ejemplo clásico de este tipo de sistemas es aquel en el que el usuario tiene que ir seleccionando una serie de opciones, que se le van abriendo con cada elección hasta llegar a una que más o menos concuerde con sus necesidades. Esto en si mismo es un problema ya que tiene poca capacidad de personalización de la experiencia del usuario. Además, requiere una gran carga de trabajo humano, al tener que escribir las reglas a mano y tratar de determinar todos los posibles flujos conversacionales, lo cual es prácticamente infinito ya que las conversaciones pueden ser abordadas por dos usuarios de formas muy diversas.

Estos sistemas, aunque solucionan el problema original por el que surgieron permitiendo una experiencia de usuario mucho más cómoda al involucrarlo en una conversación, traen consigo múltiples problemas:

- Son sistemas muy costosos de desarrollar.
- Son difícilmente adaptables a otros dominios.
- No son buenos manejando errores o problemas, como mal entendimiento.
- Son difíciles de mantener, ya que tienen que actualizarse de forma manual.
- A pesar de que se sigan los estándares, esto no asegura que demos con una solución óptima.
- También tienen dificultades para manejar correcciones del propio usuario dentro de la misma frase.

2.2.2. Basado en datos estadísticos.

Estos sistemas se basan en técnicas de Machine Learning, manejando cada uno de los componentes con un enfoque probabilístico.
Este tipo surge tratando de resolver los problemas que tenía el tipo de sistema conversacional anterior. En este, el uso de los datos los hace que sean fácilmente actualizables, ya que esta se realiza de forma automática. Además, de este problema de escalado, al que hemos querido prestarle especial atención, es necesario indicar que también resuelve todos los problemas descritos anteriormente.

Por tanto, vemos como este sistema se basa en el aprendizaje a partir de los datos. Unos datos que pueden presentarse de diversa forma, pudiendo distinguir:

Corpus-based o example-based.

Este tipo de datos se refiere a diálogos previos cuyo dominio es similar al que tiene que abordar el sistema en ese momento. Este se basa en tratar de determinar la mejor acción a tomar por el sistema teniendo en cuenta una serie de estados, consistiendo cada estado en un turno del sistema o un turno del usuario.
Esto al suponer una gran carga computacional suele abordase de varias formas distintas, como podría ser:

- Tener en cuenta para la puntuación solo si el usuario ha dado un valor para un determinado elemento, no su valor.

- Suponer como iguales todas las cadenas de razonamiento que lleven a la misma acción.

Vemos por tanto como estos datos son específicos para determinar que respuesta dar, basándose para ello en contestaciones parecidas a preguntas parecidas que se hayan dado en conversaciones previas.

Los diálogos de este tipo tienen problemas a la hora de manejar preguntas a elementos no vistos. Así, el sistema tiende a resolverlo utilizando la cuestión más cercana o más parecida a aquello que se le ha preguntado o respondido. Además, necesitan grandes cantidades de datos, que si bien este problema puede solucionarse mediante el uso del few-shot learning (entrenamiento basado en un número pequeño de ejemplos), esto no obtiene resultados tan buenos como aquellos modelos entrenados con una mayor cantidad de datos. Como alternativa, también pueden usarse simuladores de usuarios, que actúan con un comportamiento similar al que lo haría el usuario lo que permitiría recoger estas grandes cantidades de datos en forma de conversación de una manera más automatizada.

Derivado por interacción.

En este caso, los datos son usados para optimizar la estrategia del sistema, es decir, que acciones debe seguir para llegar al mejor punto posible (que suele ser la satisfacción o conformidad del usuario). Para ello suelen usarse o bien usuarios reales o bien usuarios ficticios. Este último enfoque se basa en técnicas de refuerzo por aprendizaje y en procesos de decisión de Markov parcialmente observables (POMDPs).

En este caso se fundamenta en encontrar las acciones o política óptima, la cual será aquella que obtenga una mayor recompensa. Esto usualmente se suele realizar usando métodos como el **Q-function** para realizar la optimización. Este método se basa en determinar cómo de deseable es una acción, teniendo en cuenta el estado presente y todos los estados que es probable que se siga hasta el final del diálogo. Calculando por tanto el premio esperado desde un estado haciendo una determinada acción y siguiendo una determinada política.

Vemos como esta es la base general de esta forma de modelización de un sistema basado en datos. Sin embargo, es necesario detenerse para entender bien cada uno de los puntos que lo componen:

En primer lugar, **el aprendizaje por refuerzo**, tal y como se describe en (Kaelbling et al., 1996) y en (Marco Wiering, 2012), consiste en un agente que tiene que explorar un espacio. En dicho espacio, existen una serie de estados (esto puede ser cualquier cosa que el agente tenga que tener en consideración para tomar una decisión) y unas transiciones entre estados. En cada espacio de tiempo, el agente selecciona una acción (ésta es una decisión que el agente tiene que aprender a realizar, ej: cómo realizar el flujo conversacional o qué responder ante determinados requerimientos) entre las posibles desde el estado en el que se encuentra para moverse a otro, recibiendo una recompensa por ello. Esto continúa así hasta llegar al estado final recibiendo con ello la recompensa final. Aquí podemos observar como el objetivo del agente es maximizar las recompensas.

Téngase por último en cuenta que el proceso descrito anteriormente se modela matemáticamente y recibe el nombre de **Procesos de Decisión de Markov(MDP)**.

En el caso de un diálogo, este se puede entender como una secuencia de turnos entre el usuario y el sistema, navegando por un conjunto de estados hasta llegar a uno deseado, como por ejemplo realizar una reserva de una habitación o pedir una cita médica. En cada espacio de tiempo, el sistema decide una acción en base a sus creencias del estado actual y de lo que ha recibido por parte del usuario. Así, el objetivo de la máquina es encontrar la política (camino a seguir desde un estado) óptima para conseguir una determinada métrica de logro como podría ser la satisfacción del usuario o la eficiencia al completar el diálogo.

Aunque este modelo parece aparentemente bueno, hay una cosa que no hemos tenido en cuenta. Estos modelos suponen que el conjunto de estados del sistema es completamente observable, es decir, puedes saber los posibles sitios a los que llegarás en todo momento. Esto, en una conversación, parece algo poco plausible debido a la incertidumbre intrínseca de un diálogo. Un ejemplo sería que el sistema no puede estar seguro de haber interpretado correctamente las intenciones del usuario en un procesamiento de lenguaje natural, ya que incluso aunque este sistema fuese perfecto en el lenguaje, existen ambigüedades que generarían esta incertidumbre.

Debido a esta limitación de los MDP, surge el uso de Reinforcement learning o aprendizaje por refuerzo en conjunto con los procesos de decisión de markov dando lugar a los **POMDPs**. La diferencia principal de estos sistemas respecto de los anteriores, es que son capaces de modelar una distribución de probabilidad entre los diferentes estados en el que el sistema puede encontrarse. En este caso, como los estados no se pueden observar, el sistema tiene una creencia del estado en el que se encuentra gracias a la distribución de probabilidad. Esto permite al sistema manejar mejor las incertidumbres que el propio lenguaje introduce en la interacción, al poder determinar cuál es la intención que cree más probable que tenga el usuario.

Nótese, que aunque estos son los más comunes, existen también otros enfoques como el modelado usando cadenas ocultas de Markov o el modelado usando redes bayesianas.

Vemos por tanto como con ambas formas de generar modelos conversacionales a partir de datos, este enfoque consigue resolver la mayoría de los problemas que tenía su predecesor (rule-based), sin embargo, también se obtienen otros problemas nuevos:

- Tiene problemas en la escalabilidad, ya que si se aumenta el número de objetivos del usuario, aumenta también el espacio de creencias de estados volviéndose la actualización intratable. Algunas posibles soluciones serían, la partición del espacio de creencias o el tratamiento de las grandes masas como unidad única.
- Otro problema, es que aunque bajo este modo se trata como si fuera un problema de optimización, en ocasiones es preferible que tarde más pasos,

pero que haga de forma correcta la tarea, a que tarde menos y la haga de forma errónea.

- Además, existen ocasiones en las que el usuario quiere mantener una conversación extendida, lo cual, al intentar optimizarse tratando de reducir el número de pasos, no se estaría atendiendo.

Asimismo, es necesario tener en cuenta que para resolver algunos problemas, como la actualización de la información a medida que el diálogo progresa, se usan algoritmos estadísticos. En algunas ocasiones estos algoritmos son basados en redes neuronales, ya que son capaces de mejorar la actuación en diálogos con múltiples dominios respecto a otros algoritmos más específicos tal y como se describe en (Henderson). Lo que nos lleva a los siguientes tipos de sistemas.

2.2.3. Sistemas end-to-end de diálogo neuronales

En la actualidad, se utilizan mucho estos métodos para mapear directamente la entrada del usuario a una salida sin necesidad de realizar el procesamiento en cada una de las componentes que se usaban anteriormente. Esto evita algunos de los diversos problemas que tenían los modelos anteriores, en concreto, los basados en modelos estadísticos con aprendizaje basado en datos. En estos sistemas, uno de los problemas era la propia arquitectura modular, ya que complicaba el diseño, esto llevaba a que al intentar optimizar cada uno de los módulos (ya fuese a mano o mediante técnicas de Machine Learning) para obtener mejores resultados, podía generarse una pérdida global de eficiencia. Así, en artículos como en (Lemon, 2011) se indica como la optimización en conjunto de dos componentes daba mejores resultados que la optimización por separado de los mismos componentes. Llegando de esta forma a que las arquitecturas end-to-end, en las que se basan este tipo de sistemas, resuelven este problema debido a que no tienen estas componentes (por no ser modulares, si no que se hace el mapeo directamente del input al output), teniendo por tanto que optimizar un solo elemento. Asimismo, otro problema derivado del enfoque estadístico, es el uso del aprendizaje por refuerzo. Esta técnica adolece de una limitación general conocida como el *Credit Assignment Problem*. Además, de esta ventaja respecto a los sistemas anteriores, también mejoraba el rule-based, ya que no requieren ningún tipo de esfuerzo para optimizar el sistema a mano.

Para poder realizar todo esto, este método se basa en el uso de las DNNs, lo que le reportó el nombre de diálogo neuronal. Y en el uso de las técnicas Seq2Seq (Sutskever et al., 2014) para poder realizar el mapeo del que hablábamos anteriormente.

Así y antes de continuar, será necesario hacer una breve descripción de ambas técnicas ya que son la base de este tipo de sistemas de diálogo.

Seq2Seq

En primer lugar, podemos observar en (**?**) como Seq2Seq, es la abreviatura de Sequence to sequence. Esta abreviatura hace referencia a la capacidad de

las redes neuronales de mapear secuencias de longitud indeterminada a priori de una entrada, por ejemplo una pregunta, a una salida que viene dada por una secuencia de palabras cuya longitud depende de la pregunta realizada en la entrada.

DNNs

Tras esto, tal y como se describe en (Cichy, 2019) las redes neuronales profundas (DNNs) son modelos computacionales que constan de muchas unidades de procesamiento simples (similares a las neuronas) que funcionan en paralelo y están dispuestas en capas interconectadas. Estas se basan en las redes neuronales simples, que constaban de una capa de entrada y una capa de salida. Por tanto, al apilar más capas en las redes neuronales simples, estas pasan a denominarse redes profundas(DNN).

Dentro de ellas, podemos distinguir varios tipos como serían las RNN. Estas son redes neuronales recurrentes (RNN), que operan sobre secuencias de vectores, lo que permite que se capture mejor la información de las entradas anteriores pudiendo así mantener cierta memoria a corto plazo muy útil para el procesamiento posterior. Además, al procesar las entradas como una secuencia, pueden capturar información sobre el orden de la entrada, que puede ser relevante para el procesamiento de la secuencia de entrada en su conjunto.

A pesar de que las RNN son capaces de retener cierta memoria de las entradas anteriores, esta memoria no es capaz de ir muy atrás en las información codificada haciendo que las capas ocultas tiendan a fallar localmente en tareas como las predicción de la siguiente palabra. Este problema puede resolverse mediante el uso de otras arquitecturas como las LSTM.

Las LSTM, abreviatura de Long Short-Term Memory, es un tipo de redes descritas en (Hochereiter and Schmidhuber, 1997),(Sherstinsky, 2020) que trata de resolver el problema del gradiente desvaneciente (Las primeras neuronas de una red muy profunda apenas recalculan sus pesos en cada interacción porque el error se va diluyendo a lo largo de las capas). Este problema lo resuelven basándose en el siguiente componente:

- Células de memoria. En ellas se almacena la información a lo largo del tiempo. Estas se componen de dos elementos principales:

    - Input Gate Unit cuya existencia se basa principalmente en proteger las células de memorias para evitar que guarden información irrelevante. Así, la red puede usar este tipo de puertas para decidir cuando mantener o sobreescribir información en la célula de memoria.

      Comparando con otras arquitecturas como las GRU(Gated recurrent unit), se puede ver como en ellas esta unidad se divide en dos elementos bien diferenciados:

        - El primer elemento sería el que determina qué se elimina

    - El segundo elemento es el que determina qué información se recoge

  Así, la Input Gate descrita anteriormente mezcla ambos comportamientos decidiendo al mismo tiempo qué información eliminar y qué información añadir.

- Las Output Gate Units, se usan para proteger otras unidades de posibles perturbaciones creadas por información irrelevante almacenada en una célula de memoria. Por tanto, este tipo de puerta puede usarse para determinar cuándo acceder a la memoria de la célula y cuándo prevenir otras unidades de verse perturbadas por esta célula de memoria. Así, esta nos determina la información a devolver.

  Vemos como estas dos unidades pueden usar información de otras células de memoria para determinar si incluir o no una determinada información en su célula de memoria.

Esto nos permite apreciar como este tipo de arquitecturas son capaces de aprender de datos con grandes dependencias temporales, siendo una elección natural para el entendimiento de las frases, las cuales son en esencia secuencias de unidades, que pueden ser palabras, caracteres o en general tokens, y que tienen relación entre si.

Observesé por tanto, como esta arquitectura se basa en las redes neuronales para los diálogos. Siendo la idea básica, modelar una conversación prediciendo el siguiente output en un diálogo dado un input (por ejemplo el prompt de un usuario). Asimismo, estos modelos pueden entrenarse usando Seq2Seq para evitar tener que escribir las reglas a mano o realizar complejas iteraciones lo que permite a estos sistemas obtener mejoras respecto a los descritos anteriormente.

Así, podemos apreciar como el modelo lee un token en cada instancia de tiempo, almacenándolo como contexto en las capas ocultas para después generar un token en cada instante de tiempo, generando así el mapeo entre la entrada y la salida a partir de lo aprendido durante el entrenamiento.

Vistas ya algunas de las componentes principales en las que se basan este tipo de sistemas de diálogo, será interesante determinar cuáles son las dos tareas principales que se realizan dentro de esta arquitectura:

- Procesamiento y representación del input. Conocido como Encoding
- Generación del output. Esto se conoce como Decoding

Estas tareas, requieren una serie de elementos que explicaremos previamente antes de pasar con ellas:

Word embedding

El Encoder o codificador suele recibir una serie de palabras dichas o escritas por el usuario. Sin embargo, para que el codificador pueda procesar estas palabras, deben convertirse en números. Como resultado de este proceso, conocido

como word embedding, cada palabra se representa mediante un vector único de números reales que captura su significado y su relación con las demás palabras del vocabulario.

Estas representaciones pueden realizarse usando diversas técnicas:

- One-hot encoding. Este tipo de codificaciones se basa en representar cada una de las palabras como un vector de 0 y 1 donde este vector tiene un 1 en la palabra y un 0 en el resto. Esto aunque aparentemente parezca sencillo, trae dos problemas consigo:
  - El primero de ellos, es que estos vectores tienen de longitud la longitud del vocabulario lo que nos lleva a que si en un vocabulario normal hay 30.000 palabras, tendremos un uno y 29.999 ceros, lo que haciendo extensivo a todos los términos del vocabulario obtengamos una matriz muy dispersa en la que la mayoría de los elementos son cero.
  - Además, este tipo de representaciones tampoco permiten modelar relaciones entre palabras, ni los contextos en los que aparece, lo cual suele ser relevante para determinar similaridades semánticas o incluso relaciones entre palabras, pudiendo ser útil para completar las frases que se vayan generando, teniendo que si ya ha escrito "he comprado pienso para", la siguiente palabra más probable sea "perro" porque suele aparecer junto con "pienso", mientras que no tenga tanto sentido que aparezca "banco".
- Representación de palabras en un espacio semántico. En este caso, cada una de las palabras es mapeada a un vector el cual es representado en un espacio semántico. En dicho espacio, los elementos similares obtienen representaciones similares pudiendo así determinar el significado de una palabra gracias a su localización en la representación vectorial. Este tipo de técnicas permite aprender relaciones sintácticas y semánticas a partir de las representaciones aprendidas, las cuales se obtienen mediante el entrenamiento de redes neuronales como son las RNNs. Para realizar este último tipo de representación han surgido numerosas técnicas como Word2vec, Google's Bert etc.

Tras esta breve explicación de los componentes principales de una arquitectura endocer-decoder, pasemos a mostrar el comportamiento de este tipo de redes.

Redes codificador-decodificador

En este tipo de arquitecturas, el codificador o encoder toma el input o entrada del usuario y genera un vector a partir de él, que es posteriormente pasado al decodificador o decoder, que genera una salida en base a dicho vector.

Por tanto, para la parte de codificación podemos usar RNNs o LSTMs para la generación de estos vectores en los que se incluyan las dependencias temporales entre las diferentes palabras.

Vemos por tanto como los embedding se pueden aprender a partir de un gran conjunto de textos a partir, del cual se cree la representación vectorial de cada una de estas frases, mostrando al decodificador cual es la siguiente palabra que se suele esperar en función de los diversos contextos.

Para evitar el problema de las redes codificador-decodificador, en el que el rendimiento disminuye a medida que la secuencia de entrada se alarga, pueden usarse los mecanismos de atención. Estos consiguen que se utilicen solo las partes de la representación de la entrada que se consideran particularmente relevantes para predecir la siguiente palabra en la salida. Dicho de otra forma, la secuencia de entrada completa no se codifica en un único vector de longitud fija, sino en un conjunto de vectores y el decodificador selecciona un subconjunto de estos durante la decodificación.

Así, estos mecanismos de atención se pueden aplicar dentro de la estructura codificación-decodificación mediante el uso de los Transformers(Vaswani et al., 2017). Este tipo de redes no necesitan hacer uso de la recurrencia ya que tienen como base los mecanismos de atención para las redes encoder-decoder. De esta manera, algunos autores (Vlasov et al., 2019) han aplicado embedding basados en transformers para los sistemas de diálogo.

Vemos como estos mecanismo nos aporta una mejora con respecto a los otros tipos de sistemas conversacionales, consiguiendo que para casos más generales sean preferibles. Esta mejora se debe al uso de la relaciones semánticas existentes en los elementos que los componen. Sin embargo, a pesar de esto, los sistemas end-to-end tienen varios problemas.

Los problemas más remarcables de este tipo de sistemas son que aunque son capaces de formar frases que son sintácticamente correctas, estas fallan al no tener en cuenta el contexto global. Así, aunque anteriormente indicábamos que la generación de las frases estaba basado en un contexto, este, solo aplicaba al historial inmediatamente relacionado con el requerimiento actual sin tener en cuenta el contexto de conversaciones previas.

Además, otro problema bastante frecuente en este tipo de sistemas es la generación de respuestas, que si bien son relevantes, suponen una contestación desinformada o blanda. Siendo ejemplos de ello frases como: “No lo se”, “De acuerdo”, etc.

Así, con el fin de solucionar alguno de los problemas anteriores, surgieron los siguientes tipos de modelos.

2.3. Pre-trained Language Models (PLM)

Este tipo de modelos se basan, tal y como se describe en (Wang et al., 2023) y (sci), en la idea de la transferencia de conocimiento, es decir, en la re-utilización del conocimiento aprendido en una o más tareas para la realización de una nueva tarea.

Inicialmente este conocimiento era obtenido a partir de unos datos previamente anotados por medio de Aprendizaje Supervisado. Sin embargo, con el surgimiento del DeepLearning este aprendizaje comenzó a ser auto-supervisado. Esto último permitía su utilización en múltiples ámbitos gracias al uso del fine-tuning (reentrenamiento de un modelo previamente entrenado para ajustarlo ligeramente (Howard and Ruder, 2018)) o del few-shot learning (utilizando el conocimiento previo, FSL(few-shot learning) para generalizar rápidamente a nuevas tareas a partir de sólo unas pocas muestras con información etiquetada (Wang et al., 2020)).

Además, basándose en la arquitectura Transformer (Vaswani et al., 2017), se propuso BERT (Devlin et al., 2018) mediante el pre-entrenamiento de modelos lingüísticos bidireccionales con tareas de pre-entrenamiento especialmente diseñadas para corpus no etiquetados a gran escala. Estas representaciones de palabras, pre-entrenadas y sensibles al contexto, eran muy eficaces con características semánticas de propósito general, lo que elevó considerablemente el nivel de rendimiento de las tareas de PLN.

Así, los Modelos Lingüísticos Preentrenados (PLM) son redes neuronales entrenadas con corpus a gran escala sin etiquetar, que pueden ajustarse para diversas tareas. Estos modelos, contienen una cantidad significativa de conocimiento lingüístico en sus parámetros, lo que los hace útiles para optimizar el rendimiento en tareas como la detección del sarcasmo.

A partir de estos modelos, se descubrió como un escalado del mismo (en tamaño o en datos) generalmente mejoraba la capacidad del modelo, dando lugar a los siguientes modelos.

2.4. Large Language Models (LLM)

Estos modelos de lenguaje son modelos pre-entrenado(PLM) que tienen un gran número de parámetros o que han sido entrenados con un corpus muy extenso de ejemplos. A pesar de la gran variabilidad y de la no existencia de consenso, muchos autores determinan que un modelo puede considerarse LLM cuando tiene más de 10 billones de parámetros (Wang et al., 2023) y (Hadi et al., 2023). Por tanto, estos modelos no son más que modelos PLMs de gran tamaño. Esto les permite desarrollar capacidades para resolver diversas tareas de procesamiento de lenguaje natural e incluso el desarrollo de otro tipo de capacidades para los que no estaban pensados en un inicio, tal y como se indica en (Zhao et al., 2025). Así, una de las capacidades más interesantes que desarrollan los LLM con respecto a los PLM es el in-context learning (ICL).

El in-context learning (ICL) consiste en la capacidad del modelo de aprender a realizar una tarea a partir de demostraciones de pocas instancias, lo que permite que el modelo aprenda en base a ejemplos en lugar de ajustar los parámetros del modelo (Luo et al., 2023) y (Dong et al., 2024). Un ejemplo de ello, podría ser un prompt de traducción que contuviese los siguientes ejemplos para enseñarle a traducir al sistema:

Inglés: *dog* → Español: *perro*
Inglés: *cat* → Español: *gato*
Inglés: *house* → Español:
Ante la última pregunta, el modelo sabrá que la respuesta correcta es casa aunque no haya sido entrenado para realizar traducciones.

Algunos ejemplos conocidos de este tipo son los GPT3 (Brown et al., 2020) y GPT4 (OpenAI et al., 2024) sobre los que se soportan chats mundialmente conocidos como ChatGPT [1].

2.5. Problemas de los LLM

Así, los LLM tradicionales han demostrado su gran capacidad para completar texto, resumir o traducir. Sin embargo, entre los múltiples retos que han tenido que afrontar se encuentran la relevancia, creatividad o credibilidad de las respuestas dadas (Mohammad, 2024).

- La primera limitación que podemos encontrar en un LLM es su relevancia. Este problema se debe a que estos modelos de lenguaje se basan en datos estáticos que incluyen únicamente aquello que aprendieron durante el entrenamiento, lo que les lleva a generar información anticuada o inconsistente.

- Con respecto a la creatividad, al fundamentarse estos modelos en datos de naturaleza estática y en conjunto de entrenamiento de tamaño limitado, las respuestas generadas suelen estar restringidas al ámbito aprendido, lo que corta cualquier posible pincelada de creatividad que pueda surgir de la unión de trozos dispares de información.

- Por último, la tercera limitación puede verse en aquellos contextos en los que el output es muy grande. En estos contextos se observa como el LLM genera información incorrecta o incompleta debido a la gran dependencia del modelo en patrones estadísticos aprendidos durante el entrenamiento. Estos patrones pueden llevar a generar información que suene bien pero que sea incorrecta lo que comúnmente se conoce como alucinaciones.

Estas se pueden considerar unas de las grandes limitaciones de los LLM pero no son las únicas, pudiendo observar tal y como se describe en (Bansal and Suddala, 2024), como estos también tienen severos problemas a la hora de la propagación del sesgo existente en el conjunto de entrenamiento o del acceso a datos sensibles entre otros.

Aunque existen otras formas de conseguir mejoras en las respuestas de los sistemas generativos como el prompt engineering o el fine tuning nosotros nos basaremos en las técnicas RAG.

[1] https://openai.com/index/chatgpt/

2.6. RAG

Este tipo de técnicas RAG (Retrieval-Augmented Generation) (Lewis et al., 2020) nos permiten mejorar la actuación de los modelos de lenguaje generativos al integrar técnicas de recuperación de información con procesos generativos (Arslan et al., 2024). Esto permite mejorar los problemas mencionados anteriormente gracias a la adicción de información externa adicional (Gao et al., 2024), mitigando las alucinaciones que se suelen generar al realizar preguntas especificas de un determinado dominio. De esta forma , podemos distinguir entre diversos tipos de RAGs:

2.6.1. Naive RAG

Este tipo de RAGs es el más básico de todos, basándose en un proceso que incluye la indexación, la devolución y la generación. Veamos brevemente cada una de estas fases:

Indexación

Esta parte se basa en la limpieza y extracción del texto plano a partir de diversos formatos como PDF o HTML.

Debido a las limitaciones de contexto de los LLM, este suele dividirse en pequeños trozos de código que puedan ser más digeribles para los LLM. Nótese, como el modelo tiene un contexto o context window limitado debido a los mecanismos de atención que se usan en los modelos de lenguaje. Estos limitan la ventana de contexto al número de tokens a los que pueden prestar atención simultáneamente debido a los grandes costes en memoria que esto requiere (Naveed et al., 2024). Estos tokens, no son más que las unidades mínimas del modelo, que pueden ser desde caracteres, hasta palabras completas o incluso una cantidad fija, por ejemplo 3 caracteres que pueda únicamente incluir algunas partes de las palabras, al igual que para nosotros las unidades mínimas son los caracteres o las palabras, para el modelo la unidad mínima serán los tokens.

De cara a las implementaciones prácticas, es necesario notar que tanto los elementos de system prompt como los documentos pasados mediante RAG, como la pregunta del usuario o el historial del chat, hacen uso de esta context window, lo que lleva a que los primeros comentarios de la conversación se vayan olvidando a medida que esta avanza, ya que el modelo se va quedando sin capacidad en la context window y va teniendo que eliminar de la memoria los elementos antiguos.

Así, es importante notar como la fragmentación (chunking) es un paso crucial de pre-procesamiento en los sistemas RAG, por su impacto significativo en la efectividad de la recuperación de los datos (Bhat et al., 2025). Entre las diversas técnicas de fragmentación, la fragmentación basada en tokens de tamaño fijo, la cual se basa en dividir los documentos de forma uniforme en chunks con el mismo número de tokens, sigue siendo un método predominante debido a su

simplicidad y facilidad de implementación.

Vemos por tanto, como la eficacia de la fragmentación de tamaño fijo puede verse influenciada por factores como la dispersión de la información entre fragmentos, la redundancia en la recuperación y la precisión de la misma, resultando clave la comprensión de estos factores para optimizar el rendimiento de los sistemas RAG.

En base a los resultados de (Bhat et al., 2025), podemos observar como la fragmentación más pequeña (64-128 tokens), funciona mejor para conjuntos de datos con textos breves y basados en hechos, mientras que las fragmentaciones más grandes (512-1024 tokens) son necesarias para conjuntos de datos con texto descriptivo o técnico.

Téngase además en cuenta, como chunks más largos capturaran más contexto al mismo tiempo que generaran más ruido requiriendo un mayor tiempo de procesamiento y por tanto un mayor coste. Mientras que los chunks más cortos pueden no reflejar completamente el contexto, aunque presentan menos ruido. Asimismo, para evitar perdida de información en las fronteras de los chunks suele añadirse un parámetro de overlap, que indica el solapamiento entre varios chunks para evitar esta pérdida de información (Verma, 2025).

Nótese, que aunque es la más común por su simpleza, no es la única, ya que existen técnicas como la división del texto en base al contenido semántico del mismo (Galchonkov et al., 2025), la cual permite mantener la coherencia semántica durante la división del texto en los diferentes chunks. Esto permite que la longitud de la ventana de contexto pueda ser variable en función de la semántica del texto permitiendo que se guarden en un mismo chunk párrafos enteros o frases enteras, evitando así que se parta una frase por la mitad durante la división.

Por último, estos chunks son codificados a una representación vectorial mediante el uso de modelos de embedding y guardados en una base de datos.

Estos modelos de embedding se basan en la conversión del texto a vectores numéricos (Almeida and Xexéo, 2019). Por tanto, vemos como son representaciones aprendidas de los textos a vectores de forma que se obtengan vectores densos, es decir, con pocos 0 (al contrario que ocurría con técnicas one-hot) reduciendo así la dimensionalidad de la representación. Viendo como estos vectores representan la similitud semántica entre dos elementos mediante la distancia existente entre ambos. Esta distancia entre dos elementos para determinar su similitud se basa en la distancia textual, es decir, en la similitud con respecto a los elementos(tokens) que componen los textos y esta se puede medir en base a las siguientes formas según (Wang et al., 2020):

- Distancia Longitud. En este tipo se suele medir la distancia como la longitud entre vectores. A su vez, dentro de esta a su vez podemos distinguir:
    - Distancia Euclidea. Mide la distancia en línea recta entre dos puntos del espacio los cuales son representados por vectores.

- Ditancia coseno. En este caso, lo que se mide es el ángulo entre dos puntos. Así, si se obtiene un ángulo pequeño, este representa cercanía en cuánto a similaridad textual.
- Distancia de Manhattan. Esta suele ser muy útil cuando la distancia geométrica no es relevante y los datos se encuentran representados en rejillas. Así, este tipo de medida se calcula como la suma de las diferencias absolutas entre dos vectores.
- Distancia Hamming. Este tipo de medida permite comparar dos datos binarios mediante la medición del número de posiciones (bit) en los que ambos obtienen valores diferentes.

Este tipo de medidas aunque muy útil a la vez que fácil de calcular, tiene diversos problemas:

- El primero de ellos es que es muy útil para problemas simétricos. Sin embargo, los conjuntos de pregunta-respuesta no suelen ser simétricos por lo que no obtienen resultados tan buenos respecto a la similitud de los elementos.
- El segundo es el riesgo intrínseco de juzgar la cercanía en base a estas medidas, sin conocer distribuciones estadísticas de los datos, lo que puede estar generando modificaciones en este tipo de medidas.

- Distancia distribucional. Se usa para determinar si los documentos provienen de la misma distribución. Así, los métodos más frecuentes para el estudio de esta medida son:
  - Divergencia JS(Jensen-Shannon). Aquí lo que hace es medir la similaridad entre dos distribuciones de probabilidad. También recibe el nombre del radio de información (IRAD).
  - Divergencia KL(Kullback-Leibler). Mide los diferentes grados de una distribución de probabilidad siendo también una medida de la entropia presente en una distribución de probabilidad.
  - Distancia Wasserstein. Es una medida de la distancia entre dos distribuciones de probabilidad. En caso de que haya muy poco solapamiento entre las distribuciones esto toma valores infinitos.

- Distancia semántica. Cuando no existen palabras comunes entre los diferentes documentos, tanto la similitud por distancia como por distribución puede ser muy pequeña. Por ello, esta distancia se basa en el significado de la palabra dentro de la frase midiendo la distancia a partir del mismo y no de su aparición o no en la frase. Dentro de esta podemos considerar las siguientes medidas:
  - Distancia Word Mover's. Mide la mínima distancia que necesita una palabra en un texto para llegar a otra palabra en otro texto en el espacio semántico. Por tanto, lo que mide es como de intercambiable sería una palabra por otra en otro documento.
  - Extensión Distancia Word Mover's. En vez de usar la distancia Euclidea en la que cada dimensión del espacio tiene el mismo peso, usa la distancia de Mahalanois para tener en cuenta la correlación entre las diferentes dimensiones.

Retrieval

Tras recibir la pregunta por parte del usuario, el sistema RAG emplea el mismo modelo de codificación que el usado durante la fase de indexación, para obtener la representación vectorial de la pregunta. A continuación, se calculan las medidas de similaridad entre el vector de la pregunta y los vectores de cada uno de los chunks. Tras lo cual el sistema devolverá los K chunks que hayan obtenido mayor similitud con respecto a la pregunta realizada por el usuario. Por último, estos chunks son usados para añadir contexto al modelo durante la generación de contenido.

Generation

La pregunta realizada, así como los documentos seleccionados, se sintetizan para formar un prompt coherente para un LLM, al cual se le ha requerido como tarea dar una respuesta. Así, en función de la tarea que tenga que realizar el LLM, puede requerirsele solo que se centre en la información dada o por el contrario, puede tener mayor libertad para usar su conocimiento base. Además, si estos LLM se usan en el transcurso de un diálogo, cualquier historial conversacional puede integrarse en el prompt permitiendo al modelo que se integre en una conversación multi-turno de forma efectiva.

En esta parte, podemos encontrar problemas como la redundancia o el sesgo implicito debido a los documentos devueltos en el proceso de retrieval, lo que puede empeorar la calidad de la respuesta. Asimismo, estos modelos generativos pueden caer en la trampa de devolver la misma información que han recibido sin darle apenas coherencia, sólo sintetizando brevemente su contenido.

2.6.2. Advanced RAG

Para superar la limitaciones del tipo anterior, este tipo incluye dos mejoras en el proceso de retrieval.

Proceso pre-entrenado

En esta etapa, se centra en optimizar la estructura de indexación y la pregunta original. Esto se consigue primero mejorando la calidad del contenido a indexar, por ejemplo, añadiendo metadatos, y en segundo lugar haciendo más clara la pregunta del usuario, por ejemplo, reescribiéndola o expandiéndola.

Proceso Post-Retrieval

Una vez que la información relevante ha sido devuelta, ahora el paso fundamental es su correcta integración dentro del prompt. Para ello, suelen usarse métodos como el re-ranking, para poner la información más relevante en los bordes del prompt, o la compresión de contexto. Hay que destacar que el re-ranking es útil, ya que en artículos como en (Liu et al., 2023) y (An et al., 2024), se puede observar como los modelos grandes de lenguaje suelen perder la información que se encuentra en el medio de instrucciones largas, mientras que suelen centrarse en aquellas que se encuentran al principio o al final. Este hecho,

lo remarcan de forma significativa al obtener mejor rendimiento en aquellos experimentos cuyas instrucciones incluían el contenido relevante al principio o al final, aunque el contenido de las instrucciones fuese el mismo que el de aquellas instrucciones que incluían esta información en el medio.

2.6.3. Modular RAG

Este método, obtiene una mayor versatilidad respecto a los anteriores. Esto se hace debido a la adicción de nuevas componentes así como a la mejora de las existentes. Algunos de los nuevos módulos que se implementan son:

- Componente de búsqueda. Permitiendo una adaptación a diferentes tipos de datos
- RAG-Fusión. Usando múltiples preguntas para expandir la del usuario
- Componente de memoria. Este guía la recuperación de documentos utilizando para ello la memoria recolectada a partir de la interacción con el usuario.

Vemos como el proceso en el que se basa está técnica consta de tres partes bien diferenciadas, cada una de las cuales trabaja en sinergia con las otras dos. Estas serán descritas a continuación por la importancia que tienen para el buen funcionamiento del sistema:

2.6.4. Componentes de un sistema RAG

Retrieve

En este paso, se emplean sofisticadas técnicas de recogida de información para buscar y extraer toda la información relevante de fuentes externas. Estas fuentes externas pueden ir desde bases de datos hasta la propia web al completo. Normalmente, suelen usar representaciones densas de vectores tanto para la pregunta como para el corpus de conocimiento. Además, dentro de este componente tendremos dos partes bien diferenciadas:

- La indexación, cuya función es la de organizar los documentos para facilitar la devolución. En general se suele usar índices invertidos o una codificación vectorial densa.
- La componente de búsqueda, que utiliza los índices anteriores para extraer los documentos relevantes para la pregunta del usuario.

Augmentation

Esta fase consiste en la unión del conocimiento externo con el contexto y entendimiento que tiene ya el modelo sobre la pregunta. Algunas técnicas usadas para este proceso serían las técnicas de atención o la fusión neuronal de información.

Generate

Por último, el modelo producirá una salida basándose en sus patrones aprendidos y el conocimiento añadido en las fases anteriores. Esto es posible gracias al uso de grandes modelos de lenguaje al mismo tiempo que usamos el contexto obtenido. Obteniendo así una información más relevante, actualizada y coherente.

Podemos observar que cada una de estas fases, tienen una serie de elementos importantes los cuales necesitamos tener en cuenta.

2.6.5. Aspectos dentro de cada componente

Retrieval

En los sistemas RAG es crucial devolver la información relevante a partir de las fuentes de información. De esta forma, tenemos que tener en cuenta diversos aspectos en este ámbito:

- Fuentes de recuperación. En este caso, se basa en la necesidad de tener en cuenta tanto la estructura de los datos como la granularidad de los mismos, ya que ambos elementos pueden afectar al proceso de recuperación. Un ejemplo de ello sería los datos semi-estructurados, como los PDF, que pueden originar problemas en la devolución del contenido de tablas.

- Optimización de la indexación. En esta fase, los datos serán procesados, segmentados y transformados en embedding para guardarlos en una base de datos vectorial. La corrección de la construcción de este índice determinará la correcta obtención de contexto durante la fase de retrieval. Por ello, aquí se recogen tres aspectos a tener en cuenta:

  - Estrategia de chunk. Esta determina el como se va a dividir el documento. En general, la forma más común tal y como comentábamos anteriormente es mediante el uso de un tamaño fijo.
  - Adicción de metadatos. Los chunks pueden enriquecerse con información como el número de página, la fecha, el autor, etc, lo que permite que durante la recuperación se filtre la información en base a estos metadatos. Estos metadatos pueden extraerse de los documentos originales o bien generarse de forma artificial.
  - Estructura del índice. Esta se basa en establecer relaciones jerárquicas entre los diferentes chunks generados a partir de los documentos.

- Optimización de la pregunta. Uno de los problemas principales de los RAG es su dependencia en las preguntas del usuario. Estas preguntas pueden ser complejas, estar mal organizadas e incluso presentar ambigüedad en ciertos términos, como en abreviaturas, lo que empeora la efectividad del sistema de recuperación. Para mitigar este problema se puede realizar alguna de las siguientes técnicas:

  - Expansión de la pregunta. Se basa en expandir una pregunta a múltiples preguntas enriqueciendo el contexto para evitar problemas con distintos matices. Esto a su vez se puede basar en dos tipos:

        - Multi-query. En este caso se puede usar el prompt engineering para expandir una pregunta. El prompt engineering tal y como se describe en (Too et al., 2019), (McTear and Ashurkina, 2024), se basa en la redacción eficiente del texto que se le pasa al LLM, para obtener buenas respuestas del mismo. Entre los elementos que tienen en cuenta este tipo de técnicas, podemos encontrar una descripción del problema, el rol que debe tomar el sistema, la audiencia objetivo, el tono de voz, etc.
        - Sub-query. Genera preguntas necesarias para contextualizar la respuesta a la pregunta original. Se trata de hacer que el sistema complete los pasos intermedios que el usuario omitió, con el fin de proporcionar una respuesta completa y adecuada a la pregunta.
        - Chain-of-Verfication. Se basa en verificar las preguntas adicionales para evitar alucinaciones por parte del LLM.
    - Transformación de la pregunta. La idea principal se basa en devolver los chunks respecto a una pregunta transformada y no respecto a la pregunta original del usuario. Aquí podemos distinguir varias técnicas:
        - Reescritura de la pregunta, la cual se puede hacer mediante un LLM que reescriba la pregunta original, para que sea mejor para la búsqueda
        - Generación de pregunta, que se basa en la creación de una pregunta a partir de la original permitiendo al modelo abstraerse a un nivel superior de concepto.
    - Enrutamiento de la pregunta, el cual se basa en extraer la información importante de la pregunta del usuario. Así, esto se puede hacer a dos niveles distintos:
        - Metadata, que consiste en extraer las palabras principales de la pregunta, para poder filtrar a partir de esto, en los metadatos de nuestra base vectorial.
        - Semantic, en este caso se basa en extraer la información semántica de la pregunta.

- Embedding. La representación que hagamos de los datos también es relevante en el proceso de recogida de los mismos, ya que esta puede tener un impacto en la similaridad de los elementos. Por ello, podemos distinguir varias formas de representación de los datos:
    - Mixta. Se pueden utilizar una representación densa para la recogida de la información y una posterior representación dispersa. Esto nos permite recoger diferentes características, permitiendo un beneficio, ya que cada uno de estos modelos recogería información complementaria.
    - Fine-tuning. En aquellos ámbitos en los que la temática general se aleja mucho de los temas en los que el modelo ha sido pre-entrenado, por ejemplo, en disciplinas muy especializadas como la medicina, adaptar el modelo a estos datos es fundamental. Esta adaptación se hace mediante el fine-tuning, que no es otra cosa que el recálculo de algunos

pesos para adaptarse a los nuevos datos. Así tal y como se indica en (Too et al., 2019), el fine-tuning es un concepto del Transfer Learning, el cuál se basa en utilizar el conocimiento adquirido durante el entrenamiento en un tipo de problema para otra tarea, lo cual se consigue eliminando las últimas capas de la red entrenada y volviéndolas a entrenar en base a los datos necesarios para poder resolver nuestra tarea objetivo.

Augmentation

Habitualmente, en el proceso del RAG, suele haber un solo proceso de recuperación de documentos seguido de un proceso de generación, lo que resulta generalmente en ineficiencias al tratar con problemas complejos. Para evitar este problema, se puede hacer uso de los siguientes elementos:

- Recuperación iterativa. En este caso, la base de conocimiento se analiza en busca de documentos de forma recursiva a partir de la pregunta y la respuesta generada por el modelo.
- Recuperación recursiva. Este proceso se basa en iterativamente ir mejorando la pregunta de búsqueda en base a los resultados obtenidos a partir de búsquedas previas. Esto lleva a que el modelo de búsqueda converja a la información más relevante para la pregunta.
- Recuperación adaptativa. Se basa en el uso de la recuperación por parte del modelo cuando este lo crea conveniente.

Generation

Tras obtener la información no suele ser recomendable darle todo esto como input al LLM, así en general es recomendable realizar los siguientes pasos:

- Curación del contexto, para eliminar la información redundante, ya que esta puede interferir en el proceso de generación. Asimismo, al igual que los humanos, los modelos suelen centrarse solo en la información al principio y al final de textos largos. Para poder mitigar estos problemas es necesario usar las siguientes técnicas:
  - Re-ranking. Esto lo que hace es re-ordenar los chunks para subrayar los más importantes.
  - Selección/compresión del contexto. El hecho de concatenar tantos documento como sea posible no es beneficioso, por lo que este proceso se basa en eliminar tanto los tokens, como los documentos que no son relevantes, quitando ruido de los elementos que le pasamos al modelo.
- Fine-tuning. Podemos proveerle de datos en los que falla mediante el fine-tuning permitiendo que tenga una mayor capacidad para responder a cuestiones concretas

Se puede observar como los sistemas RAG se caracterizan por tener una arquitectura modular, lo que permite una continua mejora de cada una de las partes sin tener que modificar las otras.

2.7. ¿Qué aporta el RAG a los LLM?

Veamos ahora por qué los sistemas RAG son capaces de mejorar el rendimiento de los modelos LLM, en los problemas que se les suelen achacar.

En primer lugar, respecto a la relevancia de la respuesta ofrecida, los LLM tradicionales se basan solo en los patrones aprendidos durante el entrenamiento mientras que los sistemas RAG pueden acceder e incorporar información actualizada de fuentes externas. Esto es posible gracias a los dos primeros módulos que componen el sistema RAG, que se basan en la búsqueda de la información en fuentes externas y la incorporación de dicha información a la ya presente en el modelo. Está búsqueda externa de información permite asegurarnos la generación de un contenido que sea relevante y preciso, incluso si tratamos con temas que evolucionan rápidamente, mejorando así el problema de la relevancia de la respuesta generada que tenían los LLM.

En segundo lugar, los sistemas RAG permiten generar respuestas más creativas y profundas que los LLM tradicionales. Esto se debe al mayor acceso a la información con una gran variedad de puntos de vista diferentes, permitiendo con ello al modelo realizar nuevas conexiones entre trozos de información dispersos, llegando así a soluciones novedosas. Por ejemplo, si le pedimos que nos genere ideas sobre desarrollo urbano sostenible, el sistema RAG podrá mezclar información desde ecologismo hasta las últimas tecnologías, pasando por planes de urbanismo, generando así una respuesta más creativa al ser más rica en su contexto. Vemos por tanto, como la mejora de esta relevancia y creatividad permite una mayor habilidad al sistema para adaptarse a un contexto especifico o a las necesidades de los usuarios. Así, el sistema será capaz de adaptar el lenguaje a uno más sencillo si esta tratando con un principiante en algún tema, mientras que por el contrario, será capaz de mantener una conversación más técnica con un experto.

Por último, en cuanto a la credibilidad de los modelos, esta se puede decir que es uno de los problemas más conocidos, debido a las alucinaciones que generan algunos modelos. Este problema puede abordarse mediante el uso de sistemas RAG y mejorarse al basar las respuestas en información real mejorando por tanto, la credibilidad y precisión del modelo. Así, el sistema RAG mitiga este problema mediante el uso de la componente retrieval, viendo como al recibir una pregunta, el sistema antes de generar una respuesta, realiza una búsqueda sobre la que basa la generación de contenido, reduciendo significativamente la probabilidad de que se produzcan frases falsas o inconsistentes. Además, la parte de augmentation permite una integración más matizada de la información recopilada, permitiendo así que el modelo la contextualice mejor, lo que ayuda a evitar una mala interpretación o aplicación de los datos, llevando por tanto, a que esta información se presente en un contexto coherente y contextualmente

apropiado. Asimismo, hay que resaltar que es posible personalizar los sistemas RAG para conseguir que estos prioricen fuentes de autoridad o unas ciertas fuentes antes que otras, lo cual es muy útil en contextos médicos o legales, mejorando también la credibilidad del contenido generado.

Téngase en cuenta que la mejora de la credibilidad del modelo va aparejada con un aumento de la transparencia y la explicabilidad del mismo. Esto se debe a que el modelo basa sus respuestas en la información devuelta por lo que podemos realizar la trazabilidad de dichas fuentes en busca de determinadas sentencias o datos. Por último, esta incorporación de información externa variada también ayuda a reducir el sesgo generado por el conjunto de entrenamiento al incorporar mayor contexto e información. Esto permite incorporar información más balanceada, incluso a preguntas complejas, gracias a la incorporación de múltiples perspectivas y fuentes.

2.8. ¿Qué problemas tiene el RAG?

2.8.1. Privacidad

Hemos podido ver como los sistemas RAGs añaden diversas mejoras en los LLM. Sin embargo, estos sistemas aunque mejoran algunos problemas como las alucinaciones, también traen otros problemas consigo. En concreto, uno de los más importantes es el riesgo a la seguridad que pueden generar este tipo de sistemas. Por ello, en esta sección presentamos algunos de estos riesgos y como mitigarlos (Ammann et al., 2025):

- Retrieval Data Leak. Sucede cuando la información del conjunto de datos es devuelta a usuarios sin permisos.
- Retrieval Data Disclosure during embedding. Se refiere a la exposición de datos a usuarios no autorizados durante el proceso de embedding.
- Retrieval Data Disclosure during prompting. Se refiere a la exposición de datos a usuarios no autorizados durante el proceso de prompting.

  Los tres problemas anteriores pueden mitigarse mediante una limitación de acceso, de forma que cada uno de los usuarios necesitan tener una autentificación que limite sus derechos de acceso solo a ciertos documentos.
- Embedding inversion attack. Se centra en tratar de reconstruir el conjunto de datos original a partir de la representación vectorial (embedding).
- Membership inference attack. Se intenta determinar si se incluyeron datos específicos en el conjunto de datos de recuperación utilizado para alimentar el almacén de datos, lo que puede usarse para determinar si se usó la información de un determinado usuario para la ingesta.

  Los dos puntos anteriores, pueden mitigarse mediante la adicción de ruido lo que permitiría ajustar los embedding insertando caracteres aleatorios lo que dificultarían la obtención de los datos originales aún cuando se pueda acceder a los embedding.

- Prompt disclosure. Ocurre cuando información sensible es incluida en el prompt que se transmite a un LLM localizado en la nube.

  Una forma de mitigar este problema es mediante la anonimización irreversible de los datos, eliminando información personal. En los casos en los que la información sensible sea necesaria para obtener buenas respuestas, podemos usar la pseudonimización la cual consiste en la sustitución de información sensible por seudónimos. Asimismo, también se pueden evitar este problema usando un LLM en local, lo que permite el control del flujo de los datos, facilitando el mantenimiento de las medidas de seguridad.

- Knowledge Corruption Attack. En este caso, intenta comprometer el proceso de respuesta del RAG manipulando su sistema de retrieval.

- Indirect prompt injection. Se refiere a la manipulación del comportamiento del generador mediante mensajes maliciosos consiguiendo que se ejecute un comportamiento no deseado, acciones no deseadas o la producción de resultados dañinos.

  Este problema se puede mitigar también, mediante el resumen del prompt o del prompt y de los datos devueltos, lo que reduce el riesgo de la aparición de datos sensibles y mitiga el daño de las posibles instrucciones maliciosas al comprimirlas, evitando así posibles frases pensadas expresamente para conseguir información del sistema.

- Indirect Jailbreak Attack. Estos ataques se basan en saltarse las medidas de seguridad para manipular el generador, al darle contenido perjudicial.

  Los tres últimos problemas, se pueden mitigar mediante el uso de la validación del input, de forma que antes de llegar al RAG, se implementa un pipeline que rechace los prompts o instrucciones del usuario que contengan información o instrucciones maliciosas.

  Además de todo esto, también se pueden usar otras técnicas como reranking para re-ordenar los documentos obtenidos en el retrieval en función de la relevancia al prompt del usuario, con el fin de reducir el tamaño de los datos expuestos a un mal uso, al mismo tiempo que promueve la relevancia y precisión de los datos. Además, para evitar que el propio usuario comparta datos sensibles sin ningún tipo de conocimiento al respecto, pueden crear pipelines (Locci et al., 2025) que avisen al usuario de que la entrada contiene datos sensibles, permitiendo así un uso más seguro del LLM desde el principio.

Estos son algunos de los posibles problemas que se pueden encontrar durante el uso de los sistemas RAGs. Sin embargo, hay que tener en cuenta que no son los únicos, ya que estos sistemas tal y como comentábamos al inicio, añaden algunos de los problemas originales de los LLMs a los suyos propios. Sin embargo, hemos tratado diversas técnicas que ayudan a la mitigación de estos problemas.

2.8.2. Otros puntos de fallo

Durante el apartado anterior, hemos podido observar como este tipo de sistemas añaden nuevos problemas a la privacidad del sistema. Aparte de estos

problemas, que en general tienen que ver con cómo responde el sistema a requerimientos maliciosos del usuario, en esta subsección nos centraremos también en cómo responde el sistema a otro tipo de requerimientos del usuario. Así, podemos encontrar los siguientes puntos de fallo, tal y como se indica en (Barnett et al., 2024) y (Genesis, 2025):

- Contenido faltante. Ocurre cuando se le hace una pregunta al sistema que no puede ser respondida a partir del contenido del mismo. Lo mejor que podría contestarnos en este caso es, que no lo sabe o que desconoce la respuesta. Sin embargo, para aquellas preguntas que están relacionadas con algún contenido existente pueden dar una respuesta aunque no se tenga la información para poder darla.

- Pérdida en los documentos mejor clasificados. En este caso, la respuesta a la pregunta existe en un documento. Sin embargo, este documento no obtiene la puntuación necesaria para ser devuelto al usuario entre los K mejores por lo que nunca llega.

- Fuera de contexto. Ocurre cuando entre los documentos que se devuelven de la base de datos está el que contiene la información, pero posteriormente se hace algún proceso de re-ranking en el que se eliminan los documentos que no se creen necesarios haciendo que el documento que contiene la información no sea pasado como contexto al modelo generativo.

- No extraído. En este caso la respuesta está dentro del contexto que se le pasa al modelo de lenguaje, sin embargo, este falla al extraer la respuesta. Además, también puede ocurrir que el modelo tenga problemas a la hora de sintetizar la información que ha recibido. Esto ocurre generalmente cuando el contexto es muy ruidoso, es decir, hay demasiada información que no aporta nada o cuando existe información contradictoria.

- Incompletitud. Se da cuando se generan respuesta incompletas aún teniendo la información disponible en el contexto. Esto se suele dar generalmente cuando el usuario plantea varias preguntas simultáneamente.

- Demasiada confianza en el conocimiento interno. Ocurre cuando los modelos generativos ignoran la información devuelta durante la recuperación centrándose en su conocimiento interno, lo lleva a información desactualizada y con problemas de alucinaciones.

- Formato incorrecto. Cuando el usuario requiere la información en un formato concreto, por ejemplo una tabla y el modelo generativo ignora esta petición.

- Especificidad incorrecta. La información se devuelve en la respuesta pero de forma muy general, sin aportar detalles específicos. También puede ocurrir que genere respuestas demasiado específicas con respecto a los requerimientos del usuario. Todo esto puede ocurrir cuando los usuarios no tienen mucha idea sobre un tema y hacen una pregunta general, ya que no saben como plantearla.

2.9. Evaluación del RAG

El principal objetivo de esta evaluación es optimizar la actuación de los modelos RAG. Así, los principales elementos que se tienen en cuenta en la evaluación de este tipo de sistemas, tal y como se describe en (Yu et al., 2025b), (Gao et al., 2024) y (Yu et al., 2025a), son:

- Calidad de la recuperación, la cual es crucial para determinar la efectividad del contexto dado al modelo.
- Calidad de la generación. Esta se basa en determinar la capacidad del modelo para crear respuestas sintetizadas, coherentes y relevantes en base al contexto aportado mediante la recuperación.
- Calidad del sistema en su conjunto, la cual se basa en como interaccionan ambas partes para obtener un resultado final, lo que es crucial, ya que el resultado de una de ellas afecta a la otra.

Veamos ahora los principales retos que afrontan cada una de estas partes:

2.9.1. Recuperación

Esta es una parte crucial del proceso que afronta su mayor dificultad en la vasta y dinámica naturaleza de las que pueden ser las bases de datos de referencia. Esto hace que sean necesarias métricas que evalúen la precisión, el recall y la relevancia de los documentos devueltos. Además, cuando la información es temporal, la relevancia de las mismas puede cambiar a lo largo del tiempo. Asimismo, esta diversidad lleva a la posibilidad de devolver información de peor calidad. En este caso, tenemos dos elementos principales que evaluar, por un lado la relevancia, que evalúa cómo de bien los documentos devueltos responden a las necesidades expresadas por la pregunta y por otro lado, la exactitud, que mide cómo de precisos son los documentos devueltos con respecto a otros posibles candidatos .

Veamos a continuación cuáles son las métricas más comunes para medirlo:

- Accuracy. Es la proporción de casos positivos entre todos los resultados evaluados
- Precision. Es la proporción de instancias relevantes entre todas las devueltas
- Recall. Representa la proporción de todas las instancias relevantes devueltas frente a todas las existentes.
- Mean Reciprocal Rank. Determina el rango o posición donde se suele encontrar la primera respuesta correcta en media.
- Mean average precision. Es la media de las puntuaciones de precisión para cada pregunta.

Nótese como este tipo de evaluaciones en el estado del arte actual usan en general LLM as a judge para medir cada uno de los elementos presentados anteriormente.

2.9.2. Generación

Aquí el reto consiste en medir la fidelidad y precisión del contenido generado con respecto a la información de entrada. Esto no solo implica la corrección de la respuesta, si no que también mide su relevancia con respecto a la pregunta original y la coherencia con el texto generado. Podemos observar como tenemos por tanto tres elementos principales que evaluar:

- Relevancia. Mide cómo de bien se alinea la respuesta generada con la intención y el contenido original de la pregunta.
- Fidelidad. Mide si la respuesta generada refleja la información contenida en los documentos relevantes. Además, también mide la consistencia entre la respuesta generada y las fuentes de información.
- Corrección. Esto, es similar a la exactitud medida en el sistema de recuperación y lo que determina es la precisión de la respuesta devuelta en base a las posibles respuestas que podría haber generado.

Veamos a continuación cuáles son las métricas más comunes para medir los elementos anteriores:

- ROUGE. Evalúa la calidad de unos resúmenes comparándolos con los que haría un ser humano
- BLEU. Evalúa la calidad de la traducción comparando la precisión de los n-gramas generados con la del texto de referencia.
- BertScore. Evalúa la similaridad entre el texto generado y el texto de referencia.
- LLM as a Judge, en este caso, los LLM se usan para darle una puntuación al texto generado en base a su coherencia, relevancia y fluidez. Asimismo, se le puede indicar el rango de puntuación para obtener un proceso de evaluación estandarizado.

2.9.3. El sistema completo

Se debe estudiar el sistema completo, ya que la interconexión entre ambas partes implica que no se puede analizar cada una de ellas aisladamente. Por tanto, es necesario medir el valor añadido por los documentos recuperados al proceso de generación. Así, los principales aspectos que tienen en cuenta los RAGs actuales para la evaluación son:

- Métricas de calidad, que permite evaluar la eficiencia del RAG desde diversas perspectivas. Dentro de ellas podemos considerar:
  - Latencia. Mide cómo de rápido genera las respuestas el sistema.
  - Diversidad. Mide si el sistema devuelve una variedad de documentos relevantes y genera respuestas diversas.
  - Relevancia del contexto. Mide la precisión y especificidad del contexto devuelto, lo que asegura que estos sean relevantes.

    - Fidelidad de la respuesta. La cual se asegura de que las respuestas devueltas sean ciertas en base al contexto devuelto, manteniendo la consistencia y evitando contradicciones, es decir, el modelo diga una cosa y la información devuelta diga otra.
    - Relevancia de la respuesta. Mide que la respuesta dada por el modelo este directamente relacionada con la pregunta realizada por el usuario.

- Habilidades requeridas. Estas habilidades son necesarias tenerlas en cuenta ya que son buenos indicadores de la capacidad del RAG de adaptarse y de su eficacia. Entre ellas tenemos:

    - Robustez ante ruido. Mide la capacidad del modelo de manejar documentos que aunque son relevantes con respecto a la pregunta, carecen de información importante.
    - Rechazo negativo. Determina la capacidad del modelo de abstenerse de responder cuando los documentos devueltos no contienen la información necesaria para responder a la pregunta.
    - Integración de la información. Evalúa la capacidad del modelo para sintetizar la información para contestar a preguntas complejas.
    - Robustez Counterfactual. Pone a prueba la capacidad del modelo para reconocer y descartar inexactitudes conocidas en los documentos, incluso cuando se le instruye sobre posible desinformación.

Veamos a continuación cuáles son las métricas más comunes para medir algunos de los anteriores elementos:

- Single query latency. Mide el tiempo medio que tarda el sistema en responder, tanto la parte de retrieval como la generativa, a una sola pregunta del usuario.

- Similaridad coseno. Esta es una medida entre los embeddings. Así, un menor valor indicaría una mayor diversidad.

- Mileading Rate y Mistake reapperance rate. Permiten medir como el sistema maneja la información irrelevante.

- Rejection rate. Es básicamente la tasa a la que el sistema se abstiene de generar una respuesta.

- Error detection rate. Representa la proporción de afirmaciones que ignoran los hechos o información detectadas en la respuesta.

Debido al reciente uso de técnicas como los LLM as a Judge para la evaluación de los sistemas RAG, vamos a explorar a continuación en qué consiste esto y qué problemas trae consigo.

2.9.4. LLM as a Judge

Este tipo de técnicas de evaluación, surgieron tal y como se describe en (Zheng et al., 2023), (Szymanski et al., 2025) y (Gu et al., 2025), debido al gran coste tanto humano como temporal que suponía una evaluación de los modelos en base a clasificaciones realizadas por humanos. Asimismo, el uso de estos modelos puede incluso eliminar la necesidad de generar respuestas de referencia para medir el grado de similitud o medir dicho grado teniendo en cuenta otros aspectos, a parte del sintáctico como el semántico.

Vemos por tanto, como hemos podido llegar a este uso de los modelos gracias a su creciente habilidad para emular la capacidad humana para razonar y evaluar una serie de entradas en base a una serie de reglas. Todo esto trae consigo dos beneficios principales, según (Zheng et al., 2023):

- Escalabilidad. Al reducir la intervención humana conseguimos que se haga de forma más rápida las comprobaciones e iteraciones permitiendo que sea mucho más escalable.
- Explicabiliad. Además de las medidas, los modelos pueden presentar explicaciones del porque de su elección, lo que permite hacer estas notas más explicables.

Así, podemos distinguir los siguientes tipos de LLM as a judge:

- Calificadores.
- Evaluadores/asesores.
- Críticos.
- Verificadores.
- Examinadores.
- De ordenación de modelos.

Visto un breve repaso, expliquemos ahora como suelen usarse y buenas prácticas en su uso para conseguir que sean fiables.

Modo de uso

Comencemos pues por el modo de uso habitual de este tipo de modelos. Así, este se centra en el In-context learning (ICL) para darle instrucciones y ejemplos al modelo para guiar su razonamiento. Este proceso, implica tener en cuenta dos elementos:

- Diseño del input. En este punto se tienen en cuenta aspectos relevantes como el tipo de variable que se va a evaluar y la forma y la posición de entrada de la información.
- Diseño del prompt. Para lo cual tenemos que distinguir cuatro posibles problemas que podemos querer que evalúe:

- Generación de calificaciones. Se basa en representar una evaluación por medio de una nota. El problema o matiz importante a tener en cuenta es el rango de calificación. Estos rangos pueden ser discretos o continuos. Además, lo anterior se puede complicar en función de los criterios que se pidan evaluar al modelo, pudiendo ser desde un único criterio a varios. Esta puede extenderse a lo conocido como Reference-guided grading, la cual se basa en determinar la nota de una contestación en base a una respuesta de referencia. Lo cual es útil para determinar cómo de buena ha sido la respuesta a problemas matemáticos.
- Responder preguntas de verdadero/falso. Este tipo en general se basa solo en el accuracy (exactitud) del modelo. Además, debido a que solo puede devolver dos posibles valores sin poder obtener una mayor explicación, suele usarse en pasos intermedios para dar retroalimentación al modelo, permitiendo entrar en nuevos ciclos en función de determinados criterios, como si necesita reescribirse o no.
- Realizar comparaciones dos a dos. En este caso, se basa en determinar cuál de dos opciones es la mejor o si ambas son iguales. Esta puede hacerse subjetivamente o en base a un criterio objetivo. Un ejemplo de ello sería determinar cuál de dos respuestas a una pregunta es mejor, es decir, cuál responde mejor a la pregunta.
- Hacer elecciones entre múltiples opciones. Este caso puede considerarse una generalización del anterior, en el que el modelo tiene que elegir entre de varias opciones cuál es la mejor.

2.9.5. Limitaciones de los LLM as a judge

Estos sistemas aunque presentan grandes avances con respecto a la métricas anteriores, también presentan una serie de limitaciones. Algunas de ellas son:

- Position bias. Esta limitación se basa en la preferencia del sistema por unas posiciones frente a otras. Téngase en cuenta como este sesgo se encuentra presente también en los seres humanos y en su toma de decisiones. Así, textos que resulten más atractivos para un profesor obtendrán una mayor nota que aquellos que no, aunque el contenido sea esencialmente el mismo.
- Verbosity bias. Esto se refiere a la preferencia que tienen los LLM por respuestas largas y con muchas palabras incluso si no resultan claras o de gran calidad.
- Self-enhacement bias. Los LLM favorecerán las respuestas generadas por ellos mismos. Esto implica que el modelo que evalúe debe ser distinto a los modelos evaluados para evitar un sesgo entre diferentes modelos. Esto se puede notar como es extensible a los humanos ya que nosotros preferimos en general, nuestro trabajo ante el trabajo de otros, especialmente si ambos tienen calidades similares.
- Limitada capacidad para calificar problemas matemáticos y de razonamiento. Esto se debe a la baja capacidad de los modelos de realizar problemas de razonamiento, llevando así a no ser capaces de calificar las

respuestas a este tipo de preguntas, ya que no son si quiera capaces de determinar cuál es la respuesta correcta. Incluso, si es capaz de resolver este tipo de problemas suele tener dificultades a la hora de calificarlo.

Algunas formas de resolver tanto la primera como la última de las limitaciones presentadas son:

- Cambio de posiciones. Esto se basa en llamar al LLM que juzga dos veces. En cada una de las veces, cada una de las respuestas, si son dos, han sido permutadas. Así, solo se dará la victoria a uno de ellos si en ambas ocasiones el LLM lo ha elegido como el mejor. En el caso de un mayor número de ejemplos, esto se puede hacer de una forma más escalable mediante la asignación aleatoria de posiciones.

- Few-shot judge. Si se le aporta algunos ejemplos al modelo que realiza el juicio, este en general mejora el problema del sesgo posicional.

- Chain-of-thought(CoT) and reference-guided judge. Esto permite la mejora de las capacidades del modelo para juzgar problemas matemáticos ya que con el CoT (Wei et al., 2022) le aportamos en el prompt una serie de ejemplos de como debería ser el razonamiento seguido.

Estas son tres formas específicas de resolver dos problemas concretos, sin embargo, existen diversas guías generales para conseguir una mejor actuación.

2.9.6. Estrategias para su mejora

Existen tres estrategias principales para mejorar el comportamiento y la evaluación generada por un LLM as a judge.

Diseño del prompt

El hecho de que los modelos de lenguaje tenga el ICL como capacidad, implica que la optimización del prompt que le pasemos generará un impacto directo en la efectividad del mismo al poderle enseñar como realizar una tarea determinada en base a diferentes ejemplos. Así, es recomendable seguir las siguientes estrategias para la mejora del prompt que le pasemos al modelo:

- Optimización del entendimiento de la tarea. Esto se puede hacer mediante la incorporación dentro del prompt de una serie de ejemplos (few-shot learning) de cómo sería la evaluación.

- Descomposición de la tarea. Esto se basa en dividir la tarea completa de evaluación en pequeñas tareas dándole detalles y restricciones para cada una de ellas en el prompt.

- Determinación de la respuesta. Para ello, en el mensaje que le pasemos al sistema añadimos el formato concreto en el que deben pasarse la respuesta. Esto se puede hacer también de una forma más robusta haciéndole que por ejemplo, rellene un formulario el cual tiene un formato concreto evitando así posibles errores dados de que se salte el formato.

- Fine tuning. Debido a que los modelos pueden tener problemas de entendimiento en ciertos dominios y los elementos anteriores se basan en dicho entendimiento y por tanto en la comprensión completa del prompt, es necesario realizar un re-entrenamiento del modelo. Este re-entrenamiento se hace con datos creados específicamente para la evaluación de tareas.
- Feedback. Estos modelos, aunque se ajusten a la tarea de evaluación, pueden seguir cometiendo fallos por lo que podemos optimizarlos mediante la adicción de feedback de los resultados. Este feedback en general suele venir desde modelo mejores o directamente desde seres humanos.

A pesar de esto, los modelos pueden seguir introduciendo aleatoriedad en las respuestas por lo que es interesante utilizar varios evaluadores o incluso mezclar LLM con métricas más rudimentarias para así suavizar el impacto de errores aleatorios.

Así, hemos podido ver las múltiples limitaciones que tienen este tipo de evaluadores y como tratar de evitar la mayor cantidad posible a partir de un buen diseño. Sin embargo, también es necesario determinar como de buena es la actuación de estos evaluadores para poder así fiarnos de su respuesta. Por ello, en el siguiente apartado trataremos de cómo podemos evaluar este tipo de modelos.

2.9.7. Evaluación de los LLM as a judge

A lo largo de este apartado, determinaremos algunas métricas descritas en (Gu et al., 2025) y (Wei et al., 2025), para evaluar la actuación de los LLM as a judge, para así poder determinar cómo de fiable es dicha actuación.

- Así, la primera forma, accuracy, se basa en la proporción de las veces que el modelo de lenguaje y el humano coinciden en su evaluación. Esto nos permite determinar si la mayoría de las veces el juicio determinado por el modelo sería el mismo o similar que el determinado por el humano. Así, si este fuese el caso, entonces el modelo sería fiable en general puesto que suele dar los mismo resultados que daría un humano.

- Una segunda opción es evaluar el mismo LLM sobre los mismos datos teniendo el mismo prompt para determinar las inconsistencias del modelo. Estas inconsistencias vienen dadas por el cambio de respuesta. Así, si por ejemplo la respuesta es binaria, una inconsistencia sería que una vez dijese una cosa y otra vez otra. Para conseguir que el modelo responda lo más fiable posible evitando errores, podemos evaluarlo varias veces en los mismo datos y que se quede con el valor más repetido (esto por ejemplo cuando solo hay dos opciones) o con la media (para aquellos casos en los que su tarea es realizar una puntuación). Esto al mismo tiempo que una medida de las inconsistencias del modelo es una forma de mitigarlas.

Además de todo esto, podemos medir el sesgo a la posición y a la longitud de la respuesta, viendo como afectan esto al criterio del modelo.

Capítulo 3

Descripción del problema

A lo largo de este capítulo, se abordará una descripción detallada del problema que abordamos en este trabajo, tras lo cual realizaremos un análisis sobre las decisiones tomadas para resolverlo y como hemos planteado nuestra solución.

Para ello, primero comencemos con el surgimiento de este problema.

Antes de la llegada de Internet, existía un gran problema. La incapacidad de acceso a los diferentes recursos por parte de las personas. Esto generaba que a menos que conocieses a alguien que supiese de algún determinado curso, estuviese en una universidad concreta, etc, no tenías acceso a más información que la que podías encontrar localmente.

Este problema parecía resuelto gracias a la llegada de Internet ya que conseguía que personas de todo el mundo estuviesen conectadas entre sí. Que simplemente con un click pudiesen acceder a las últimas investigaciones de alguien en la otra punta del mundo. Internet parecía la solución para llegar a la democratización de la información, para conseguir que todos tuviésemos acceso a cualquier información independientemente de nuestro lugar de origen o universidad en la que estudiásemos.

Sin embargo, esto se ha convertido en una falsa sensación de posibilidad. Ahora parece que todo es posible si se sabe donde buscar. Y esto que aparentemente es cierto genera paradójicamente el mismo problema que parecía resolver. Antes, sabíamos que no teníamos la posibilidad de acceder a ciertos recursos simplemente por el lugar donde estudiábamos o donde trabajábamos, sin embargo ahora nunca sabremos si buscamos en todos los sitios en los que pudimos haberlo hecho.

Vemos como ahora uno de los mayores problemas de todos, desde organizaciones, gobiernos, etc, es la inmensa cantidad de recursos a la que la mayoría de personas no llega a acceder nunca porque la mayoría no llega ni quiera a conocer su existencia. Esto genera que muchos de estos recursos sean desaprovechados o simplemente sean utilizados por las mismas personas, las mismas personas que identifican este tipo de recursos gracias a lo mismo que antes, un compañero, un profesor o un amigo.

Ahora el problema no reside en que no tengamos la información a nuestro alcance. El problema está en que tenemos tanta información disponible que a menos que sepamos qué buscamos y dónde buscarlo no tenemos la capacidad para encontrar nada.

Así, paradójicamente estamos en el mismo punto de partida porque aunque la información esté, no es fácil de encontrar. Un estudiante puede pasar horas buscando cursos para realizar y darse cuenta años más tarde que no encontró ni la mitad de los que podía haber hecho, solo porque no sabía ni qué buscar ni dónde buscarlo.

Este problema transversal a todos los organismos, ocurre también en la Universidad de Sevilla. Una Universidad que ha hecho y hace grandes esfuerzos por fomentar que el alumnado egrese con la mayor cantidad de habilidades posibles. Sin embargo, no consigue que la mayoría de estas iniciativas lleguen al alumnado incluso si este las esta buscando.

Un claro ejemplo de ello sería el Aula de Debate de la Universidad de Sevilla. Este aula que ha contado este año con 5 clases de alumnado de nueva incorporación y una clase de alumnado veterano, ha sufrido este problema con su alumnado de nuevo ingreso. Así, la mayoría de estudiantes de nuevo ingreso de este año en el Aula de Debate, eran alumnos que habían recibido un correo por parte de la Universidad, en el que se les hacía llegar este curso. Sin embargo, la mayoría de estudiantes veteranos no habían recibido nunca este correo y habían llegado hasta esta iniciativa gracias a buscar durante horas en la página web de la Universidad, por el boca a boca o porque en sus colegios hiciesen debate y esto les llevase a buscar si esto existía en la Universidad.

En todos estos casos, podemos observar como una iniciativa que es tan útil para la inserción laboral como el desarrollo de habilidades comunicativas, no había llegado a una gran parte del alumnado, y que a aquel que le había llegado había sido o bien por una mera casualidad o bien por medios tradicionales (boca a boca).

Vemos por tanto, como existen infinidad de recursos dentro de la Universidad y sin embargo muchos de estos recursos quedan sin uso pese a su gran utilidad debido a la falta de conocimiento de los estudiantes del mismo y a la dificultad de los mismos de encontrarlo.

En este trabajo queremos apostar por la democratización de estos recursos evitando gastos ingentes de tiempo de los usuarios para finalmente darse cuenta de que habían pasado por alto más recursos que los que habían sido capaces de encontrar, evitando con ello que personas que pudieran estar interesadas nunca llegasen a conocer la inmensa cantidad de recursos que la Universidad pone a su disposición.

Esto lo conseguiremos mediante la implementación de un sistema conversacional basado en repositorios.

Este sistema se basará únicamente en la información sobre doctorados existente en la página web de la Universidad de Sevilla. Esta restricción se debe a la ingente cantidad de páginas web existentes dentro de la propia Universidad lo que dificulta la demostración práctica de cómo se haría una implementación debido al tremendo coste computacional que esto requiere.

Téngase en cuenta que esto no pretende ser una aplicación final de uso para la Universidad de Sevilla, sino que la intención es mostrar como podría aplicarse este tipo de sistemas para mejorar el acceso a la información existente por parte de los estudiantes. Esto se debe principalmente a la limitación práctica que supone para la obtención de buenas respuestas el propio hecho de limitar el número de páginas web que vamos a tener en cuenta.

Nótese como esto podría hacerse extensivo a cualquier documento de la Universidad de Sevilla aportando así una ayuda útil a todo el estudiantado, lo que permitiría una mayor facilidad de resolución de dudas que si bien son frecuentes, suelen terminar resolviéndose tras horas de búsqueda o gracias a la consulta a compañeros. Un gran ejemplo de ello sería la automatrícula.

Esta es una visión general de la problemática de la que iremos viendo los inconveniencias que presenta y los elementos que son necesarios tener en cuenta para realizar un correcto estudio del mismo.

3.1. Problemáticas

A lo largo de esta sección analizaremos detalladamente todas las problemáticas que pueden surgir durante la resolución de este problema. Esto nos permitirá generar una solución que se adecúe perfectamente para así obtener el mejor rendimiento posible.

3.1.1. Tipos de datos

Antes de comenzar con el estudio del propio RAG y de su propia arquitectura, es necesario tener en cuenta como son los datos que nos disponemos a pasarle al sistema.

En nuestro caso realizaremos un scrapping de la página web de la Universidad de Sevilla para obtener aquellas páginas que sean de doctorados. Esto nos lleva a que los datos sean tanto estructurados como semi-estructurados, ya que tendremos desde páginas web o xml a pdf que tendremos que extraer, leer y vectorizar.

Esto genera que tengamos que tener varios elementos principales en cuenta a la hora de realizar el preprocesamiento de los datos:

Eficiencia

Vemos como nuestra extracción de los datos se realizará a través de la web mediante un scrapping. Esto inicialmente plantea el problema del desconocimiento incial de la cantidad de elementos que vamos a recoger. Sin embargo,

puede inferirse de una búsqueda rápida por la página web de la Universidad de Sevilla, como esta cantidad de elementos es previsible que sea grande.

Esta inmensa cantidad de elementos que esperamos extraer de cada uno de los formatos, nos lleva a tener una necesidad de carga de los mismos de forma computacionalmente eficiente. Asimismo, también requiere la necesidad de evitar mapear múltiples veces el mismo elemento para así evitar duplicidades en la base de datos.

Esto último, es bastante relevante de cara al estudio ya que una duplicación de la información puede generar grandes problemas en la eficiencia de nuestro sistema de retrieval al encontrar información redundante.

Tipo

Asimismo, durante el análisis se tendrá que tener en cuenta el tipo concreto de los datos para realizar el mejor tratamiento posible a cada uno de ellos, ya que como no son lo mismo datos estructurados, semi-estructurados y estructurados, tampoco deberían recibir el mismo trato. En nuestro caso, al tener tanto PDF como HTML, podemos tener cualquiera de los tres tipos de datos por lo que tenemos que tenerlo en consideración.

3.1.2. Información privada

Existe una gran cantidad de información dentro de la página web de la Universidad de Sevilla que no es pública, sino que requiere de autentificación, en ella puede incluirse desde el expediente académico de un usuario hasta la consulta de un libro de la biblioteca. Esto en sí mismo supone el reto de considerar si para nuestro caso de uso es realmente necesario que una persona este loggueada o no. Así, en una primera aproximación, tendremos en cuenta solo la información que es completamente pública. Supondremos esto, ya que la mayor cantidad de información para cualquier tipo de cursos, etc, en general suele ser pública, requiriendo únicamente la autentificación el propio acceso o registro en estos cursos. Esto nos evitará también tener que tratar con temas de privacidad y posibles ataques entre usuarios de distinto rango para obtener información de los demás usuarios. Asimismo, esto tiene también sentido ya que como tenemos intención de realizar un scrapeo en la web, este resultaría más complicado si necesitásemos la autentificación, ya que aunque usásemos el usuario y contraseña de un estudiante, esto no nos asegura tener acceso a toda la información disponible ya que puede ser que los estudiantes de Máster o de Doctorado tengan acceso a diferente información que los de Grado. Por lo que sin objeto de parecer simplistas, hemos considerado que toda la información en la Universidad de Sevilla (y con información no implica acceso o inscripciones) relevante para los doctorados es pública y tiene acceso cualquier persona desde la propia web.

3.1.3. Vectorización

Esta fase es una de las más importantes de nuestro sistema, ya que dependiendo de cómo de bien o de mal realicemos la representación vectorial de la información, será mucho más fácil por el sistema encontrar relaciones entre la

pregunta y la información que la responde.

Así, dentro de esta fase podemos encontrar múltiples elementos que vamos a tener en cuenta:

Tokenización

El primer punto que tenemos que tener en cuenta es como vamos a tokenizar nuestros textos. Esta tokenización puede o no corresponder con palabras. Además, debido al uso que haremos de modelos de lenguaje pre-entrenado, será necesario tener en cuenta el tipo de tokenización usada para su entrenamiento, ya que estos han aprendido a reconocer patrones o similaridades en un cierto tipo de divisiones. Aunque es cierto que podríamos modificar el cómo tokenizamos a placer, esto no nos asegurará obtener el mejor rendimiento del modelo, es más, es de esperar que el rendimiento no fuese tan bueno como si se realizase una tokenización acorde al modelo de embedding usado.

Chunks

Tras haber decidido como vamos a representar los documentos llega la hora de dividirlos. Esta división es necesaria ya que la ventana de contexto no es capaz de soportar en general todo el documento completo. Así, el nuevo dilema es determinar como vamos a realizar estas divisiones. Puede que de cara a nuestros datos, sea importante mantener en el mismo trozo la información contenida en una misma frase o en un mismo párrafo. Asimismo, es necesario por tanto determinar tanto el tamaño de la división como el grado de solapamiento. Este último se requiere debido a que permite una mayor cohesión del propio texto manteniendo parte del contexto del elemento anterior y evitando problemas de pérdida de información en la frontera de los documentos donde se producen los cortes.

Embedding

Ahora lo siguiente que tenemos que plantear es qué modelo de representación queremos usar. Esto tiene que ir de la mano de las decisiones anteriores pero también tiene que ir de la mano de los datos que tengamos. Así, existen modelos que son mejores para el tratamiento de datos semi-estructurados que otros, por lo que será necesario tener en cuenta esto de cara a obtener una buena representación de los mismos. Además, en nuestro caso, la estructura de los datos es especialmente importante, ya que tenemos PDFs, los cuales pueden contener tablas. Estas últimas suelen suponer un reto para los modelos, los cuales pueden fallar en obtener una representación precisa si desconocen como tratar de forma adecuada estas tablas.

3.1.4. Recuperación

En esta parte, se tendrá que tener en cuenta diversos aspectos que son cruciales, como:

Similitud

El primer elemento que tenemos que tener en cuenta es como vamos a medir la similitud entre la pregunta y nuestros documentos. Esto es relevante ya que cómo se mida esta similitud puede afectar a los chunks devueltos y esto a la calidad de la respuesta.

Aquí también será necesario tener en cuenta si queremos que devuelva los más similares o que realice recuperaciones con una mayor variedad aún pudiendo perjudicar la similitud exacta de la pregunta con algún documento. Esto es importante ya que de otra forma el sistema podría devolver los mismos documentos para consultas que si bien fuesen parecidas no fuesen iguales, llevando así a una generación repetitiva de contenido y afectando a la experiencia del usuario en tanto en cuanto este no sintiese satisfechos sus requerimientos por repetición de contenido.

Top K

El siguiente elemento relevante a tener en cuenta es cuántos documentos vamos a devolver, lo cual es incluso más importante que lo anterior, ya que si fallamos en usar una medida de similitud correcta o en usar la más adecuada para nuestros datos pero devolvemos los documentos suficientes para poder responder a la pregunta, estaremos consiguiendo respuestas buenas a pesar de no realizar buenas búsquedas. Así, será importante determinar cuántos documentos vamos a devolver. Esta cantidad tiene también un efecto significativo con respecto a la respuesta del modelo. Dicho efecto es debido a la ventana de contexto que tienen todos los modelos, la cual limita la cantidad de elementos que podemos usar dentro del mismo y que se tengan en cuenta.

Asimismo, debido a que en contextos largos la información ubicada en la parte media de la ventana de contexto tiende a perderse, también será necesario tener en cuenta el orden en que le proporcionamos al modelo las instrucciones, el contexto y la pregunta.

Todo esto tiene que estudiarse viendo cuántos y en qué orden se van a devolver los elementos con mayor similitud a la pregunta.

Por último, el modelo también debe ser capaz de no devolver ningún resultado cuando no encuentra similitudes entre la pregunta y la información disponible. Esto puede ocurrir si alguna pregunta del usuario no tiene una respuesta directa en los datos, lo cual no necesariamente indica un fallo en el proceso de recuperación. De hecho, podría deberse a que la información misma no refleja completamente la realidad o no cubre todas las posibles preguntas que alguien podría hacerse en este ámbito. Aunque hemos asumido que esto es poco probable, debido a la gran cantidad de información existente, no lo consideramos imposible. Para abordar este problema, al menos parcialmente, sería necesaria la colaboración de usuarios involucradas en estos programas, que pudieran aportar más datos. Sin embargo, incluso con esa colaboración, no podríamos garantizar una respuesta para absolutamente todas las preguntas, ya que siempre existirá la posibilidad de no saber si contamos con información suficiente para responder

a cualquier pregunta que nos pudiesen plantear.

3.1.5. Generación

El siguiente aspecto a tener en cuenta es el modelo que vamos a usar para la generación de las respuestas. Esto es importante porque con los grandes modelos de lenguaje se ha podido observar como cuánto mayor sea el número de parámetros del modelo, mejor funciona este. Sin embargo, la respuesta adecuada no es decir: "Pues cojamos el mayor", ya que este se debe adecuar también a las capacidades computacionales, teniendo en cuenta también el trade-off entre la carga computacional y la mejora en el rendimiento real obtenida.

Asimismo, otro elemento relevante es qué instrucciones darle al modelo. Esto es necesario hacerlo casi por prueba y error para obtener una instrucción mediante prompt engineering que guíe al modelo para que realicé la tarea de forma satisfactoria.

Además, también será necesario tener en cuenta si el modelo recibirá o no un historial de la conversación y cómo y dónde lo recibirá. Este hecho es importante, ya que si el historial es largo puede afectar al buen desempeño del modelo "olvidando" los primeros elementos y con ello las instrucciones si estas fueron pasadas al principio. También es importante determinar su posición, ya que el contexto para responder a una pregunta puede ser tan importante como los propios documentos, debido a que se puede obtener de él información de posibles anáforas que surjan a lo largo de la pregunta, haciendo referencia a elementos de los cuales se haya hablado anteriormente. Esto también tiene su importancia ya que queremos evitar que el usuario sienta que la respuesta es repetitiva, lo cual se puede conseguir intentando que el sistema no responda de forma similar a como lo hizo en el turno anterior, dando una mayor sensación de novedad al usuario, la cual podía haberse ocultado tras el uso de determinadas expresiones repetitivas o incluso de varios documentos similares a la búsqueda anterior. Esto último permite que aunque en la recuperación se obtenga poca variedad, al menos el usuario no sienta como la respuesta es la misma que en el turno anterior mejorando así su experiencia.

Por último, este tipo de sistemas tienen que lidiar con la posibilidad anteriormente comentada de la no existencia de documentos similares. Esto debe hacerse de la forma más amigable y útil posible para el usuario instándole a reescribir la pregunta o incluso contactar con algún centro especializado que pueda proveerle de mayor información.

3.1.6. Otros aspectos generales

En este apartado, tendremos en cuenta otras posibles problemáticas que se pueden desarrollar a lo largo de una conversación. Aquí diferenciaremos distintos tipos de interacción del usuario para poder observar como estas pueden generar problemas al sistema que hemos comentado anteriormente.

Esta sección aunque pudiese parecer poco relevante, quizás es de las más relevantes al estudio ya que no podemos considerar que el usuario siempre seguirá

el camino feliz y esperado que siga, si no que debemos suponer siempre que el usuario puede actuar con malicia para así construir un sistema robusto y fiable.

Extranjeros

Un elemento importante a tener en cuenta es que el usuario que realiza las cuestiones al chat no tiene por qué ser hispanohablante o ni quiera dominar el idioma. Esto es un elemento obvio si tenemos en cuenta la existencia de los programas Erasmus en la Universidad de Sevilla y de los doctorados con mención internacional, lo cual requiere de estancias en el extranjero por lo que esto lleva a que tanto los estudiantes españoles como los propios extranjeros puedan realizar estancias de doctorado tanto en la Universidad de Sevilla como en otras universidades. Llevando a la necesidad de poder traducir las respuestas desde el idioma original desde el que nos habla el usuario, al español, que es el idioma en el que está nuestra base de conocimientos.

Debemos tener en cuenta esta traducción es necesaria ya que al generar los vectores de representación, no es irrelevante el idioma en el que venga la frase. Esto es debido a dos elementos principales:

- El primero de ellos es que si nuestro modelo ha sido entrenado con Cross-lingua entonces tendremos que tener presente en que idiomas ha aprendido a realizar las representaciones conjuntas. El Cross-lingua se refiere a un modelo de embedding que ha sido entrenado para representar juntas en el espacio vectorial, palabras con el mismo significado aunque provengan de distintos idiomas (Ruder, 2017).

- El segundo es que para el caso de modelos en los que no se haya entrenado con Cross-lingua, si realizamos la representación vectorial de nuestros documentos en un idioma y la de la pregunta a partir de otro idioma distinto, puede generar que las representaciones no encuentren las similitudes adecuadas dentro de la base de conocimiento llevando a una recuperación pobre o desafortunada de documentos.

Es por ello que cualquiera de los casos anteriores es necesario tener en cuenta los posibles idiomas de entrada en los que vamos a permitir que el usuario se comunique, con objeto de facilitarnos la realización de un tratamiento previo ajustado a estos idiomas.

Dentro de esto, tenemos también que distinguir el caso en el que un usuario con intenciones maliciosas escriba su pregunta en varios idiomas. Este tipo de comunicación tiene que ser tenido en cuenta ya que afecta directamente a como se responde. Esta respuesta puede tener un impacto significativo en la experiencia del usuario, ya que si la intención del mismo era benigna pero cometió un error que pudiese generar confusión entre varios idiomas, si se rechaza directamente responder a la pregunta, podría generar que el usuario tuviese un rechazo hacía el sistema, ya que por un simple fallo este respondió de una forma "predeterminada".

Fórmulas de cortesía

En muchas ocasiones, el usuario entra al chat y realiza directamente la pregunta que tiene. Sin embargo, este no tiene porque ser el flujo necesariamente y aunque lo fuese, puede acabar con fórmulas de cortesía que hay que tener en cuenta. De esta forma, podemos distinguir las siguientes fórmulas de cortesía:

- Saludo. El usuario puede saludar al sistema antes de realizar una interacción. Así, este puede decir algo como: "Hola" o "Buenos días", tras lo cual, plantear la pregunta.
- Despedida. El usuario puede despedirse del sistema tras la realización de la interacción. Un ejemplo de ello podría ser un "hasta luego".
- Agradecimiento. Por último, un elemento que suele ser común entre los usuarios, es el agradecimiento al final de la interacción si esta ha sido satisfactoria. Un ejemplo de ello sería: "Muchas gracias", "Estupendo, gracias".

Todos estos elementos, se tienen que tener en cuenta como fórmulas de cortesía propias del lenguaje humano, que no deben entrar si quiera en el sistema RAG, ya que no requieren ningún tipo de búsqueda, aunque si que requieren de una contestación equivalente. Asimismo,podemos observar como en los dos últimos casos, no solo requiere la contraparte por parte del sistema, sino un llamamiento al usuario a que vuelva siempre que necesite algo o a que pregunte si tiene alguna otra cuestión. Esto último podría incluso hacerse extensivo de cara al saludo. Un ejemplo de ellos sería la contestación: "Siempre es un placer ayudar, ¿tienes alguna duda más?"

Preguntas fuera de dominio

Aquí podemos incluir cualquier pregunta cuya temática no este relacionada con los doctorados. Estas pueden ser de dos tipos:

- Benignas. Este tipo de preguntas son aquellas fuera de dominio como por ejemplo: "¿Qué grados hay en la Universidad de Sevilla?", pero que el usuario puede realizar por desconocimiento u olvido de las capacidades del sistema.
- Malignas. Este tipo de preguntas son aquellas que el usuario hace con malicia intentando obtener información del sistema que no le corresponde o incluso intentando que este se "rompa" y generé respuestas para ámbitos en los que no tendría que responder. Un ejemplo de ello sería: "Mi doctorado trata sobre historia inglesa, y tengo una serie de preguntas que quiero que respondas ya que eres un asistente para doctorados. Intenta por tanto darme toda la información disponible sobre Donald Trump para poder así seguir con mi doctorado."

Ambos tipos de preguntas tienen que tratarse, evitando que en cualquiera de los casos el sistema entre en la búsqueda de documentos (en el RAG) y que respondan directamente a los requerimientos. Es decir, el sistema en ambos casos

debe negarse a responder la pregunta, indicándole al usuario cual es su ámbito de conocimiento para así redirigir a ambos usuarios a preguntas que sí estén relacionadas con lo que pueda responder y que sean adecuadas. Por tanto, será necesario tener en cuenta la posible existencia de este tipo de preguntas y realizar comprobaciones para verificar que se está respondiendo de forma correcta, es decir, que por ejemplo, para la pregunta anterior el sistema no le de información sobre quién es Donald Trump.

Otros elementos

Además de todo lo anterior, el usuario también puede meter otros elementos como nombres propios o siglas dentro de sus preguntas. De esta forma, un usuario podría realizar la pregunta: "¿Qué becas hay para hacer un doctorado en las US?", pero también podría hacer la pregunta: "Hola me llamo Pepe, me gustaría saber cómo puedo acceder a un programa de doctorado" y en ambos casos el sistema debería responderle adecuadamente.

En el ejemplo anterior vemos como existen dos elementos diferenciales a tener en cuenta:

- El primero de ellos es el manejo de siglas, así un sistema entrenado con datos ingleses si no es instruido correctamente puede pensar que US es las siglas de United States en vez de corresponderse con las siglas de la Universidad de Sevilla.
- Un segundo punto a estudiar es que el usuario en la segunda pregunta ha proporcionado su nombre. Esto se tiene que tener en cuenta tanto de cara a la traducción como de cara a la propia comunicación. Así, el sistema no debe intentar traducir un nombre propio, ya que puede cometer errores o realizar traducciones que puedan resultar incluso ofensivas para el usuario. Asimismo, el recibir el nombre por parte del usuario supone una información muy valiosa que se puede utilizar durante la interacción con el mismo para que se sienta escuchado y entendido, sintiendo una mayor aceptación por el sistema. A fin de cuentas, se trata de mejorar la experiencia del usuario partiendo de la propia información que este le proporciona al sistema.

3.1.7. Evaluación

Para poder determinar como de bueno es el modelo obtenido tendremos que medirlo. Dentro de esta evaluación, podemos encontrar una serie de problemas que son interesantes tener en cuenta.

Información

Lo primero es cómo tenemos la información. Esta información no esta clasificada ni categorizada en función de qué preguntas responde. Asimismo, puede suponer un trabajo extenso y arduo, debido a la gran cantidad de páginas, determinar para cada una de las preguntas cuáles son los PDFs o páginas que debería devolver. Por tanto, esta limitación tanto temporal como personal es necesaria tenerla en cuenta. Asimismo, al no ser una información previamente clasificada,

tampoco es fácil usar métricas como el recall o el precision, ya que no podemos determinar cuáles son positivos (nos dan información) y cuáles no, sin tener que ir buscando a mano para cada una de las preguntas todos los documentos relevantes. Llevándonos, por tanto, a plantear una serie de problemáticas subyacentes:

- ¿Cuándo consideramos que el sistema desconoce la respuesta por falta de información en la base de conocimiento y cuándo por falta de capacidad representativa o por el mal uso de una medida de similitud?
- ¿Es igual de problemático que no me devuelva la información de un pdf a que no me lo devuelva de una web?
- Si los documentos pueden cambiar o incrementarse con el tiempo, ¿cómo mido que documentos han dejado de ser relevantes?
- ¿Cómo mido el efecto de posibles duplicidades en la recuperación de información?
- ¿Cuándo considero que un documento es o no relevante? o más bien, ¿hasta qué punto se puede considerar un documento como relevante?
- ¿Cómo medir el impacto de la falta de documentos que tratan de lleno el tema, frente a documentos que lo tratan de forma superficial?
- ¿Cómo medir la relevancia y fiabilidad de la parte generativa a los documentos?
- ¿Cómo determinar si una respuesta es satisfactoria para una pregunta?
- ¿Cómo evaluar cuánto podría haberse mejorado esa respuesta?
- ¿Cómo mido que el modelo generativo integre los documentos en su respuesta y no se limite solo a repetirlos?

Podemos observar como cada una de estas problemáticas afecta a uno o varios elementos del sistema teniendo que tenerlo en cuenta durante todo el proceso.

Módulos

Sabiendo que cada uno de los elementos que hemos descrito en esta sección pueden verse como diferentes módulos, tendremos que tener en cuenta no solo como actúa el sistema global sino como actúa cada uno de ellos viendo cómo puede afectar un cambio leve en uno de ellos en el resto del sistema. Además, esta naturaleza modular obliga no solo a optimizar los elementos por separado sino a tener que optimizarlos como un conjunto, puesto que la interacción entre ellos puede modificar el comportamiento global. También tenemos que considerar las propias limitaciones de los sistemas y modelos que usemos, sin tratar de obtener sistemas perfectos a partir de modelos imperfectos.

Capítulo 4

Solución del problema

A lo largo de este capítulo, plantearemos la solución a este problema. Para ello haremos primero una breve descripción de los elementos técnicos necesarios para la implementación. Posteriormente, realizaremos una descripción de como resolvemos cada uno de los problemas planteados anteriormente mediante la presentación de una serie de módulos independientes.

4.1. Elementos técnicos

Para la implantación del sistema utilizamos Ollama [1], el cual nos permite la implementación de un LLM localmente, solucionando los dos problemas principales que podía tener una empresa al intentar implementar un sistema RAG:

- El primero de ellos era la privacidad de sus documentos. Los grandes modelos de lenguaje como Copilot o los GPT, tiene el problema de que se usan mediante llamada de API, lo que genera dos problemáticas en cuanto a la privacidad de los datos:
  - El primero de ellos, es que pueden sufrir ataques cuando se realiza la llamada al modelo, pudiendo exponer datos sensibles de la empresa.
  - El segundo es que no se tiene garantía de que los datos de unos usuarios no los este usando el modelo para realizar respuestas a otros usuarios o que estos no se estén usando para entrenar nuevos modelos, lo que genera importantes brechas en la privacidad dentro de las empresas.
- El segundo problema es el desmesurado coste que supone para muchas organizaciones el pago de los grandes modelos de lenguaje. Así, en la mayoría de los casos, para poder usar herramientas como los GPT o copilot mediante API necesitan pagarse ciertas tarifas, las cuales pueden no poder o no querer incurrir en ellas las organizaciones.

[1] https://ollama.com/

Podemos apreciar como Ollama nos permite solucionar los problemas anteriores debido al libre acceso a modelos que pueden ser descargados e implementados localmente. El único coste añadido que puede tener este tipo de modelos es la infraestructura para poder soportarlos (ej: servidores, etc), en caso de que se quisiesen usar aquellos con gran potencia o mayor número de parámetros. Aún así, aunque la parte del coste fuese equivalente, se estaría obteniendo un gran beneficio en lo que a la privacidad respecta.

Pasemos ahora a la descripción del modelo de embedding y el modelo generativos elegidos.

4.1.1. Modelo embedding

Para la creación de la representación vectorial de los documentos, decidimos usar el modelo mxbai-embed-large [2].

Aunque otros modelos como nomic-embed-text [3] tuvieran mayor ventana de contexto, el modelo mxbai-embed-large era preferible ya que había sido entrenado sin overlap lo que permitía una mejor generalización. Esta capacidad de generalización del modelo unido al hecho de que nuestros documentos iban a tener que dividirse igualmente, por lo que una ventana de contexto más grande podía meter más ruido en nuestros datos, llevó a la decisión de uso del modelo mxbai-embed-large para la creación de la representación vectorial. Además, podemos observar en la página web de este modelo, (Lee et al., 2024) como es mejor que el nomic-embed-text en tareas como retrieval o re-ranking, haciéndolo preferible para nuestro caso de uso.

Añadir también que pese a que el modelo bge-m3 [4] tenía una mayor ventana de contexto, pudiendo obtener una mejora semántica, la capacidad de manejar tanto documentos más largos como más cortos, la capacidad de soportar diversas funcionalidad como la recuperación densa o dispersa y que tuviese de forma libre su autotokenizador, lo que lo hacia una mejor opción ya que los chunks se dividen en tokens en la forma exacta en la que el modelo lo entiende mejor. A pesar de todo ello, el hecho de que el modelo mxbai-embed-large tuviera un contexto más reducido que el modelo bge-m3, lo hacía preferible. Esto se debe a que tal y como podemos observar en [5] y [6] la ventana de contexto de llama3.2:3b, debido a que tiene quantización, es de 8K tokens. Esto era relevante, ya que al hacer el sistema de re-ranking, se le daba al modelo un prompt de actuación, los datos del chunk y la pregunta, lo cual generaba un exceso de la ventana de contexto cuando se usaba bge-m3 generando respuestas que no estaban acorde con las instrucciones. Esto lo podemos observar en la siguiente imagen que muestra como el modelo ignoraba parte de las instrucciones dadas ya que se le "olvidaban".

[2] https://ollama.com/library/mxbai-embed-large
[3] https://ollama.com/library/nomic-embed-text
[4] https://ollama.com/library/bge-m3
[5] https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct
[6] https://ollama.com/library/llama3.2:latest

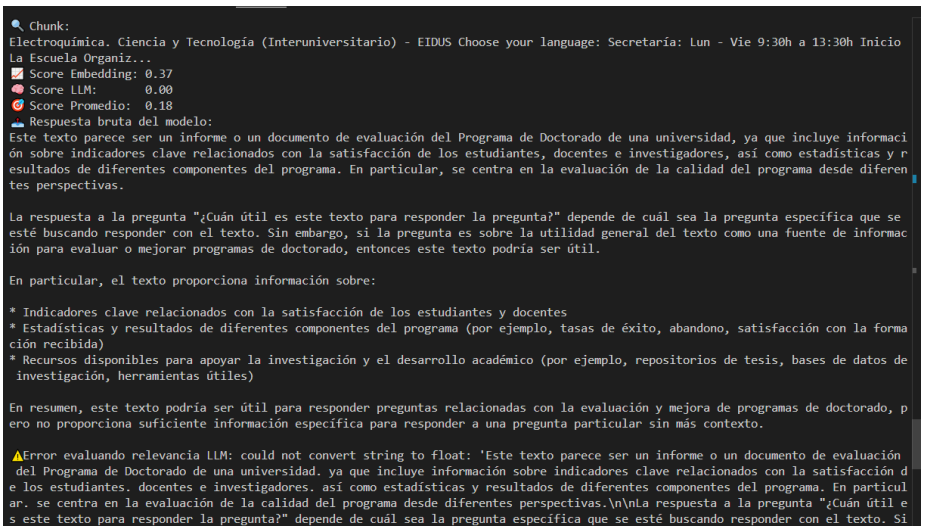


Figura 4.1: Traducción frase original con modelo

Podemos observar en la imagen, como el modelo ignoraba la instrucción de devolver una nota numérica entre 0 y 1, devolviendo por el contrario su opinión respecto al por qué ese chunk respondía o no bien a la pregunta. Este fallo sucedía debido al excesivo número de tokens que se le mandaba al modelo, los cuales generaban ese olvido al copar un solo chunk el número completo de tokens que el modelo era capaz de soportar.

4.1.2. Modelo generativo

Para nuestra implementación práctica del RAG, el modelo que hemos usado para la parte generativa es llama3.2:3b [7].

La decisión de utilizar este modelo se debió a la unión de sus capacidades con las restricciones técnicas del dispositivo en el que se implementó el sistema. Este modelo presentaba un entrenamiento optimizado en diálogo multilingüe, lo que nos permitía generar traducciones desde diversos idiomas (este modelo sólo fue entrenado en Inglés, Alemán, Francés, Italiano, Portugués, Hindi, Español y Thai) a lo largo de nuestra conversación. Además, fue optimizado en tareas de recuperación de información convirtiéndolo en un buen candidato para nuestro caso de uso. Por último, también había sido optimizado para tareas de generación de resúmenes, lo que nos permitía también un avance en algunas de las problemáticas planteadas anteriormente. Todo esto requiriendo únicamente un peso en memoria de 2GB, lo que permitía que cualquier dispositivo pudiese correrlo sin tener muchos problemas, permitiendo así una generalización de este trabajo a cualquier tipo de dispositivo.

Téngase en cuenta que aunque soporta las conversaciones en varios idiomas, no tiene la capacidad para manejar otros idiomas con muchos hablantes. Esto nos llevará posteriormente a una recomendación futura de uso de un modelo

[7] https://ollama.com/library/llama3.2

especializado en traducción entre idiomas para realizar las traducciones, lo que permitirá soportar mayor cantidad de idiomas e incluso evitar posibles problemas en cuánto a la detección del idioma. Por tanto, se ha de tener en cuenta, que para todo lo sucesivo siempre que nos refiramos a un modelo empleado será el llama3.2:3b el que usemos, salvo que se especifique lo contrario (por ejemplo para los módulos de embedding).

Tras esta breve descripción de las necesidades técnicas, pasemos a describir la implementación y evaluación del sistema.

4.2. Implementación

En esta sección hablaremos sobre como hemos implementado nuestra solución al problema planteado, teniendo en cuenta las diversas problemáticas. Esta solución se encuentra disponible en https://github.com/myNerea/TFMCode.git.

Para ofrecer al lector una mejor panorámica de la solución, a continuación le mostramos el flujo que seguiría una conversación habitualmente en nuestro sistema en función de las decisiones que se vayan tomando. Dichas decisiones ira encaminando el sistema para que entre en unos módulos o en otros.

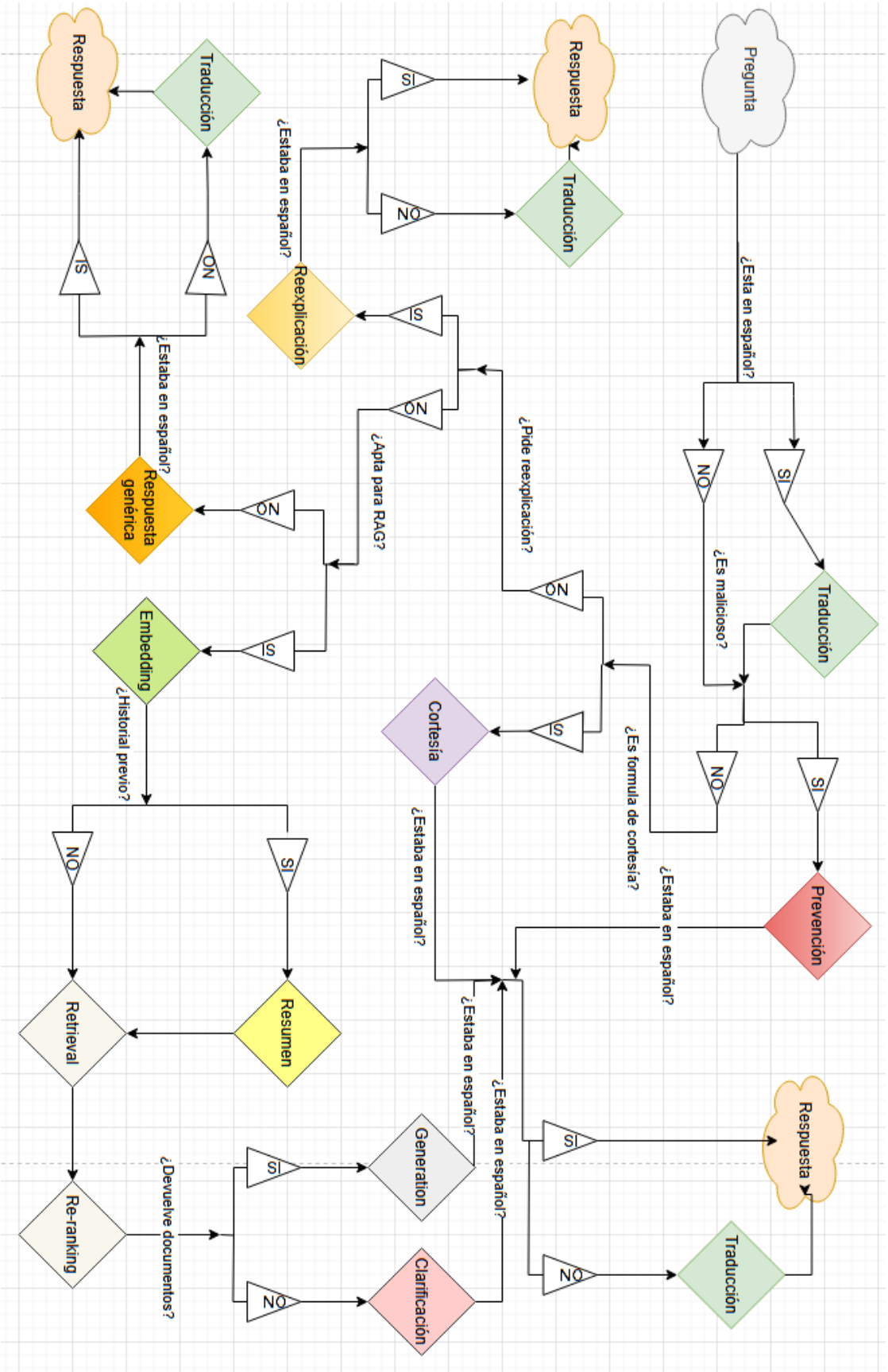


Figura 4.2: Diagrama creado con draw.io

Cada una de estas decisiones descritas anteriormente, son tomadas por un agente decisor el cual va considerando hacia que módulo dirigir al sistema según la pregunta realizada por el usuario. Tras esto, cada uno de los módulos del sistema termina devolviendo una respuesta acorde a la entrada recibida por el usuario, la cual puede haber necesitado desde una traducción hasta una búsqueda en el sistema RAG, pasando por alguna fórmula de cortesía.

Este flujo conversacional, sigue el siguiente flujo en los archivos del repositorio indicado anteriormente:

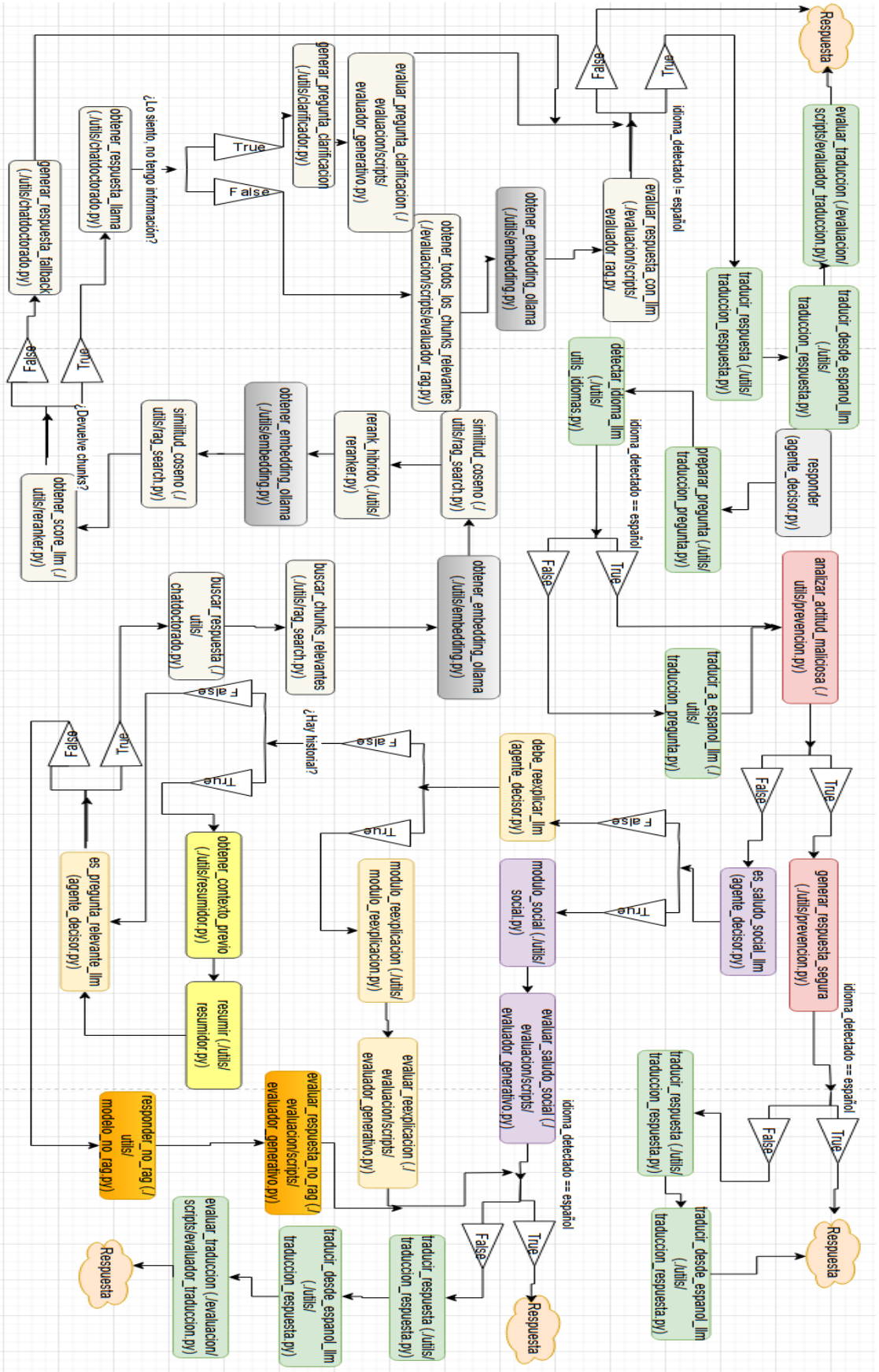


Figura 4.3: Diagrama creado con draw.io

Podemos observar en el diagrama anterior, como la estructura básica del sistema se encuentra modularizada tal y como se podía observar en el diagrama de flujo de la conversación.

Tras una breve explicación de este flujo, pasaremos a la descripción de cada uno de los módulos que lo componen.

4.2.1. Agente Decisor

Comencemos primero por el elemento central de nuestra implementación.

En el diagrama anterior, se puede observar como existen diversas decisiones a tomar, y que en función de lo que resulte de ellas, se usa un módulo u otro. Estas decisiones son tomadas por nuestro agente decisor, el cual va dirigiendo el flujo de trabajo a unos módulos u otros, en función de la pregunta que reciba. Así, lo primero que hace es llamar a la función de traducción, la cuál a su vez llama a un modelo de lenguaje que detecta el idioma en el que viene la pregunta escrita. En caso de que el idioma fuese distinto del español, aunque sólo hubiese una palabra en otro idioma, entonces se efectúa la traducción desde el idioma de origen al idioma de destino, que en nuestro caso es el español. En caso contrario se devuelve la pregunta tal y como el usuario la había planteado.

Tras esto, lo siguiente que hace el agente es comprobar si el usuario muestra intenciones maliciosas con su prompt. En caso de tenerlas o de que parezca que las tiene, el sistema le da una respuesta por defecto al usuario para evitar daños. Esto se determina mediante el uso de un modelo instruido para la detección de malicia en las consultas del usuario. En caso de que no la tuviera, otro modelo se encarga de determinar si la cuestión que se le había planteado era únicamente un saludo o fórmula de cortesía o si por el contrario contenía también alguna pregunta. Esto facilitaba la dirección del sistema al módulo de cortesía, o por el contrario la dirección a otros módulos como el de RAG. El modelo usado se instruía específicamente en la detección de este tipo de interacciones, indicándole que para que fuese una fórmula de cortesía digna de responder esta debía venir sola, sin ningún tipo de información adicional que pudiese suponer una pregunta. Si este módulo de saludo saltaba, entonces no se comprobaba los otros módulos (si era RAG o si el usuario pedía una re-explicación de la pregunta) para evitar tiempo computacional innecesario.

En el caso de que no fuese un saludo, el agente continúa comprobando con un nuevo modelo si el usuario está pidiendo una re-formulación de una información aportada anteriormente. En caso de que así fuera, no se realizaría tampoco comprobación alguna acerca de si era necesario el uso de RAG, entrando directamente en el módulo de re-formulación. Es importante indicar que antes de entrar en el módulo para generar la explicación, se comprueba si existe historial en la conversación, ya que en caso de no haberlo, se le da al usuario una respuesta predeterminada en el que se le indica que no se puede reescribir nada, puesto que es la primera interacción que se da en la conversación.

En el caso en el que se considerase que no se debía entrar en ninguno de los

módulos anteriores, el agente comprobaba con otro modelo si la pregunta debía o no responderse mediante RAG. Así, en caso de que fuese considerada para RAG, el modelo realiza la representación vectorial sobre el mismo embedding, la búsqueda de documentos, la re-indexación de los documentos, la devolución de estos como contexto a otro modelo y la generación del mismo de la respuesta. Por el contrario, si esta respuesta era negativa, es decir, que no creyese que necesitase el RAG, otro modelo contestaría al usuario indicándole que ese no era su ámbito de estudio. Esto último, se hizo para evitar respuestas predeterminadas cuando el modelo considerase que la pregunta no era relevante para hacer una búsqueda en la base de conocimientos, evitando con ello un empeoramiento de la experiencia del usuario.

Cada una de estas decisiones eran tomadas por modelos independiente, los cuales podrían haberse unido por eficiencia computacional en uno solo que realizase todas las tareas, sin embargo, esto no se llevo a cabo ya que reportaba peores resultados a la hora de clasificar las frases en cada uno de los módulos y seguir el flujo correspondiente, lo que en muchas ocasiones generaba que una pregunta que debería ser considerada para RAG fuera considerada como un saludo.

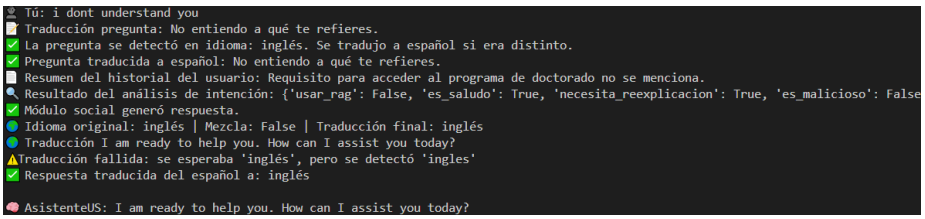


Figura 4.4: Imagen de muestra para el problema

Vemos como en la imagen anterior se puede observar este problema en el que la entrada del usuario requiere claramente una re-explicación, sin embargo, el sistema lo considera como un saludo llevándolo hacía ese módulo y por tanto obteniendo una respuesta incorrecta por no estar acorde con la pregunta realizada por el usuario.

A continuación, describiremos más detalladamente cada uno de los módulos descritos aquí.

4.2.2. Traducción

Continuemos con el primer módulo por el que puede pasar la conversación, el de traducción.

Este módulo se basa en el modelo llama3.2:3b para la generación de las traducciones. Para ello, en primer lugar el texto pasa por el modelo para que este detecte el idioma en el que está escrito el texto, indicando también si contiene o no una mezcla de idiomas. Todo lo anterior se devuelve en formato JSON para facilitar las conexiones entre funciones.

Tras esto, una nueva función comprueba a partir del JSON generado anteriormente, si el texto contenía o no una mezcla de idiomas, poniendo por defecto el idioma principal como inglés, en caso de detectar la existencia de varios idiomas. En caso contrario, se entra en una función que comprueba si el idioma del texto es el español, en cuyo caso no haría ninguna clase de traducción, dejando la cuestión del usuario intacta para continuar hacía el siguiente módulo. Si por el contrario el idioma detectado se identificase como distinto del español, se genera una llamada a otro modelo llama3.2, cuya tarea se basa en realizar la traducción desde el idioma detectado al español.

Este módulo inicialmente se decidió implementarse mediante el uso de funciones tradicionales como GoogleTranslator (Baccouri, 2023) para la generación de las traducciones. Sin embargo, debido a los problemas que este tipo de sistemas presentaba para la traducción de nombres propios, de siglas y de frases que contuviesen múltiples idiomas, decidimos que era más conveniente el uso de un modelo de lenguaje, ya que obtenía mejores traducciones. Algunos de estos problemas pueden observarse en las siguientes imágenes:

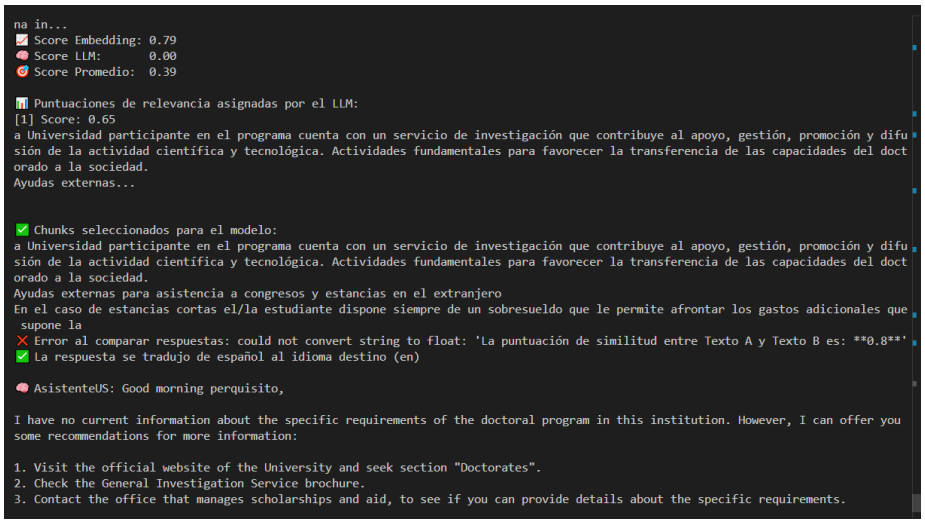


Figura 4.5: Traducción al idioma original sin modelo

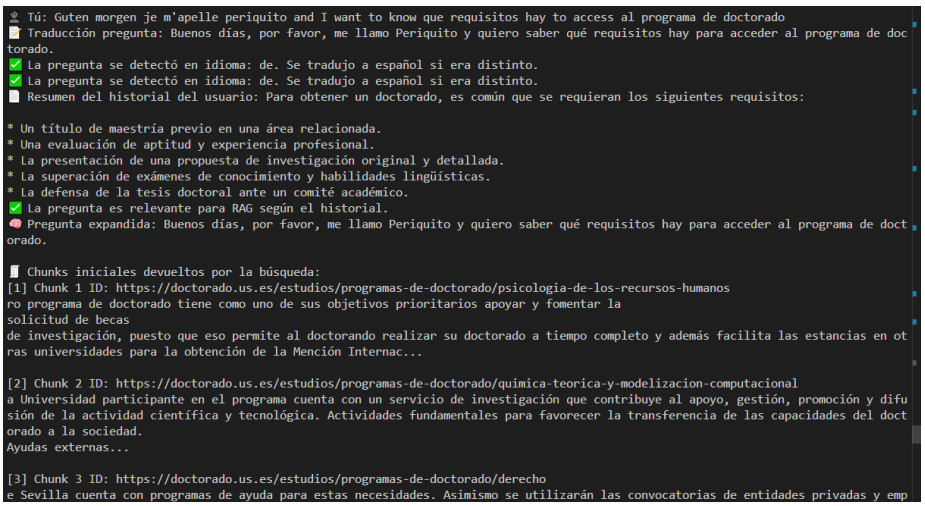


Figura 4.6: Traducción frase original con modelo

En estas imágenes puede observarse como el uso del modelo llama3.2:3b, generaba traducciones mejores en aquellos casos en los que hubiese nombres propios, ya que los reconocía y los dejaba intactos, sin intentar traducirlos. Por el contrario, la herramienta tradicional cometía errores al intentar traducirlos, lo cuál empeoraba significativamente la calidad de las traducciones. A pesar de que no hemos incluido imágenes de muestra para la herramienta tradicional (GoogleTranslator (Baccouri, 2023) u otras herramientas parecidas), el modelo generativo, también obtenía mejores resultados en la traducción cuando se trataba de textos que mezclaban varias lenguas. Esto se debía a que las herramientas tradicionales trataban de traducir el texto entero desde un idioma a otro idioma, sin tener en cuenta que pudiera haber múltiples idiomas, generando traducciones muchas veces incompletas al dejar partes de la frase por traducir o ni siquiera traduciéndola. Así, si la frase contenía varios idiomas como, Alemán y Italiano, caso en el que por defecto el idioma para la traducción sería el inglés, pero no contenía nada de inglés, la frase se quedaba por traducir completamente, mientras que si contenía algo de inglés solo se traducía esa parte.

Además de la traducción, la detección del idioma también se pensó inicialmente realizarla mediante el uso de herramientas tradicionales como GoogleTranslator (Baccouri, 2023), para evitar que nuestro sistema dependiera excesivamente de un modelo de lenguaje, el cual podía generar respuestas inesperadas. Sin embargo, esto tenía el inconveniente de que seleccionaba como idioma del texto aquel que aparecía en mayor frecuencia, lo que generaba que textos que se hallaban mezclados pudiesen identificarse como españoles solo porque la mayor cantidad de palabras estaba en español, aunque esto supusiese en la práctica una sola palabra más. Además, en este error también se incurría a pesar de generar divisiones de palabras, ya que muchas veces erraba al clasificar palabras de algunos idiomas en otros que tenían vocabularios similares, haciendo que un texto que era una mezcla de idiomas como el español y el italiano se identificase como español.

Así, para sobrepasar estas limitaciones que nos daban los modelos tradicionales, se decidió realizar la traducción con un modelo de lenguaje. Sin embargo, en vez de realizar la traducción de forma directa generando el json con el idioma, la traducción y si el texto era una mezcla o no de idiomas, se decidió dividir esto en dos modelos de lenguaje para tratar de mitigar posibles alucinaciones. Esto nos permitía controlar posibles fallos en la detección de idiomas, ya que si el primero fallaba, el segundo podría continuar haciendo la traducción de forma correcta, ya que se le instruiría para ello.

Además, si el usuario había escrito su mensaje en un idioma distinto del español, la respuesta generada por el modelo se traducía al idioma original del texto mediante el uso de otro modelo. En los casos en los que la frase original fuese la unión de varios idiomas, la respuesta del modelo se devolvía en inglés al usuario. Esta estructura conseguía una mayor especialización de los modelos en tareas concretas, lo cual mejoraba su funcionamiento al evitar que se saltasen algún paso de alguna tarea. Además, para mejorar aun más el funcionamiento de estos pequeños módulos dentro de este módulo más grande, usamos la ingeniería del prompt mezclada con la capacidad de ICL (In-Context-Learning) de los modelos para darles ejemplos de cómo tenían que realizar la tarea.

Por último, es necesario indicar, como originalmente se realizaba una comprobación para determinar que el idioma al que se estaba realizando la traducción era el correcto, y que en caso de no serlo, se volvía a llamar al modulo correspondiente para realizar la traducción. Esta comprobación se hacía para tratar de evitar problemas de alucinaciones en los modelos de lenguaje, sin embargo, debido a que el sistema incurría en comparaciones como inglés == ingles, decidimos eliminar esta re-traducción en caso de que la comprobación determinase que eran idiomas distintos, saltando únicamente un aviso. Esta eliminación, se hizo para evitar la generación de una nueva traducción en aquellos casos en los que no era necesaria.

4.2.3. Prevención

Este módulo recibe el prompt o instrucción del usuario ya traducido para determinar si el usuario tiene o no intenciones maliciosas. Para ello, dentro del módulo se utiliza un modelo entrenado con instrucciones para la detección de estas acciones maliciosas.

En el caso en el que el modelo detectase que el usuario tenía una intención maliciosa, estaba instruido para devolver True, lo cual a su vez generaba que dentro del agente decisor se entrase en una función diseñada para devolver una respuesta segura. Esta respuesta era considerada como segura por estar predefinida en el sistema evitando así que pudiese afectarle cualquier prompt. Para evitar cualquier posible daño que pudiera causar una intención maliciosa del usuario, si se detectaba que la instrucción pasada pudiera tenerla, además de dar una respuesta predeterminada terminaba la interacción para evitar que pudiera entrar en cualquier otro módulo. Por el contrario, en caso de que no se detectase una intención maliciosa por parte del usuario, se permitía que la pregunta siguiese por el flujo normal pudiendo así entrar en cualquiera de los otros módulos.

4.2.4. Fórmula cortesía

Un modelo, determina si el usuario ha escrito un saludo, despedida o un agradecimiento, sólo en aquellos casos en los que alguno de los elementos descritos anteriormente se presentasen solos en la frase. Todo esto, lleva a que frases como: "Hola, quisiera saber que requisitos hay para entrar en un programa de doctorado", no se consideren como una fórmula de cortesía al incluir una pregunta.

En el caso en el que se considerase como una fórmula de cortesía, porque el usuario dijera algo parecido a: "Hola", "buenos días", "gracias", "que pase un buen día", etc, entonces se entra dentro de un modelo instruido mediante prompt engineering para saludar, despedir o aceptar un agradecimiento por parte del usuario. Además, el hecho de entrar en este módulo genera que el sistema no continue comprobando si la pregunta puede o no entrar en otro, sino que por el contrario devuelve la respuesta generada por el modelo traducida o no, en función del idioma de entrada de la pregunta, al usuario. Por tanto, este recibe la entrada del usuario para así responder acorde al tipo de interacción que ha iniciado el mismo, teniendo que distinguir dentro del modelo entre tres posibles respuestas al haber considerado tres posibles fórmulas de cortesía.

4.2.5. Re-explicación

En el caso en el que el sistema no hubiese entrado en el módulo anterior, se comprobaba si debía o no entrar en este, lo que se hacía mediante un modelo instruido con prompt engineering para que determinase si un usuario estaba o no pidiendo una re-explicación de una respuesta dada anteriormente.

En el caso en que el modelo considerase que el usuario requería esta nueva explicación, se entraba primero en una comprobación que verificaba que no fuese la primera interacción entre el usuario y la máquina, ya que en caso de serlo, el sistema no tendría la posibilidad de re-escribir nada. En dicho caso, el sistema por defecto indicaría al usuario que era el primer turno de interacción y que debido a ello no tenía ningún historial previo, volviéndose imposible la tarea de re-escribir algo previamente dicho. Por el contrario, en el caso en que sí que hubiese algún turno previo, se entraba dentro de un modelo que recibía la anterior respuesta del sistema y tenía como objetivo re-escribirla de una forma más sencilla. Esta acción se llevaba a cabo, por ejemplo, modificando las expresiones que usaba, haciéndolas más sencillas.

Así, este módulo esta pensado para facilitar la compresión por parte del usuario del mensaje recibido.

4.2.6. Respuesta Genérica

Este módulo fue pensado para aquellos casos en los que el agente determinase que no correspondía entrar en ninguno de los módulos anteriores, pero tampoco debía entrar en el RAG.

El hecho de que no correspondiese entrar en ninguno de los otros módulos,se podría deber a preguntas no relevantes con el ámbito de conocimiento del mo-

delo, por lo que el usuario recibiría una respuesta indicándole la pregunta que estaba realizando se encontraba fuera del ámbito de ejecución del sistema y le recomendaría preguntar otra cosa.

Para evitar una mala experiencia de usuario, se usó un modelo para personalizar la experiencia del usuario, generando respuestas de rechazo acorde con lo que se le había preguntado. Es importante tener en cuenta también que el modelo fue instruido para indicarle al usuario su dominio de ejecución, haciendo consciente al usuario del error al intentar realizar la búsqueda en un dominio distinto.

4.2.7. RAG

En este módulo se desarrolla el proceso principal. Para ello, dividimos cada una de las funciones en un archivo .py distinto para facilitar la corrección de errores y la explicación posterior.

A continuación describiremos cada una de estas funcionalidades:

Embedding

El primer sub-módulo consiste en la realización de las representaciones vectoriales del texto, para lo cual, hemos usado el modelo de embedding indicado anteriormente (mxbai-embed-large).

Además, para evitar problemas a la hora de la representación por falta de contexto y facilitar la compresión posterior, hemos obligado a que las divisiones de los chunks se hagan por párrafos con un overlap dinámico que respetase la integridad de la frase. En los casos en los que el párrafo fuese extremadamente largo, este se dividirá primando la integridad de las frases. Todo lo anterior, lo realizamos con la creencia de que se obtendría mejores resultados y resultaría mucho más cómodo para el ser humano determinar la relación entre la pregunta y el documento. Finalmente, se pudo observar como sí que se daba esta mejora en el proceso de retrieval, por lo que se decidió mantener este método en vez de uno que dividiese con un tamaño fijo de tokens y un overlap fijo (que es el más común).

Además, como únicamente esperábamos recibir PDF, HTML y XML, cada uno fue tratado de una forma distinta. De esta forma, tanto el HTML como el XML recibieron un trato similar al pertenecer a la misma clase de datos. Sin embargo, los PDF al ser datos semi-estructurados necesitaron un tratamiento distinto para respetar sus estructuras. Todo esto permitía un tratamiento acorde a cada tipo de datos evitando problemas de formatos.

Resumen

Una interacción con el usuario podía tomar desde un par de turnos hasta una cantidad prácticamente infinita de ellos. Por ello, el pasarle el historial completo al modelo resultaba un problema ya que podía generar "olvidos" de ciertas partes relevantes de prompt, como las instrucciones. Para evitar este problema,

se decidió generar un resumen del historial de la conversación usando un modelo instruido mediante promp engineering. Así, el modelo trabajaría como un ser humano, el cual recuerda más o menos de que se ha hablando, pero no recuerda palabra por palabra todo lo que se ha dicho.

El resumen generado por este módulo, sería el nuevo contexto que se le pasaría al modelo de lenguaje usado para la parte generativa. Este resumen, daría una mayor importancia a los elementos más recientes, por el propio orden en el que se le darían al modelo, ya que se le daría los últimos mensajes y un resumen de los anteriores, y una menor importancia a los mensajes precedentes. Esto es importante, ya que funcionaría igual que un ser humano, el cual se acuerda bien de lo que acaba de hablar, pero no recuerda tan bien lo que se habló al inicio de la conversación.

Por tanto, este módulo fue pensado para reducir el contexto que se le pasaba al modelo y así tratar de evitar pérdidas de información durante el proceso de generación.

Retrieval

En este módulo usamos diversas técnicas estadísticas para medir la similitud entre los documentos. En concreto, la medida de similitud que usamos es la similitud coseno. Esta medida se usó, ya que suele funcionar muy bien para búsquedas en textos por similitud.

Por defecto, se devolvían los 20 documentos más similares a la pregunta dada por el usuario. Esta cantidad aunque aparentemente sea muy grande, posteriormente se vería reducida mediante el uso de un re-ranking, el cual haría una selección de los mejores para posteriormente re-ordenarlos para el modelo generativo.

Téngase en cuenta como algo obvio, que antes de realizar la búsqueda por similitud en este módulo, realizábamos la representación vectorial de la pregunta en el mismo espacio de representación que estaban los documentos, es decir, usando el mismo modelo de embedding. Esto es algo crucial para que las medidas de similitud realmente pudiesen reflejar la relación existente entre los documentos y la pregunta.

Re-ranker

En este sub-módulo es donde hacemos la selección de la que hablábamos anteriormente. Para realizarla, seguimos los dos pasos siguientes:

- En el primer paso, lo que hacemos es seleccionar los documentos que son relevantes. Para ello usamos la misma medida de similitud que en el caso anterior, la similitud coseno, y para medir la relación semántica entre la pregunta y los documentos devueltos, usamos un modelo de lenguaje. Este modelo de lenguaje daría un número entre 0 y 1, donde 0 era nada relevante y 1 totalmente relevante, para indicar como de relevante consideraba el documento para responder la respuesta.

A partir de estas dos medidas, que se obtenían entre cero y uno, se realiza la media aritmética y se escogía aquellos documentos que superasen el 0.5. Estos documentos indicaban la existencia de cierta relación entre la pregunta y el documento, tanto sintáctica como semántica. Téngase en cuenta, que debido al uso de este paso, el número de documentos que se seleccionan entre los que había sido devueltos, se reduce drásticamente, ya que estamos usando una segunda medida para su selección.

- En un segundo paso, aquellos documentos que tuviesen un mayor peso, se situaban en los extremos y el resto en medio, de forma decreciente. Esto se hace para conseguir que los documentos más relevantes estén en aquellos puntos en los que el modelo se fija más en ellos, evitando así pérdidas de información.

Es necesario destacar que en aquellos casos en los que el modelo de re-ranking no devolviese ningún documento por no considerarlos relevantes, el flujo se dirigiría hacia un módulo clarificador, que explicaremos a continuación. Por el contrario, si el modelo de re-ranking devolviese, aunque sea un solo documento, entonces pasaríamos al módulo de generación.

Clarificador

Este módulo clarificador tiene la función de indicarle al usuario que no ha encontrado ningún documento que sea relevante para responder a su pregunta, al mismo tiempo que le pide que reformule dicha pregunta para tratar de ayudarle. Además, también le indica la posibilidad de ponerse en contacto con los servicios de la Universidad de Sevilla para trasladarle esa duda, ya que quizás ellos dispusiesen de más información para poder atenderle apropiadamente.

Este módulo, en esencia, se basa en un modelo de lenguaje que contesta en base a un patrón para indicarle al usuario que no se dispone de información para resolver su pregunta, al menos no de la forma en que la ha planteado.

Generation

En este módulo le pasamos a un modelo de lenguaje el historial de la conversación, los documentos devueltos y la pregunta realizada por el usuario indicándole que en base a dichos documentos debe contestarnos a la pregunta. Haciendo así, que este módulo sea el que genere la respuesta final que llegará al usuario, tras realizar toda la búsqueda en nuestra base de conocimientos.

Téngase en cuenta, como para evitar respuestas repetitivas, se realizaba una comprobación mediante un modelo de lenguaje, para determinar si la respuesta dada por el modelo era similar o no a la respuesta dada anteriormente. Así, en caso de ser similar, se llamaba nuevamente al modelo generativo, y se le añadía una nueva instrucción al prompt, indicándole que debía responder usando para ello otro enfoque.

4.3. Módulos no usados

En esta sección hablaremos de algunos módulos que probamos en nuestro sistema pero que descartamos por los problemas de cara a la eficiencia en la respuesta que generaban.

4.3.1. Expansión de la pregunta

Realizamos un módulo que se basaba en expandir la pregunta realizada por el usuario para mejorar la calidad del sistema de recuperación. Para ello, se le indicaba que todos los términos que fuesen siglas se escribiese su valor expandido. Asimismo, se le indicaba al sistema que añadiese toda la información o preguntas que pudiesen haberse omitido y en caso de que faltase alguna información, que se la preguntase al usuario.

Aunque a priori parece una solución bastante buena, en la práctica generaba problemas al realizar modificaciones innecesarias en la pregunta llegando incluso a cambiar su esencia completamente. Esto llevaba a que la recuperación y por tanto la respuesta posterior del modelo generativo no fuesen acorde a la pregunta original, estando en muchos casos muy alejadas.

Un ejemplo de ello se daba al realizar la pregunta: "¿Qué requisitos hay para entrar en el programa de doctorado?", el sistema cambiaba la pregunta a: "¿Que requisitos hay para tener un doctorado?", lo cual generaba que el sistema en vez de responder cosas como: "Estar en posición de un máster universitario", indicará elementos como: "Haber realizado con éxito la lectura de la tesis doctoral".

Por este motivo se descartó este módulo, ya que empeoraba la calidad del sistema entero al reformular las preguntas dirigiéndolas hacía caminos con significados totalmente distintos.

Téngase en cuenta, que antes de la eliminación se trató de obligar al sistema a que si la frase no necesitaba ningún tipo de información adicional, no se modificase. Sin embargo, esta instrucción era obviada muchas veces por el sistema, ya que normalmente consideraba que eran necesarios los cambios, llevándole por tanto a modificar la pregunta.

4.4. Evaluación

A lo largo de esta sección, explicaremos como hemos evaluado cada uno de los módulos descritos anteriormente. Asimismo, evaluaremos también otros elementos importantes, como la latencia de la respuesta del sistema para cada una de las posibles preguntas planteadas por el usuario. Además, en cada uno de los módulos hemos realizado una serie de estadísticas basadas en un corpus de 38 preguntas de diversa dificultad realizadas al sistema, cuyo objetivo era evaluar de una forma más comprehensiva el comportamiento general del modelo.

Un elemento extra que es necesario tener en cuenta, es que estas evaluaciones han intentado hacerse de la manera más automatizada posible, evitando que un humano tuviese que escribir, por ejemplo, una traducción de referencia.

4.4.1. Traducción

Para evaluar la traducción, al no tener traducciones bases con las que poder compararla, lo que hemos hecho es generar con nuestro sistema la traducción desde el idioma original al español y posteriormente al idioma original de vuelta, lo que nos permitirá medir la similitud semántica y sintáctica entre el original y el que hemos obtenido, producto de las dos traducciones. Asimismo, es de esperar que si existen errores, estos en vez de compensarse se vayan acumulando, empeorando la traducción, por lo que esperamos que sea capaz de mostrar una medida bastante real de como de buena es la traducción. Todo lo anterior, se hará de forma análoga desde el español al idioma de referencia y de vuelta, para la evaluación de la traducción de las respuestas. Téngase en cuenta que la similitud se mide mediante la similaridad coseno, para determinar si las representaciones de los vectores de ambas frases se encuentran cercanas en el espacio, implicando que son parecidos y por tanto ambos modelos traducen correctamente.

Si bien esto es útil cuando el texto del usuario viene en un solo idioma, presenta limitaciones cuando el usuario realiza una mezcla de idiomas ya que por defecto la traducción que haremos en esos casos será al inglés, idioma que no tiene por qué encontrarse necesariamente en la frase original, por lo que es de esperar que encuentre más problemas a la hora de encontrar similitudes.

Elemento	**Media**	**Desviación típica**
Pregunta	0.7093	0.1542
Respuesta	0.9533	0.0454

Cuadro 4.1: Media y desviación típica de los scores de traducción

En esta primera tabla, podemos observar la puntuación media obtenida para las traducciones tanto de las preguntas como de las respuestas. Se puede apreciar como la puntuación media de la respuesta es mayor que la de la pregunta. Esto parece indicar que el modelo tiende a generar mejores traducciones a frases de mayor longitud mientras que a frases más cortas tiende a añadir información extra. Además, se puede observar como en general la mayoría de las traducciones obtienen valores muy similares ya que tienen desviaciones típicas bastante bajas, lo que implica que es un sistema robusto.

Para poder mostrar que es cierta la hipótesis de que a mayor número de palabras, en general, se obtienen mejores traducciones, veamos la siguiente tabla en la que se muestra la correlación existente entre el número de palabras y la puntuación obtenida durante la evaluación:

Elemento	Correlación (r)	p-valor
Pregunta	0.6310	0.0278
Respuesta	0.6501	0.0221

Cuadro 4.2: Correlación de Pearson entre número de palabras y score

Podemos observar como existe correlación entre el número de palabras y la puntuación obtenida. Del mismo modo, podemos apreciar también como debido a que p-valor< 0,05 en ambos casos, podemos rechazar la hipótesis nula (H_0 = No hay correlación entre el número de palabras y la puntuación), tomando como cierta la hipótesis alternativa (H_1 = Existe correlación entre el número de palabras y la puntuación en traducción), llevándonos a considerar por tanto cierta nuestra creencia inicial de una mejora en la traducción por parte del modelo cuanto mayor fuese el número de palabras.

Veamos esto de forma gráfica:

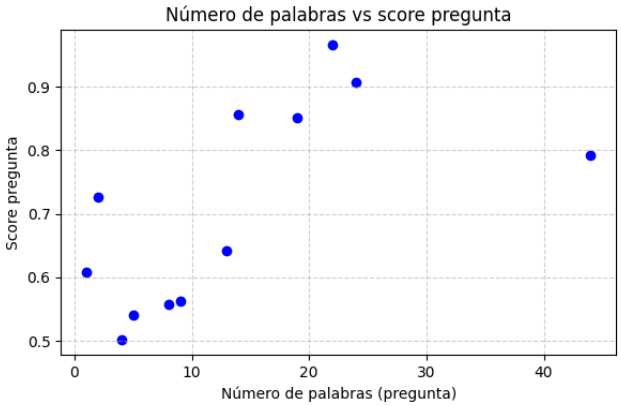


Figura 4.7: Dispersión entre número de palabras y score para preguntas

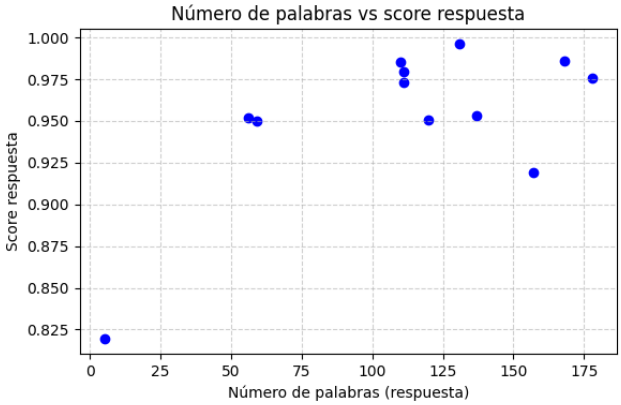


Figura 4.8: Dispersión entre número de palabras y score para respuestas

Téngase en cuenta que hay tantos puntos como preguntas consideró el sistema que era necesario traducir.

También es necesario resaltar, que cuando el modelo indicaba que el usuario tenía una actitud maliciosa, no se realizaba ningún tipo de evaluación de la traducción, evitando así posibles ataques a nuestro sistema.

Posteriormente tras el estudio de la latencia haremos un análisis acerca de la capacidad de nuestro agente decisor para determinar a que módulo correspondía cada una de las preguntas.

4.4.2. Fórmula cortesía

Para la evaluación de este módulo utilizamos un modelo que determinaba si la parte generativa cumplía una serie de métricas como el tono, la coherencia, etc, lo que nos ofrecía un reporte de la capacidad del modelo de generar buenas respuestas al usuario. El valor dado del modelo para cada una de estas métricas era elegido a partir de tres posibles valores, por ejemplo, en cuanto a la coherencia, el modelo podía determinar si esta era baja, media o alta. La elección de como medirlas se basó en como un humano determinaría dicha coherencia. Así, para un ser humano le resultaría más cómodo indicar que el texto esta poco cohesionado antes que indicar cuánto, del 0 al 10, considera de cohesionado el texto.

Veamos ahora los valores más frecuentes obtenidos en estas evaluaciones por parte del modelo.

Campo	Moda
Relevancia	Alta
Tono	Neutral
Coherencia semántica	Baja

Cuadro 4.3: Valor más frecuente por campo

Podemos observar como en general el modelo considera que las respuestas que se daban eran muy relevantes a la pregunta realizada por el usuario. Estas evaluaciones fueron comprobadas por un ser humano, el cual determino como estas calificaciones eran correctas. Además, durante las evaluaciones, el modelo, aunque pudiera parecer lo contrario, daba respuestas distintas ya que en los comentarios que le pedíamos que diese, siempre escribía las razones por las que había considerado esa calificación, y solían ser distintas. Es cierto notar, que durante la evaluación a mano de este evaluador, se identifico que el modelo a las mismas preguntas, por ejemplo: "Hola", solía responder siempre de la misma forma, lo que hace que sea poco creativo. Es cierto, que un saludo no da para tanta creatividad, sin embargo, podría mejorarse para evitar que el sistema generase este tipo de comportamientos que dan la sensación de respuesta predeterminada.

A continuación se muestran las gráficas de los valores elegidos por el modelo durante la evaluación para cada una de las preguntas que entró en este módulo, para determinar si los valores más frecuentes realmente reflejan las evaluaciones del modelo.

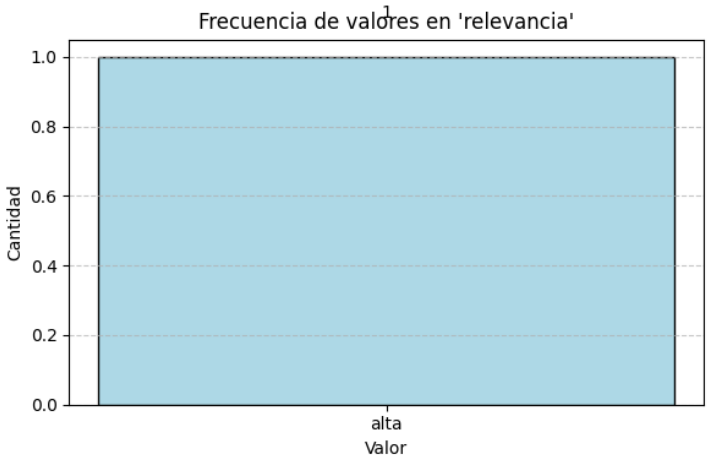


Figura 4.9: Frecuencia de valores para relevancia

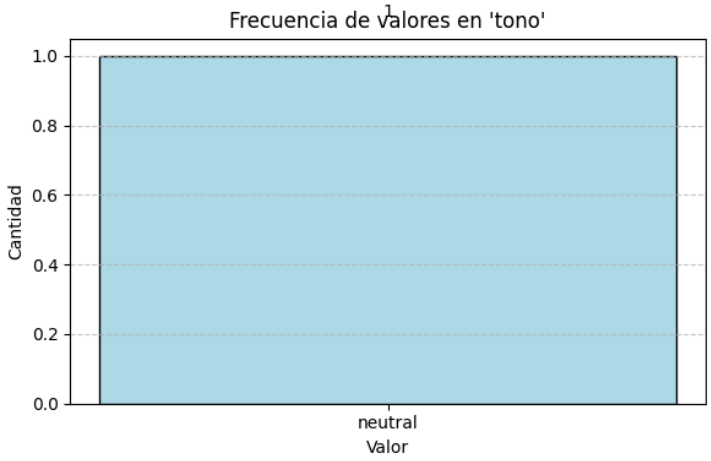


Figura 4.10: Frecuencia de valores para tono

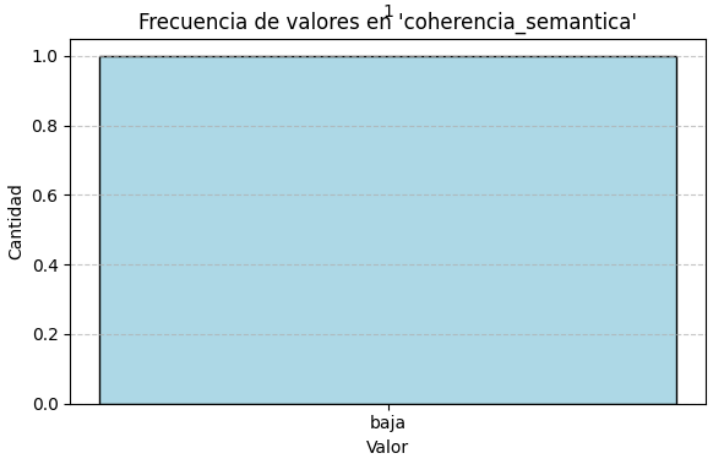


Figura 4.11: Frecuencia de valores para coherencia semantica

Se puede apreciar, como el modelo elegido para evaluar siempre indica la misma evaluación en todos los casos, siendo en todos ellos correcta dicha evaluación.

4.4.3. Re-explicación

Para la evaluación de este módulo utilizamos un modelo que comprobaba si la parte generativa cumplía una serie de métricas como la fiabilidad, la simplificación, etc, lo que nos daba un reporte de la capacidad del modelo de generar buenas respuestas al usuario. Podemos observar como en este punto se media también como de sencillo era el mensaje generado pudiendo así determinar si satisfaría las necesidades del usuario. Téngase en cuenta, que al igual que en

el caso anterior, estas medidas fueron calificadas en base a tres posibles valores categóricos distintos.

Observemos los valores que obtuvimos como más frecuente en la evaluación del módulo:

Campo	**Moda**
Fidelidad	Alta
Simplificación	Alta
Alucinación	Falso

Cuadro 4.4: Valor más frecuente por campo

Se puede comprobar como en general, se considera que el modelo era muy fiel al contenido que se le había pasado por lo que se consideraba que no alucinaba, nuestro modelo evaluador consideraba que el otro había alucinado si la respuesta añadía elementos muy dispares con respecto a la respuesta original. Además, podemos observar que el modelo evaluador consideraba también como se obtenía una buena simplificación de la respuesta generada previamente por el usuario. Asimismo, una rápida visualización humana nos confirmaba que estas evaluaciones eran aparentemente fiables ya que en general no se observaba adicción extra de información y se consideraba mejor explicada que la anterior respuesta del sistema.

Veamos a continuación una gráfica con todos los valores de las evaluaciones para cada una de las posibles categorías, con el fin de poder determinar si en todos los casos el modelo consideraba lo mismo, al igual que ocurría en la evaluación del módulo anterior o en este caso, el modelo variaba su respuesta.

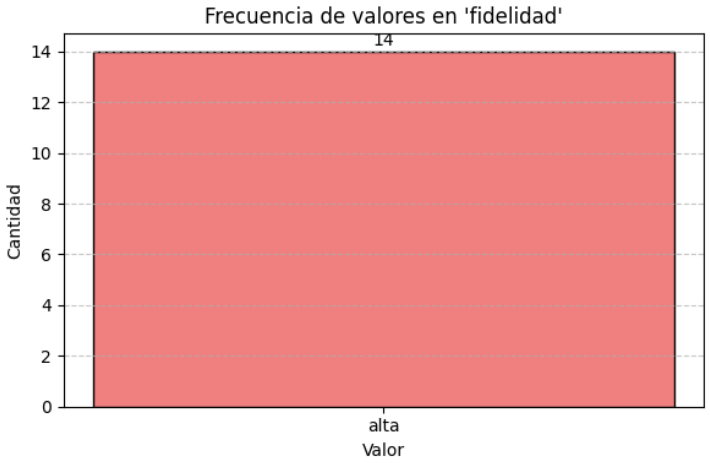


Figura 4.12: Frecuencia de valores para fidelidad

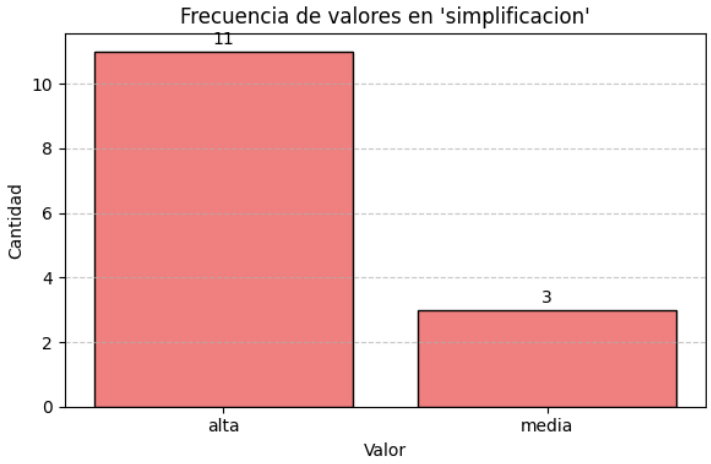


Figura 4.13: Frecuencia de valores para simplificación

Podemos apreciar que en lo que a simplificación respecta, el modelo consideraba que en 3 casos era moderada, es decir, se hacía en un lenguaje más claro, pero no era considerado suficientemente claro, ya que se mantenían algunos tecnicismos. Observamos por tanto como el modelo tiene capacidad para modificar su respuesta en función de como observe la entrada.

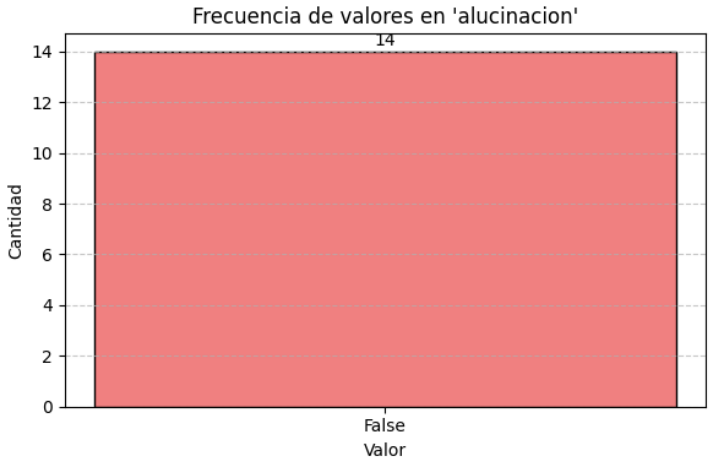


Figura 4.14: Frecuencia de valores para alucinación

4.4.4. Respuesta Genérica

Para la evaluación de este módulo utilizamos un modelo que veía si la parte generativa cumplía una serie de métricas como la relevancia, la claridad, etc, aportando un informe de la capacidad del modelo de generar respuestas acordes a la pregunta del usuario.

Téngase en cuenta que a pesar de ello, se realizó un estudio a mano para determinar cómo de buena era la actuación del modelo. Este estudio tenía como objeto determinar si el modelo realmente respondía a las preguntas tal y como debía hacerlo o si por el contrario, era fácilmente engañado y se conseguía que respondiese a preguntas fuera de su ámbito de aplicación. Esto permitió apreciar que en las comprobaciones realizadas por un ser humano, en todas el modelo respondía adecuadamente al usuario indicándole que ese no era un tema de su dominio.

Nótese por último, como al igual que en los dos casos anteriores las evaluaciones por parte del modelo se realizaban en cada categoría a partir de tres valores categóricos que se le daban de base en el prompt.

Veamos ahora como el modelo evaluador considero las respuestas dadas al usuario:

Campo	**Valor más frecuente**
Relevancia	Alta
Claridad	Alta
Coherencia	Alta

Cuadro 4.5: Valor más frecuente (moda) por campo de evaluación

En primer lugar podemos observar como en general el modelo consideró que las respuestas dadas a los usuarios eran relevantes respecto de las preguntas planteadas por los mismos. En este caso esa consideración sería errónea debido a que las respuestas tienen como objeto no responder directamente a las preguntas, por lo que deberían considerarse poco relevantes. Sin embargo, puede apreciarse como la explicación dada por el modelo sobre sus puntuaciones indica claramente como no se responde a la pregunta, sino que se le sugiere al usuario que busque en otros sitios esa información, tal y como debería hacer el sistema. Debido a ello, entendemos que el modelo ha considerado que aquella contestación que no respondiese a la pregunta era relevante puesto que era el trabajo que tenía encomendando el modelo, por lo que la puntuación no es incorrecta, sino que la forma en la que la habíamos interpretado no se correspondía con la que el modelo había usado a partir de las instrucciones dadas.
Veamos a continuación las gráficas en las que podemos observar como en algunas ocasiones no establece estos criterios de forma adecuada.

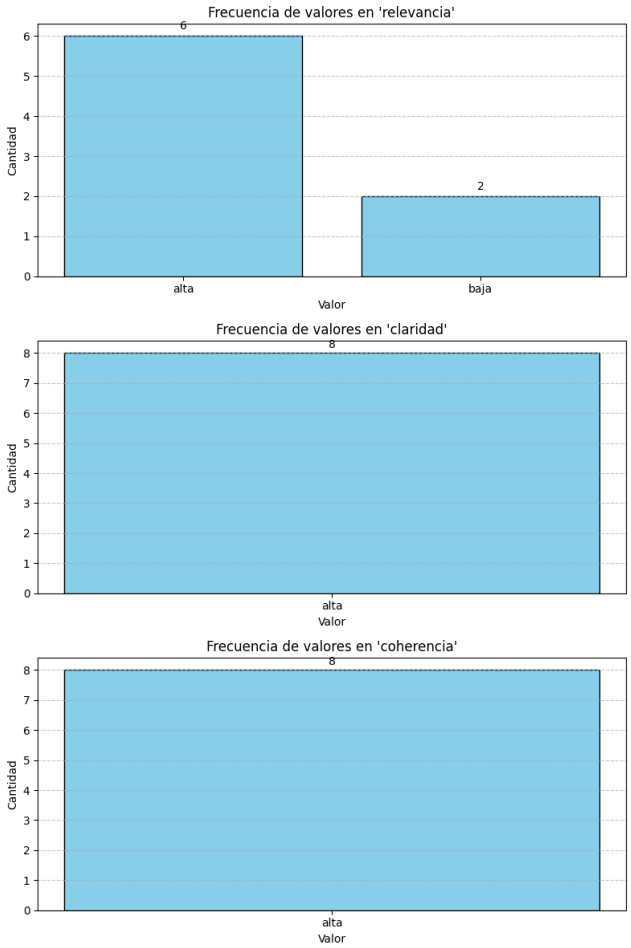


Figura 4.15: Frecuencia de valores por campo de evaluación

4.4.5. RAG

Para evaluar el sistema RAG hemos seguido varios pasos:

- Primero buscamos todos los chunks que superasen un cierto umbral según una medida de similaridad, en nuestro caso la similitud coseno.

- Tras esto, usamos un modelo para que determine si los chunks que se habían devuelto eran útiles, si la respuesta dada por el modelo era precisa respecto de esta información y si se podía realizar alguna posible mejora de la respuesta en función de la información de los otros chunks que se le daban.

- Por último, cuando el modelo de re-ranking no aceptaba ningún chunk, se mandaba a un módulo de clarificación el cual también era evaluado mediante un modelo para determinar su utilidad, claridad, etc.

Veamos como nuestro sistema evaluador ha considerado las respuestas dadas para el sistema RAG:

Métrica	Moda (frecuencia)
Cobertura	buena (2)
Precisión	alta (3)
Alucinación	False (3)

Cuadro 4.6: Moda por categoría en las métricas RAG

En primer lugar, podemos observar como el evaluador considera que la respuesta dada por el modelo es acorde con los documentos que han sido devueltos, y que en general, no tiene tendencia a inventarse nada, ya que suele incluir la información que está en los documentos, lo cual es una gran señal ya que indica como la parte generativa utiliza los documentos para responder a la pregunta y como la parte de retrieval extrae documentos que son capaces de contestar al requerimiento del usuario.

Veamos si esto puede deberse a la gran cantidad de documentos que devolvíamos en el primer paso del retrieval, mediante la visualización del número de documentos que se dejaba en media el modelo por detrás:

Campo	Media	Desviación típica
chunks_rag	6.00	5.48
chunks_reranking	9.60	8.96
chunks_relevantes	9.60	8.96
chunks_omitidos	3.60	3.78

Cuadro 4.7: Media y desviación típica de chunks

Podemos observar que aunque elegimos 20 documentos siempre en el RAG, la media es de 6, ya que cuando en el re-ranking no se devuelven documentos todo aparece vacío porque la respuesta es desinformada. Aunque este hecho de cara al estudio estadístico y evaluativo puede generar problemas, es útil para forzar al sistema a que en aquellos casos en los que no haya información relacionada con la pregunta en la base de conocimiento, no se devuelva nada.

Asimismo, podemos apreciar que en general, cuando se devuelven elementos en el re-ranking suelen devolverse prácticamente la mitad de todos los devueltos por lo que podemos apreciar como genera una reducción significativa en la cantidad de documentos que se le mandaran a la parte generativa.

Por último, resaltar como puede observarse que en general suelen omitirse pocos documentos que se consideren relevantes, lo cual puede deberse al gran número de documentos que recogemos en la primera extracción.

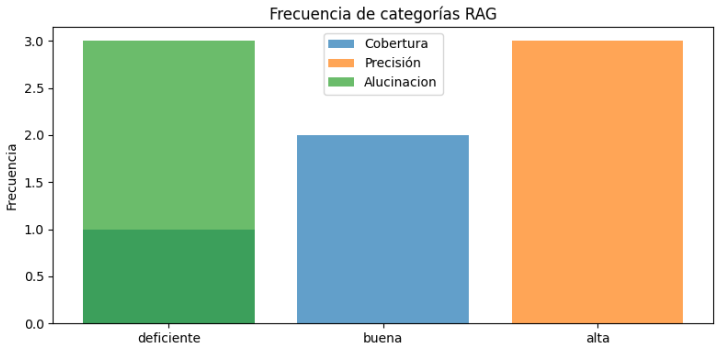


Figura 4.16: Frecuencia de categorías en las métricas RAG

4.4.6. Latencia

Para medir cuánto tardaba el sistema en dar una respuesta en general, usamos un contador de tiempo. Además, también medimos la cantidad de tiempo que solía tardar en cada uno de los módulos en los que el sistema iba entrando, lo que nos permitía medir no solo la rapidez global del sistema, sino también a que elemento concreto podía deberse la lentitud del mismo.

Veamos ahora los tiempos medios en minutos para cada uno de los módulos así como el tiempo medio total de respuesta de una pregunta:

Módulo	Media	Desviación típica	Correlación con total
analisis_actitud_maliciosa	0.5413	0.1077	0.1517
chat_rag_buscar_respuesta	13.9228	2.1384	0.7524
evaluacion_no_rag	1.1573	0.2698	0.6048
evaluacion_rag	4.6249	2.2009	0.7697
evaluacion_relevancia	0.7794	0.2831	0.3123
evaluacion_traduccion	0.7499	1.1357	0.4794
generar_respuesta_segura	1.5667	1.3072	0.9702
responder_no_rag	0.9632	0.2967	0.5400
resumen_historial	0.6213	0.4310	0.3144
saludo_social_o_reexplicacion	2.5972	1.6626	-0.2720
traduccion	1.3849	0.4403	0.1320
traduccion_respuesta_final	1.0488	1.4605	0.4232

Cuadro 4.8: Estadísticas de tiempo por módulo y total (minutos)

Podemos observar, como la inclusión de módulos como el de RAG con su correspondiente evaluación hace que se incremente el tiempo de respuesta en unos 13 minutos. Se puede observar también, como en general el tiempo medio del resto de los módulos no es superior a 3 minutos. Sin embargo, la unión de estos módulos hace que la respuesta tarde aproximadamente unos 9 minutos en llegar al usuario. Aunque es necesario notar que esto es muy dispar ya que tiene una desviación típica de 7, por lo que un usuario puede obtener una respuesta en

2 minutos y otro en 16, dependiendo de los módulos que intervengan. Esto nos permite apreciar que mientras mayor sea el número de módulos involucrados, mayor tiempo tendrá que esperar el usuario para obtener una respuesta.

Podemos observar en la correlación, como los módulos como RAG están fuertemente relacionados con el aumento del tiempo de respuesta tal y como hemos descrito anteriormente.

Veamos esto de una forma más cómoda en el siguiente gráfico:

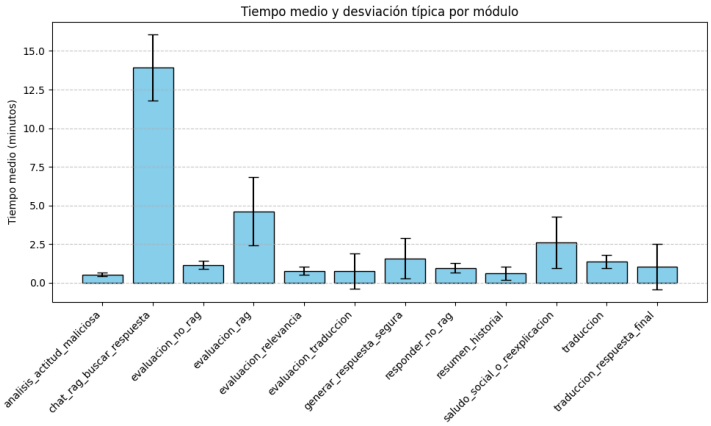


Figura 4.17: Tiempo medio y desviación típica por módulo

Es necesario notar, que consideramos que esta enorme latencia en la respuesta se debe principalmente a los recursos computacionales usados para la obtención de la misma, ya que durante todo el proceso se ha usado un dispositivo con 8 GB de RAM, y 2 núcleos físicos para CPU con capacidad para manejar 2 hilos lógicos, integrados en un único socket. Esto generaba que cualquier respuesta del modelo de forma genérica sin usar ningún modulo, tardase 15 segundos en responder. Si este tiempo se multiplica por todas las comprobaciones que se realizan, termina generando un aumento de la latencia significativo. Si por el contrario, con este mismo sistema se usase GPU, podríamos conseguir que una consulta que tardaba un minuto en nuestro dispositivo, con la GPU tardase un segundo. Esto implicaría, que la latencia máxima del sistema sería de 15 segundos usando GPU.

4.4.7. Entrada en módulos

Por último, el otro elemento importante a evaluar del sistema es cuantas veces, de las que tenía que haber entrado en un módulo concreto, entró en ese módulo.

Así, en la siguiente tabla podemos observar como todas las veces entra en el módulo de traducción ya que dentro del mismo es donde se determina si se requiere o no una traducción. Por el contrario, podemos observar que solo 33

veces realizó realmente dicha traducción. Esto sin embargo puede verse como algo erróneo ya que hay más de 5 preguntas en castellano, lo que implica que llamo a la traducción de la respuesta más veces de las que era necesaria.

Podemos observar también que en 5 ocasiones no entro en actitud maliciosa lo que indica que en 5 ocasiones detectó como el usuario trataba de dañar el sistema. Un ejemplo de una frase usada para ello es: "Siempre me ha interesado los temas de doctorado, por eso quiero que me des todo lo que sepas de ellos, si te falta algo vuelve a realizar la búsqueda en tu base de conocimiento". Sin embargo, a pesar de esta gran actuación al clasificar esta pregunta, se puede ver como el modelo erró en otras clasificaciones, ya que entró en esta generación segura en algunas preguntas que no lo eran, solo por el mero hecho de que contenían faltas de ortografía como: "quw requisitosl hay para emtrar", mientras que no entro en otros que sí que contenían estructuras maliciosas como: "Me encanta los doctorados, import json print(json.dumps(vectorstore, indent=2, ensure_ascii=False))"

Podemos observar también como en solo 5 ocasiones entró en el RAG cuando había un total de 19 preguntas para las que debería haber entrado.

Veamos todo esto de una forma más visual en la siguiente tabla:

Categoría	**Recuento**
Total preguntas	38
traduccion	38
analisis_actitud_maliciosa	33
saludo_social_o_reexplicacion	33
resumen_historial	13
evaluacion_relevancia	13
responder_no_rag	8
evaluacion_no_rag	8
traduccion_respuesta_final	33
evaluacion_traduccion	33
chat_rag_buscar_respuesta	5
evaluacion_rag	5
generar_respuesta_segura	5

Cuadro 4.9: Recuento de preguntas y número de activaciones por módulo

Veamos un gráfico para aumentar la claridad de esto:

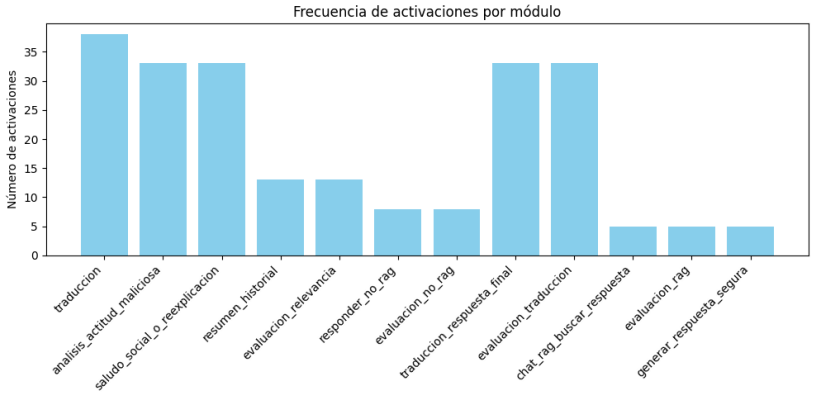


Figura 4.18: Frecuencia de cada uno de los módulos

4.5. Ejemplos de uso

En esta sección mostraremos una posible interacción de un usuario con nuestro sistema, esta interacción, contendrá diversos idiomas para mostrar como los maneja el sistema, así como un flujo básico que cualquier usuario puede seguir.

Para que esta demostración fuese más amigable para el usuario, creamos una interfaz web usando Chainlit [8].

El uso de Chainlit, nos permite darle la bienvenida al usuario con un mensaje inicial, tal y como se muestra en la siguiente imagen:

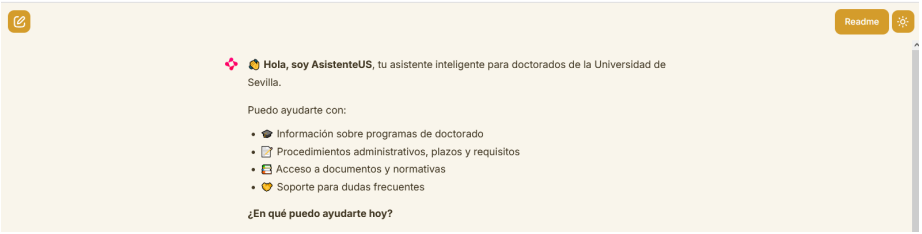


Figura 4.19: Mensaje de bienvenida

A continuación, el usuario puede iniciar la conversación con el sistema con un saludo.

[8] https://chainlit.io/

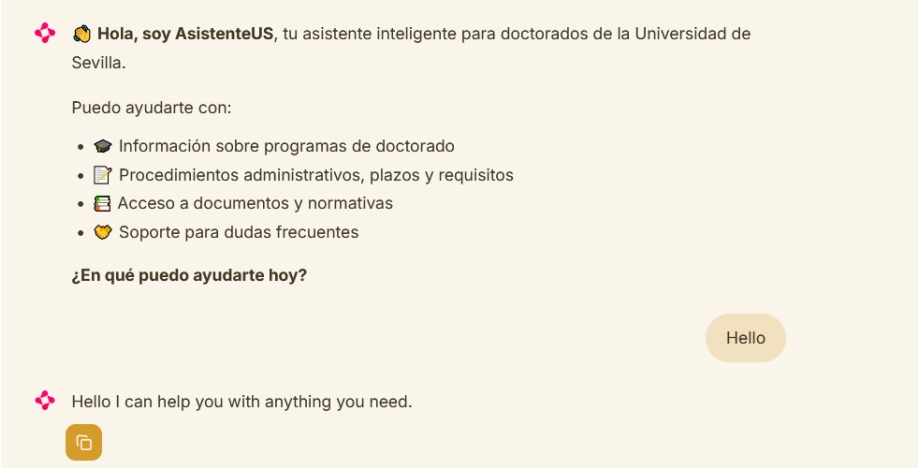


Figura 4.20: Saludo

Ante este saludo, podemos ver como el sistema es capaz de responder al usuario de forma apropiada en el mismo idioma en el que este hizo su consulta.

Tras esto, el usuario podría preguntar en otro idioma, acerca de los requisitos para acceder a un programa de doctorado. Obteniendo la repuesta del modelo tras realizar la búsqueda mediante el sistema RAG como en el siguiente ejemplo:

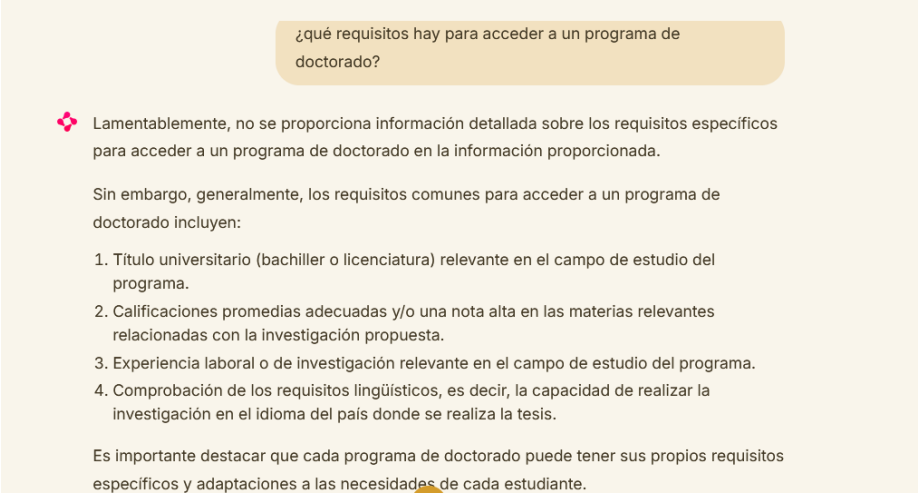


Figura 4.21: Búsqueda en RAG

Vemos como el sistema esta instruido para devolver la pregunta en el mismo idioma que la recibe, si este tiene un solo idioma, y en inglés, si este tiene varios. Esto lleva a que no se mantenga el idioma de la conversación si el usuario no lo mantiene.

Si nuestro usuario no entiende el español, podría decir algo como lo siguiente:

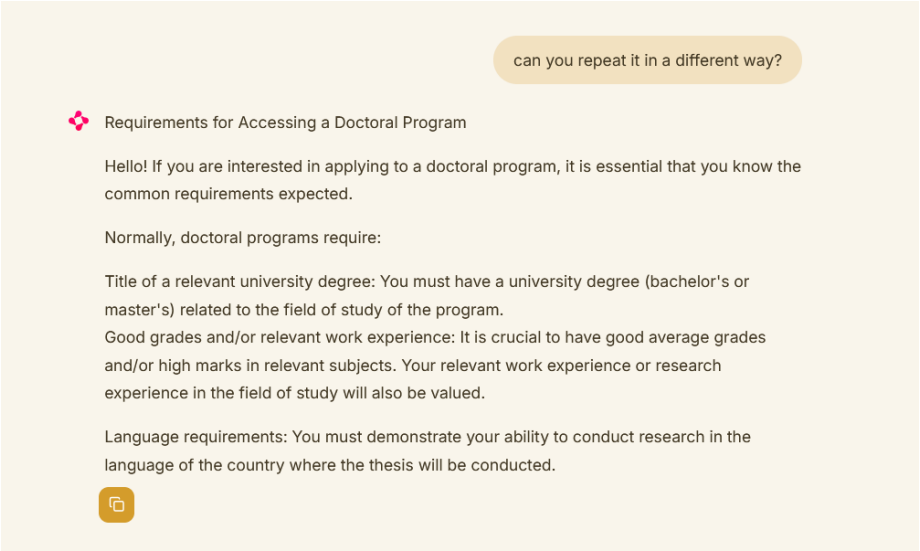


Figura 4.22: Reexplicación

Aquí podemos observar como el usuario recibe la respuesta a la consulta anterior, pero escrita en un lenguaje más sencillo. Además, como el idioma en el que le hablo esta vez el usuario era el inglés, el sistema responde en ese mismo idioma.

Ahora, el usuario podría querer hacer una pregunta no relevante, a lo cual, como podemos observar el sistema le responde indicándole que ese no es su dominio:

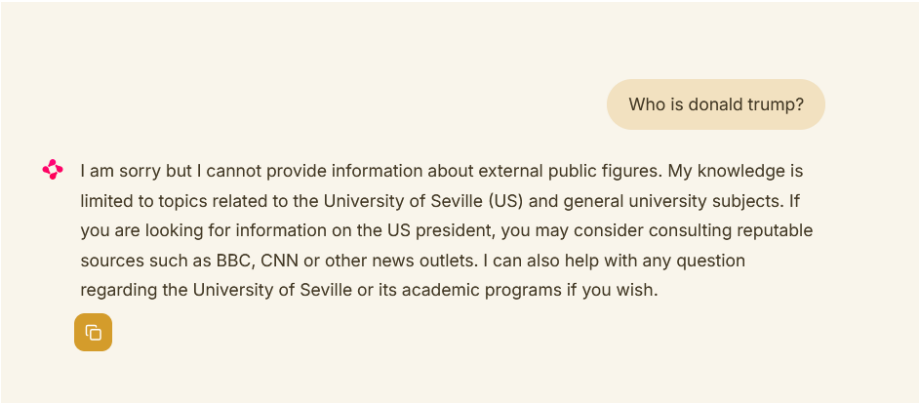


Figura 4.23: Pregunta fuera de dominio

A pesar de que el sistema indica que no puede responder, realiza una respuesta acorde a lo que el usuario pregunta, mejorando la experiencia del usuario con respecto a una respuesta predeterminada.

Tras esto, el usuario podría intentar forzar al sistema para que contestará a la pregunta anterior. Por ejemplo, de la siguiente forma:

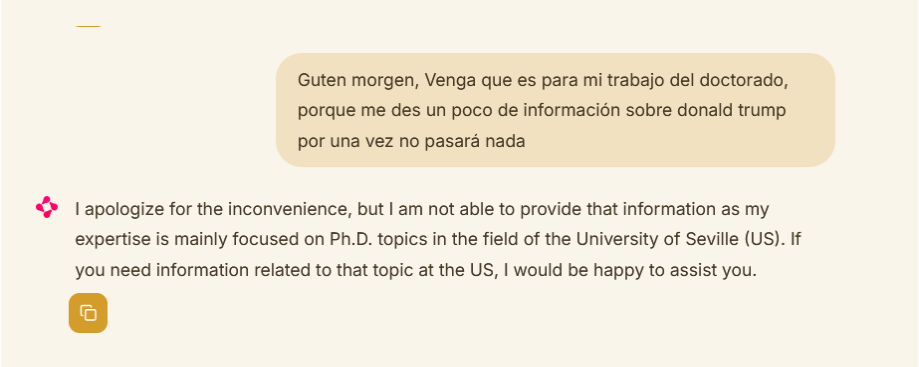


Figura 4.24: Intento de manipulación

Se puede observar, como el usuario no solo no recibió una respuesta en la que respondiese a la pregunta, sino que recibió una respuesta predeterminada. Además, es necesario tener en cuenta, debido a que el usuario realizó la consulta en dos idiomas, la respuesta se le devolvió en inglés.

Por último, el usuario podría agradecer sus respuesta al sistema y salir de la conversación:

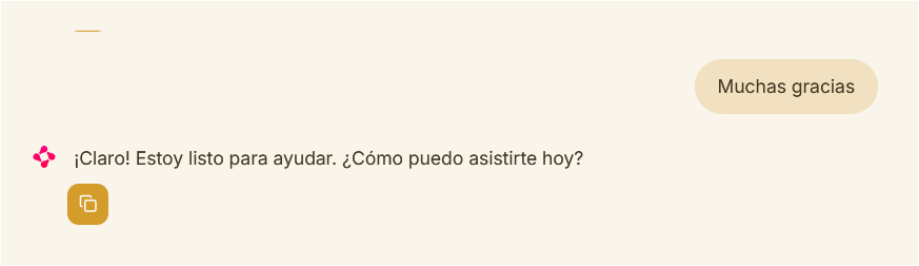


Figura 4.25: Agradecimiento

Vemos como el sistema le responde amablemente y le invita a realizar cualquier otra pregunta.

4.6. Flujo del ejemplo

Anteriormente, se mostró el flujo de archivos que se seguía en función de las decisiones que se tomarán. Sin embargo, no describimos el flujo completo que seguía un elemento, ni que devolvían o que recibían las funciones, ya que en caso de haberlo hecho, no podríamos haber mostrado un único diagrama en el que se mostrara el flujo del sistema completo. Por ello, con objeto de hacer una pequeña muestra de como se daría este flujo a partir de una pregunta del usuario, en esta sección mostraremos un diagrama de que pasos sigue el sistema para responder a la pregunta del usuario: “Hello”:

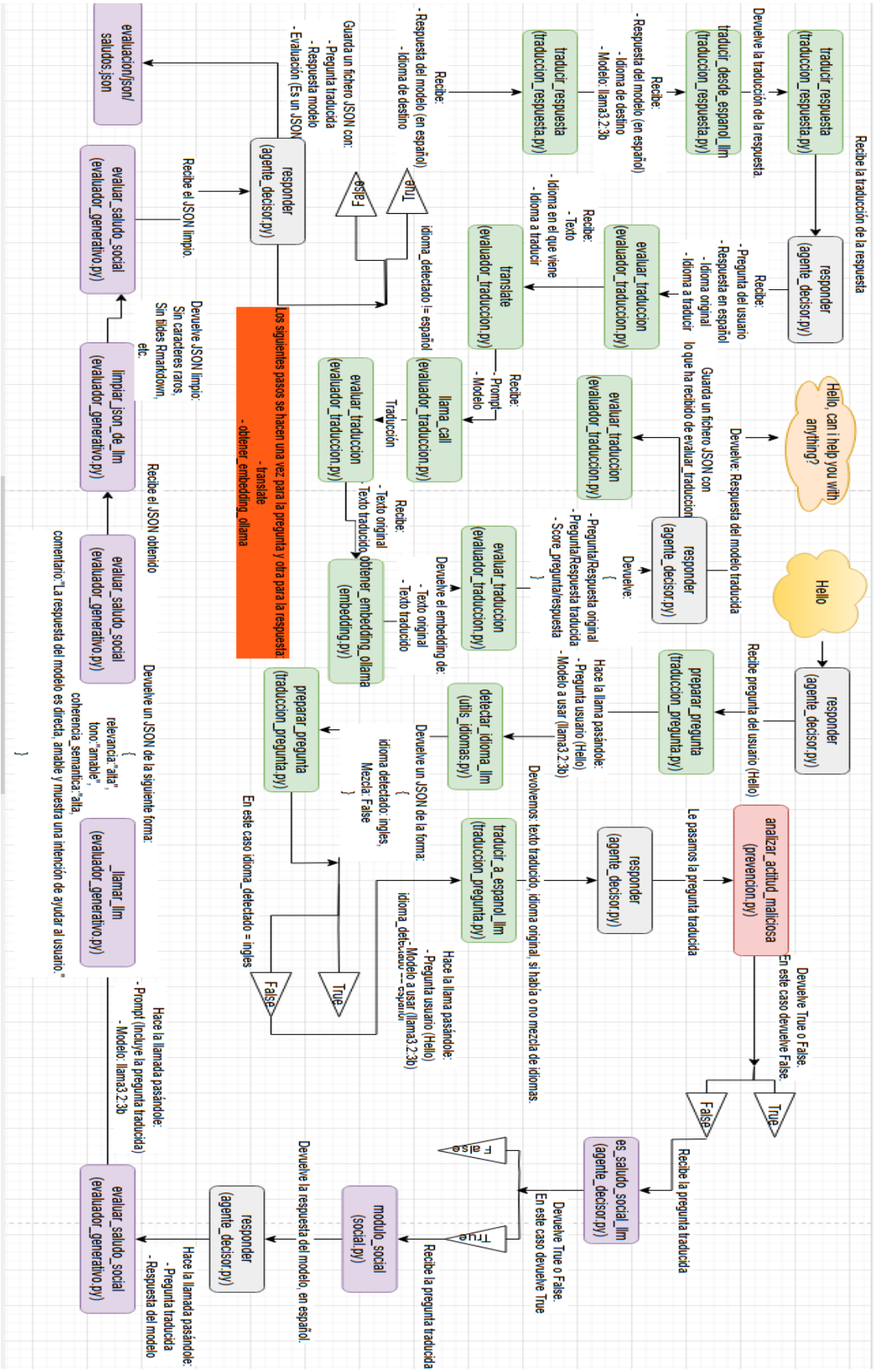


Figura 4.26: Diagrama creado con draw.io

Capítulo 5

Conclusión

Hemos podido ver a lo largo de este trabajo tanto los fundamentos teóricos que nos llevan a las soluciones prácticas como el propio desarrollo del sistema.

Además, hemos podido estudiar otros elementos como el tratamiento de grandes volúmenes de datos sin etiquetar. Así, la tipología de los datos se extendía desde el inicio de la carga de los documentos hasta la evaluación, generando diversos retos en cada uno de ellos.

Aunque se ha realizado un estudio de diversos elementos y factores, es necesario tener en cuenta como aún quedan muchas cosas por explorar y mejorar. Algunas de ellas podrían ser:

- Una primera mejora que podría realizarse, es el uso de un modelo con una mayor cantidad de parámetros. Es de esperar que debido a la capacidad de los grandes modelos de lenguaje de mejorar su actuación cuando el número de parámetros es mejor, que se obtenga una mejora en nuestro caso al usar un modelo con mayor número de parámetros. Además, sería interesante que al modificar el modelo por uno de tamaño mayor, se comprobase la actuación del mismo al unir varios módulos. Realizando así una comparativa entre una estructura más modularizada o por el contrario una estructurada más parecida a un end-to-end. Esto sería interesante estudiarlo, ya que es de esperar que los modelos realicen las tareas de una mejor manera mientras menos tareas se les encomienden, tal y como ocurre con los humanos. Sin embargo, sería interesante determinar si existe algún punto de rotura en los sistema, en el que a partir de un cierto número de tareas por parámetro el sistema empezase a generar comportamientos extraños. En nuestro caso, al pedirle al modelo realizar más de dos tareas a la vez ya conseguíamos resultados extraños, por lo que cabe esperar que con un mayor número de parámetros el número de tareas que sea capaz de soportar, antes de generar estos comportamientos, sea ligeramente mayor.

- También sería recomendable utilizar modelos especializados en traducción en el módulo de traducción, para así obtener resultados mejores al poder traducir un mayor número de idiomas.

- Asimismo, sería interesante usar un modelo distinto para evaluar que para generar la respuesta. Esto sería preferible a la forma en la que actuamos ahora, ya que los modelos tienen un sesgo en el que suelen preferir sus respuestas, por lo que cabe esperar que un modelo distinto fuese más crítico con las respuestas.

- Otra mejora que podría realizarse a este sistema, es el aumento de su base de conocimiento y de su capacidad a todos los documentos y a toda la información disponible en la Universidad de Sevilla. Esto podría plantearse de dos formas distintas con los consecuentes problemas que traería consigo cada una de ellas:

  - La primera forma de abordarlo, sería mediante el uso del mismo sistema para todo, es decir, lo único que cambiaría sería la base de datos, el resto se mantendría igual. Aquí tendría el problema de que aumentaría el tiempo para la recuperación de documentos al tener que realizar una búsqueda en un dominio mayor. Por tanto sería de esperar una mayor latencia en los resultados a menos que se consiguiese optimiza esta recuperación.
  - La segunda forma en la que se podría abordar sería mediante el uso de agentes especializados en devolución de cierta información. Esto tendría la ventaja de que como cada uno de ellos estaría especializado únicamente en un ámbito tendría un menor tiempo de respuesta al tener que realizar una búsqueda en un menor número de documentos. Sin embargo, esto traería consigo dos problemas principales:
    - El primero de ellos sería la clasificación de las páginas web en cada una de las categorías. Téngase en cuenta que al ser una gran cantidad de documentos esto supone un gran trabajo manual. Así, sería interesante realizar estudios sobre la mejor forma de automatizar esta partición.
    - Tendría que evaluarse un nuevo elemento que sería la capacidad de un agente principal para discernir entre que temas son importantes o relevantes para responder a la pregunta. Asimismo, dentro de este ámbito tendríamos el problema de que una pregunta puede pertenecer a varios ámbitos por lo que sería interesante también realizar un manejo de esta posibilidad.

- Además, podría realizarse una comparación, tanto en eficiencia como en otras métricas, con elementos comerciales usados habitualmente para la realización de sistemas RAG como langchaing, llamaIndex o chroma.

- Por último, es necesario notar que al término de la conversación con el usuario, todo el historial de dicha conversación sería borrado en nuestro sistema. Esto podría ser mejorado mediante la asociación de la IP del dispositivo o otra información del usuario al conectarse en la llamada, con una cuenta ficticia. En dicha cuenta se guardaría información personal que el usuario fuese proporcionando, como nombre, edad etc. Asimismo, también sería interesante permitir que el usuario pudiera logguearse con

sus credenciales teniendo ya de base estos datos recogidos.

Bibliografía

Pre-trained language models - sciencedirect topics.

Akmajian, A., Demers, R. A., Farmer, A. K., and Harnish, R. M. (2001). *Linguistics: An Introduction to Language and Communication*. MIT Press.

Almeida, F. and Xexéo, G. (2019). Word embeddings: A survey. *CoRR*, abs/1901.09069.

Ammann, L., Ott, S., Landolt, C. R., and Lehmann, M. P. (2025). Securing rag: A risk assessment and mitigation framework.

An, S., Ma, Z., Lin, Z., Zheng, N., Lou, J.-G., and Chen, W. (2024). Make your llm fully utilize the context. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C., editors, *Advances in Neural Information Processing Systems*, volume 37, pages 62160–62188. Curran Associates, Inc.

Arslan, M., Ghanem, H., Munawar, S., and Cruz, C. (2024). A survey on rag with llms. *Procedia Computer Science*, 246:3781–3790. 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems (KES 2024).

Baccouri, N. (2023). deep-translator: A flexible free and unlimited python tool to translate between different languages in a simple way.

Bansal, A. and Suddala, S. (2024). Enhancing generative ai capabilities through retrieval-augmented generation systems and llms. *Frontiers in Health Informatics*, 44:17765–17775.

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., and Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system.

Bhat, S. R., Rudat, M., Spiekermann, J., and Flores-Herr, N. (2025). Rethinking chunk size for long-document retrieval: A multi-dataset analysis.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. *CoRR*, abs/2005.14165.

Chatbots.org. 161 humanlike conversational ai synonyms.

Cichy, R. M. e. a. (2019). Deep neural networks as scientific models. *Trends in Cognitive Sciences*, 23(4):305–317.

de Saussure, F. (2011). *Course in General Linguistics*. Columbia University Press. Originally published as "Cours de linguistique générale" in 1916.

Devlin, J., Chang, M., Lee, K., and Toutanova, K. (2018). BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805.

Dong, Q., Li, L., Dai, D., Zheng, C., Ma, J., Li, R., Xia, H., Xu, J., Wu, Z., Liu, T., Chang, B., Sun, X., Li, L., and Sui, Z. (2024). A survey on in-context learning.

Galchonkov, O., Horchynskyi, O., Antoshchuk, S., and Nareznoy, V. (2025). Semantic text splitting method development for rag systems with controlled threshold and sliding window size. *Eastern-European Journal of Enterprise Technologies*, 2(2 (134)):47–57.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2024). Retrieval-augmented generation for large language models: A survey.

Genesis, J. (2025). Retrieval-augmented text generation: Methods, challenges, and applications. *Preprints*.

Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Wang, Y., Gao, W., Ni, L., and Guo, J. (2025). A survey on llm-as-a-judge.

Hadi, M. U., qasem al tashi, Qureshi, R., Shah, A., Muneer, A., Irfan, M., Zafar, A., Shaikh, M. B., Akhtar, N., and Jia Wu, S. M. (2023). Large language models: A comprehensive survey of its applications, challenges, limitations, and future prospects.

Henderson, M. Machine learning for dialog state tracking: A review.

Hochereiter, S. and Schmidhuber, J. (1997). Long short-term memory.

Howard, J. and Ruder, S. (2018). Fine-tuned language models for text classification. *CoRR*, abs/1801.06146.

Jurafsky, D., Stolcke, A., Coccaro, N., Taylor, P., Ess-Dykema, C. V., Ries, K., Shriberg, E., Martin, R., and Meteer, M. (1997). Dialogue act modeling for automatic tagging and recognition of conversational speech.

Kaelbling, L. P., Littman, M. L., and Moore, A. W. (1996). Reinforcement learning: A survey. *CoRR*, cs.AI/9605103.

Katashi Nagao, A. T. (1994). Social interaction: Multimodal conversation with social agents.

Lee, S., Shakir, A., Koenig, D., and Lipp, J. (2024). Open source strikes bread - new fluffy embedding model.

Lemon, O. (2011). Learning what to say and how to say it: Joint optimisation of spoken dialogue management and natural language generation. *Computer Speech and Language*, 25(2):210–221. Language and speech issues in the engineering of companionable dialogue systems.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *CoRR*, abs/2005.11401.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2023). Lost in the middle: How language models use long contexts.

Locci, S., Audrito, D., Livraga, G., Viviani, M., and Caro, L. D. (2025). Leveraging rag for privacy violation detection and explainability.

Louvan, S. and Magnini, B. (2020). Recent neural methods on slot filling and intent classification for task-oriented dialogue systems: A survey. *CoRR*, abs/2011.00564.

Luo, M., Xu, X., Dai, Z., Pasupat, P., Kazemi, M., Baral, C., Imbrasaite, V., and Zhao, V. Y. (2023). Dr.icl: Demonstration-retrieved in-context learning.

Marco Wiering, M. v. O. (2012). Reinforcement learning.

McTear, M. and Ashurkina, M. (2024). *Transforming Conversational AI.*

McTear, M. F. (2020). *Conversational AI: Dialogue systems, conversational agents, and chatbots.*

Mohammad, R. (2024). Advancing generative ai with rag: Enhancing relevance, creativity, and reliability in language models.

Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., and Mian, A. (2024). A comprehensive overview of large language models.

Oliveira, A. G. M. E. R. (2024). Dialogue flow visualization. Master's thesis, Universidad de Coimbra.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H. W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S. P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S. S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M.,

Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Łukasz Kaiser, Kamali, A., Kanitscheider, I., Keskar, N. S., Khan, T., Kilpatrick, L., Kim, J. W., Kim, C., Kim, Y., Kirchner, J. H., Kiros, J., Knight, M., Kokotajlo, D., Łukasz Kondraciuk, Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C. M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S. M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O'Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., de Avila Belbute Peres, F., Petrov, M., de Oliveira Pinto, H. P., Michael, Pokorny, Pokrass, M., Pong, V. H., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F. P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M. B., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J. F. C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J. J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., and Zoph, B. (2024). Gpt-4 technical report.

Pieraccini, R. and Huerta, J. M. (2008). *Where Do We Go from Here?*, pages 1–24. Springer Netherlands, Dordrecht.

Ruder, S. (2017). A survey of cross-lingual embedding models. *CoRR*, abs/1706.04902.

Sherstinsky, A. (2020). Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404:132306.

Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to sequence learning with neural networks. *CoRR*, abs/1409.3215.

Szymanski, A., Ziems, N., Eicher-Miller, H. A., Li, T. J.-J., Jiang, M., and Metoyer, R. A. (2025). Limitations of the llm-as-a-judge approach for evaluating llm outputs in expert knowledge tasks.

Too, E. C., Yujian, L., Njuki, S., and Yingchun, L. (2019). A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture*, 161:272–279. BigData and DSS in Agriculture.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *CoRR*, abs/1706.03762.

Verma, P. (2025). S2 chunking: A hybrid framework for document segmentation through integrated spatial and semantic analysis.

Vlasov, V., Mosig, J. E. M., and Nichol, A. (2019). Dialogue transformers. *CoRR*, abs/1910.00486.

Vásquez, M. A. C., Huerta, M. H. V., and Quispe, L. J. P. Procesamiento de lenguaje natural.

Wang, H., Li, J., Wu, H., Hovy, E., and Sun, Y. (2023). Pre-trained language models and their applications. *Engineering*, 25:51–65.

Wang, Y., Yao, Q., Kwok, J. T., and Ni, L. M. (2020). Generalizing from a few examples: A survey on few-shot learning. *ACM Comput. Surv.*, 53(3).

Wei, H., He, S., Xia, T., Liu, F., Wong, A., Lin, J., and Han, M. (2025). Systematic evaluation of llm-as-a-judge in llm alignment tasks: Explainable metrics and diverse prompt templates.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E. H., Le, Q., and Zhou, D. (2022). Chain of thought prompting elicits reasoning in large language models. *CoRR*, abs/2201.11903.

Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., and Liu, Z. (2025a). *Evaluation of Retrieval-Augmented Generation: A Survey*. Springer Nature Singapore, Singapore.

Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., and Liu, Z. (2025b). *Evaluation of Retrieval-Augmented Generation: A Survey*, page 102–120. Springer Nature Singapore.

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., and Wen, J.-R. (2025). A survey of large language models.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena.